

Modicon Modbus Driver

Filename	Modbus.dll
Manufacturer	Modicon
Devices	Any device compatible with Modbus protocol v1.1b
Protocol	Modbus v1.1b
Version	4.0.10
Last Update	03/21/2023
Platform	Win32 and Windows CE (Pocket PC ARM, HPC2000 ARM, HPC2000 X86, and HPC2000 MIPS)
Dependencies	None
Superblock Readings	Yes
Level	0

Introduction

This Driver implements the Modbus protocol, which allows an **Elipse Software** application to communicate with any slave device that implements this protocol in **ASCII**, **RTU**, or **TCP** modes.

This Driver always works as a master of a Modbus network. If users want to communicate with master devices, then **Elipse Software's Modbus Slave** Driver must be used, which can be downloaded at *company's website*.

This Driver, starting with version 2.0, was developed using **Elipse Software's IOKit** library. This library is responsible for implementing the physical layer access (**Serial**, **Ethernet**, **Modem**, or **RAS**). For more information about **IOKit** library configuration, please check topic **Documentation of I/O Interfaces**.

It is recommended to start by reading topic **Quick Configuration Guide** if the device is fully compliant with standard Modbus protocol, defined by the *Modbus Organization (modbus.org)*, and if users only want to read or write bits and registers, without using more advanced features of this Driver.

For a complete understanding of all this Driver's functionality, it is recommended to start reading, in this order, chapters **Adding a Driver to an Elipse Software Application** and **Configuration**.

To create large scale applications, it is also recommended to read topic **Optimization Tips**.

If users are not familiar with the protocol, please check topics **The Modbus Protocol**, **Recommended Websites**, **Supported Functions**, and **Special Functions**.

Quick Configuration Guide

This topic describes all necessary steps to configure this Driver for communication with devices compliant with the standard protocol defined by the *Modbus Organization*, considering the most common configuration options.

If a device is fully compliant with the standard protocol, and users only want to read or write registers or bits, the topics **Inserting a Driver**, **Configuring a Driver**, and **Configuring I/O Tags** are probably sufficient to configure this Driver.

Inserting a Driver

If using **E3** or **Elipse Power**, please check topic **Adding a Driver to an Elipse Software Application - E3 or Elipse Power**.

If using **Elipse SCADA**, please check topic **Adding a Driver to an Elipse Software Application - Elipse SCADA**.

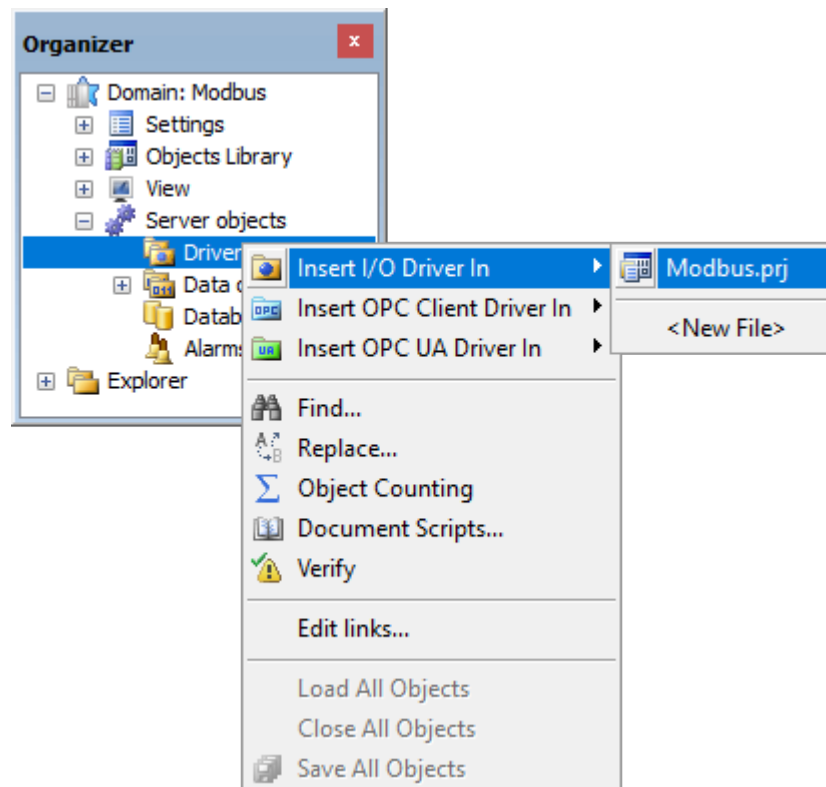
Then, check the **next step** on this Manual, which shows how to configure this Driver using its configuration window for the most common cases.

Adding a Driver to an Eclipse Software Application

This section describes how to add this Driver to **E3 or Elipse Power** and **Elipse SCADA** applications.

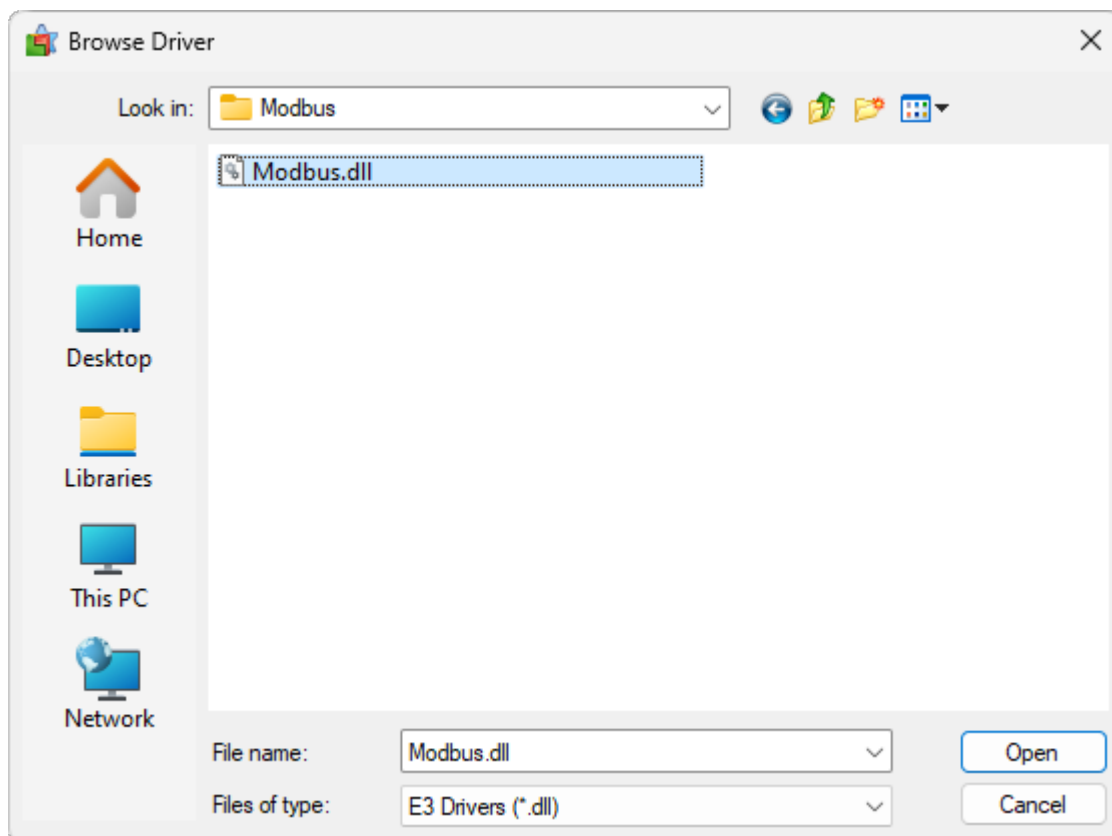
E3 or Elipse Power

On Organizer, right-click the **Server Objects - Drivers and OPC** item, select the **Insert I/O Driver in** option, and then select a project.



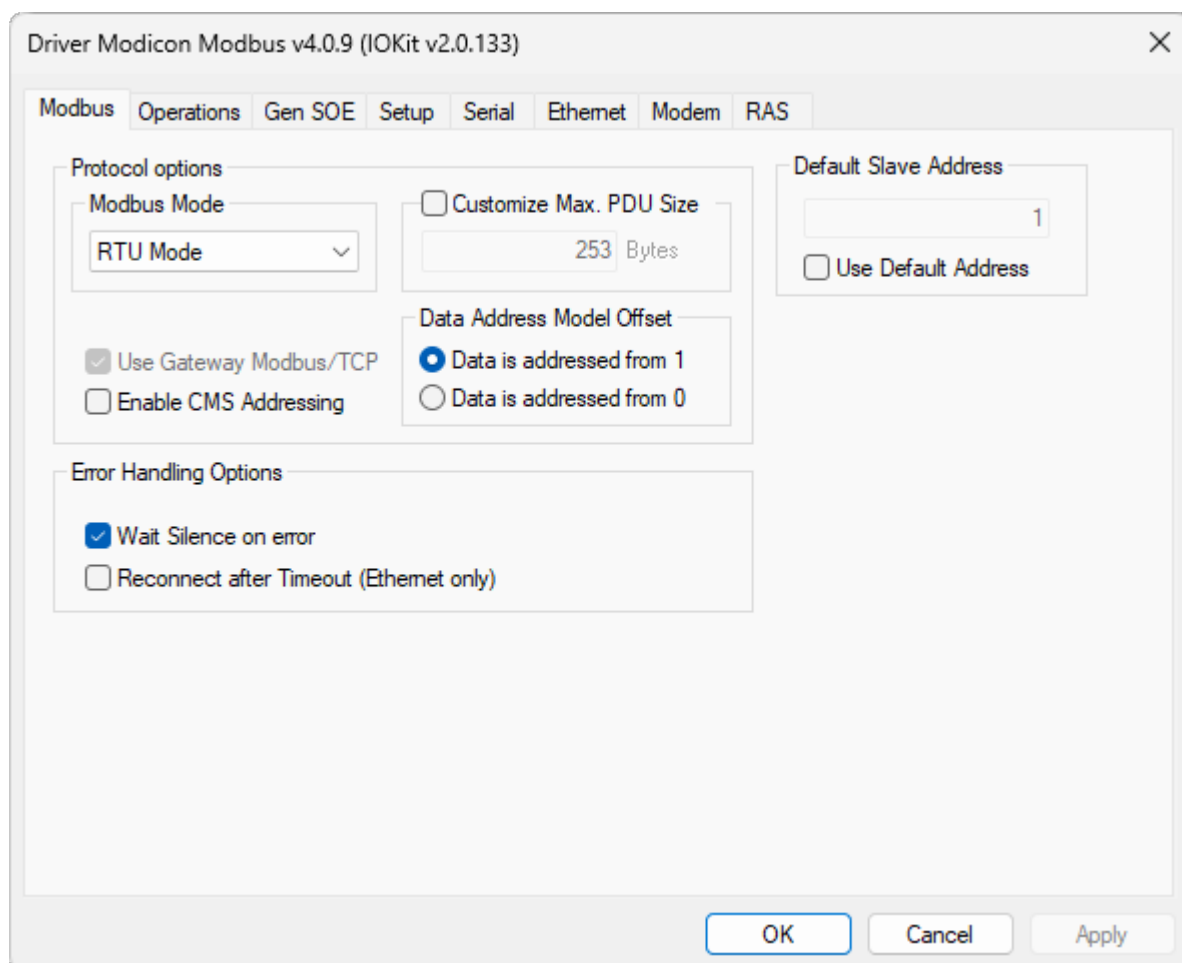
Adding a Driver to an E3 or Elipse Power application

On the window that opens, select a Driver (this file must be extracted to a folder on the computer in use) and click **Open**.



Selecting a Driver

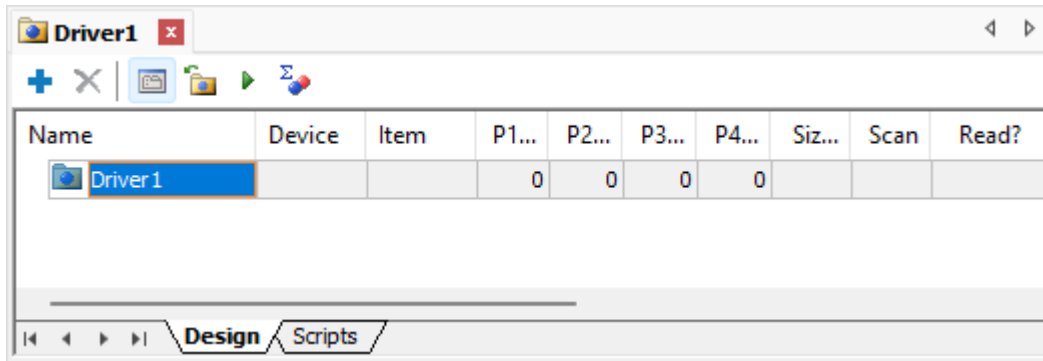
The configuration window of this Driver opens automatically, as shown on the next figure.



Configuration window of this Driver

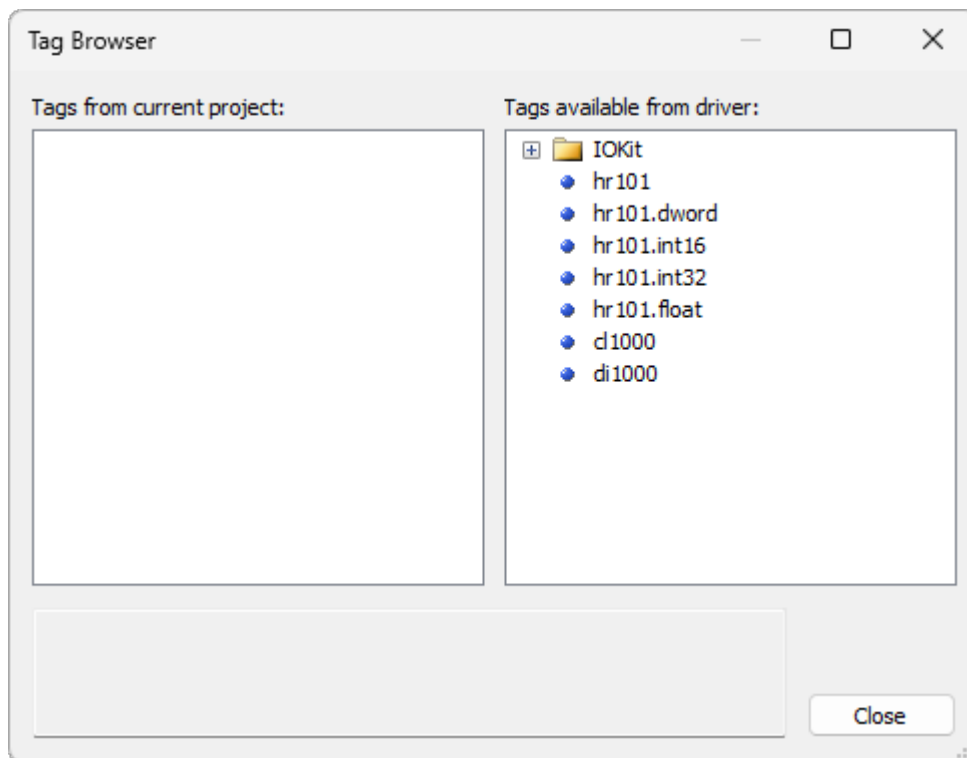
On the **second step** of topic **Quick Configuration Guide**, there is a procedure for a basic configuration of this Driver, for the most common usages. On topic **Properties** this configuration is presented in details.

This Driver's configuration window can also be opened, later, by clicking **Driver settings** , as shown on the next figure.



Driver settings option

After configuring this Driver's properties, click **OK** to open Tag Browser window, allowing to insert pre-defined Tags in the application, based on the most used settings. The next figure shows Tag Browser's window. To add Tags, drag them from the list **Tags available from driver** to the list **Current project tags**.

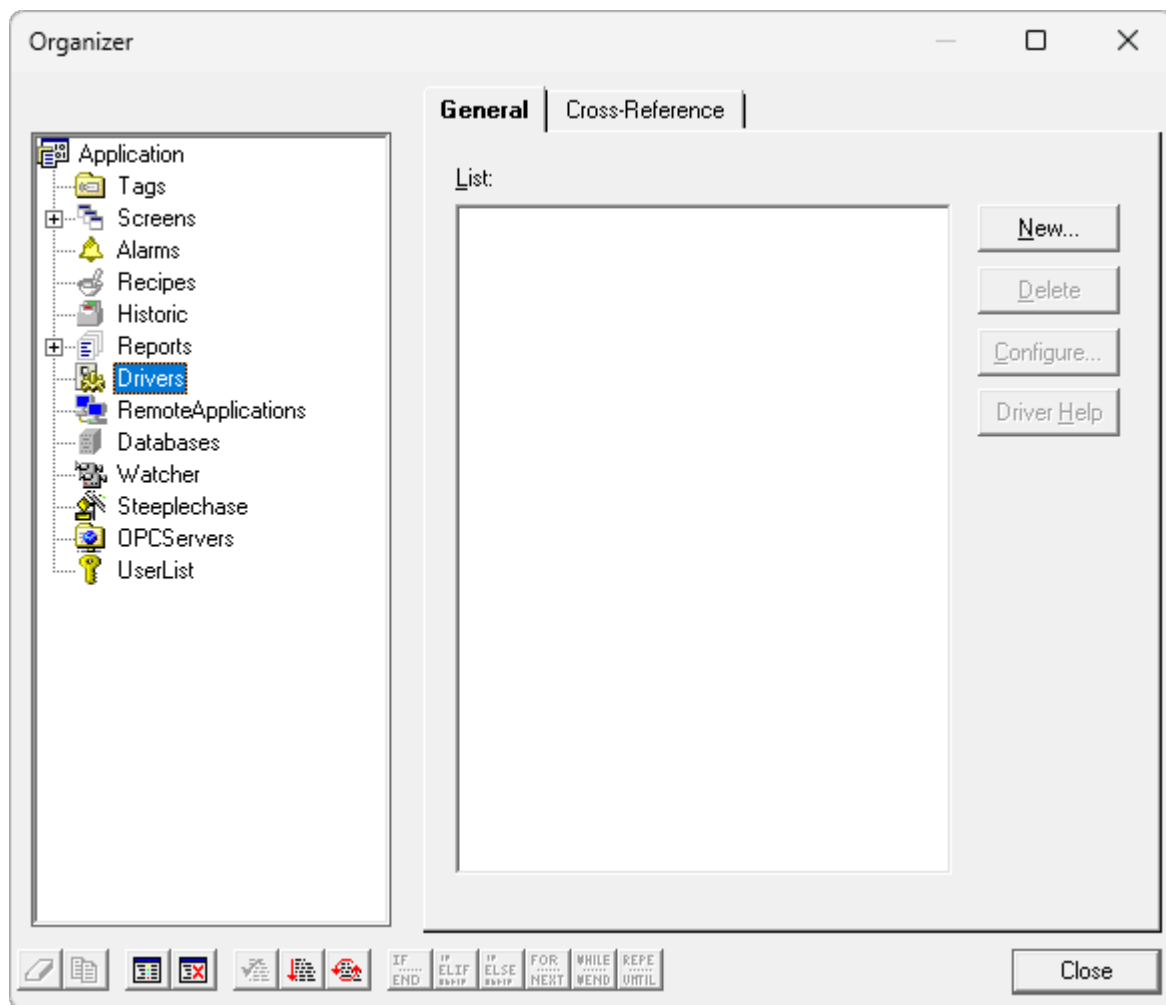


Tag Browser window

Tags available in Tag Browser are Tags **configured using Strings**, a new method that does not use the old concept of operations. Insert the most adequate ones to the application, editing their fields as needed. Tag Browser window can be opened later by clicking **Tag Browser** .

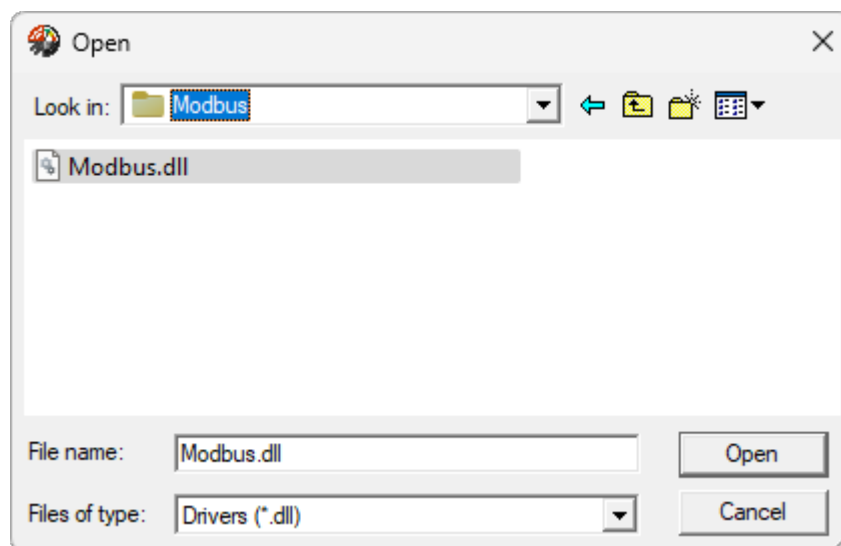
Elipse SCADA

On Organizer, select the **Drivers** item and click **New**.



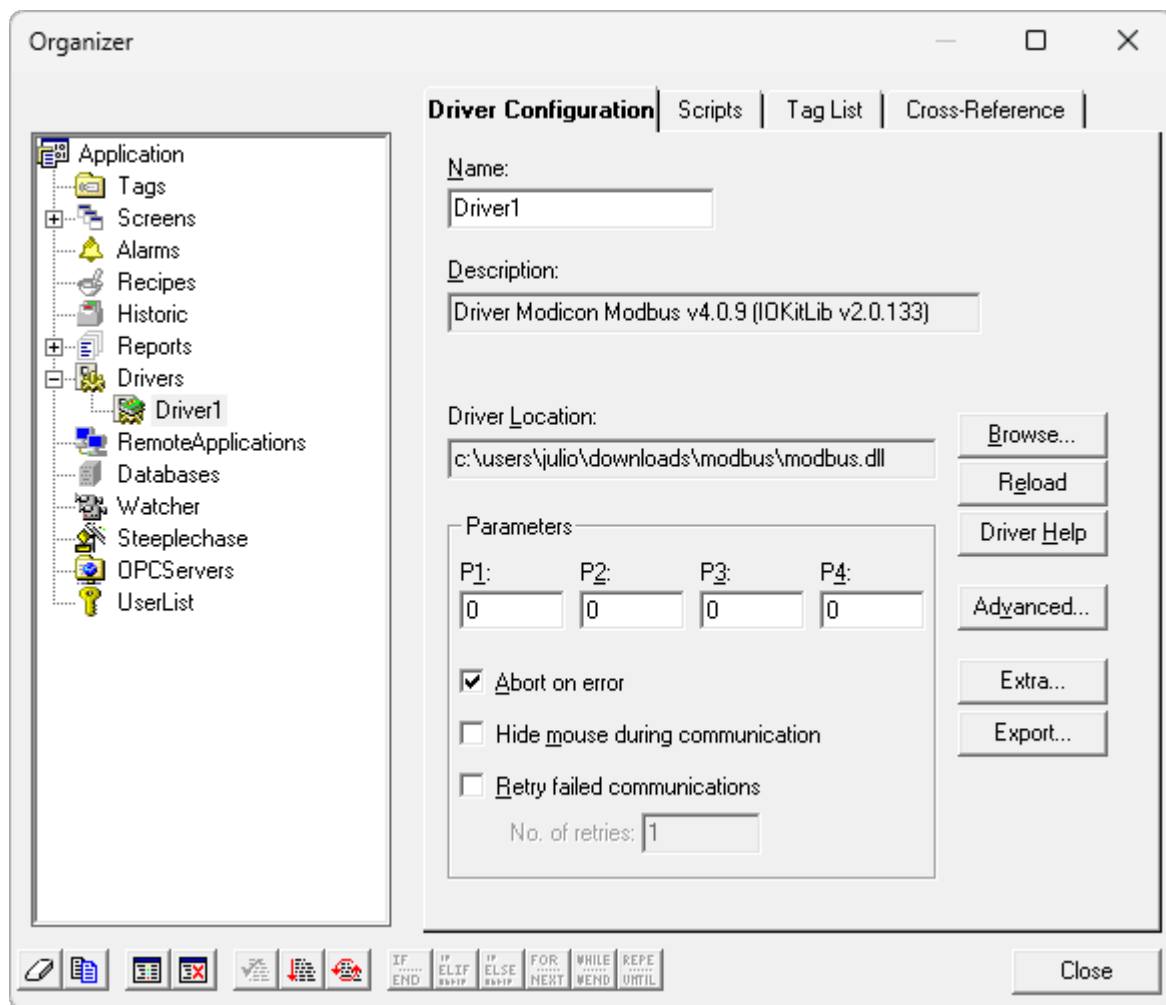
Adding a new Driver to Elipse SCADA

On the window that opens, select a Driver (this file must be extracted to a folder on the computer in use) and click **Open**.

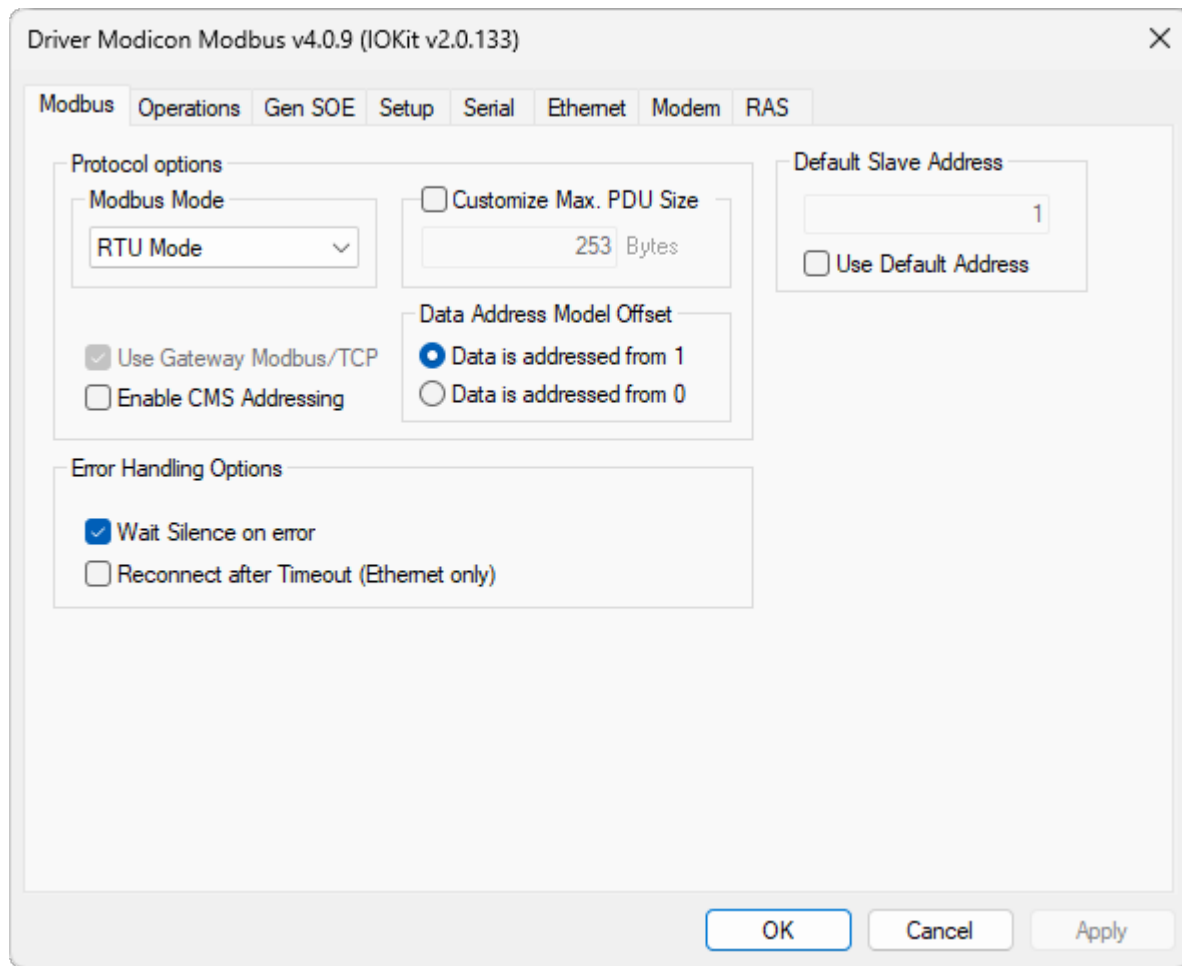


Selecting a Driver

This Driver is then added to the application.

**Driver in Organizer**

For this Driver to work correctly, users still need to configured it on its configuration window. To open this window, showed on the next figure, click **Extra**.



Configuration window of this Driver

The **second step** of topic **Quick Configuration Guide** shows how to configure this Driver for the most common usages, for devices complying to default Modbus protocol requirements. On topic **Properties** that configuration is described in details, including advanced configuration resources.

Configuring a Driver

After inserting this Driver in an application, users must open Driver's configuration window, as explained on topics **E3** or **Elipse Power** or **Elipse SCADA**. With this configuration window open, follow these steps:

1. Configure communication's physical layer:
 - a. On the **Setup** tab, select a physical layer (**Serial**, **Ethernet**, **Modem**, or **RAS**) to use when connecting with a device.
 - b. Configure the selected physical layer on its corresponding tab (**Serial**, **Ethernet**, **Modem**, or **RAS**).
 - c. For more information about configuring a physical layer, please check topic **Documentation of I/O Interfaces**.
2. On the **Modbus** tab, select the protocol mode (**RTU**, **ASCII**, or **TCP**) used by the device. As a general rule, users must select **RTU** or **ASC** (for most devices it is **RTU**) for **Serial** or **Modem** physical layers, or **TCP** for **Ethernet** or **RAS** physical layers. The other options usually can be kept with their default configurations. For more information about all options on this tab, please check topic **Modbus Tab**.

NOTE

For new applications, it is strongly recommended to avoid using **ModbusRTU** (RTU mode) encapsulated in an **Ethernet TCP/IP** layer. However, if by any reason, for legacy applications, users must use **ModbusRTU** encapsulated in **TCP/IP**, please do not forget to enable the **Reconnect after Timeout** option, described on topic **Modbus Tab**.

3. When creating applications for newer **Elipse Software** products, such as **E3**, **Elipse Power**, or **Elipse OPC Server**, users can use Tag configuration by **Strings (Device and Item fields)**. If this is the case, please go to the **next step** of this guide.
4. If users still need to use the old numerical configuration (*N/B* parameters), used by **Elipse SCADA**, it is important to check the **Operations** tab. There are default operations already pre-configured on this Driver. Operations are configurations of functions and data formats that later are referenced by application's Tags. These default operations, already available when this Driver is loaded for the first time, are the most common ones. Evaluate all reading and writing functions and the data types used for each operation, and check which ones are needed for the application. If these pre-defined operations do not fit the application needs, users must edit them or even create new operations. If this is the case, please check topic **Operations Tab**. The next table lists all pre-defined operations.

Pre-defined operations

OPERATION	READING	WRITING	DATA TYPE	PURPOSE
1	3: Read Holding Registers	16: Write Multiple Registers	Word	Reading and writing unsigned 16-bit integers
2	3: Read Holding Registers	16: Write Multiple Registers	DWord	Reading and writing unsigned 32-bit integers
3	3: Read Holding Registers	16: Write Multiple Registers	Int16	Reading and writing signed 16-bit integers
4	3: Read Holding Registers	16: Write Multiple Registers	Int32	Reading and writing signed 32-bit integers
5	3: Read Holding Registers	16: Write Multiple Registers	Float	Reading 32-bit floating point values
6	3: Read Multiple Coils	15: Write Multiple Coils	Bit	Reading and writing bits
7	2: Read Discrete Inputs	None	Bit	Reading bits from a data block of Discrete Inputs

NOTE

These default operations are configured assuming that a device complies with Modbus' default byte order, *big endian*, in which the most significant bytes come first. If a device does not comply with that standard, please check topic **Operations Tab** for more information about configuring operations for different byte orders.

For more information about configuring this Driver, please check topic **Configuration**.

The **next step** demonstrates how to configure I/O Tags based on pre-defined operations.

Configuring I/O Tags

This section describes how to configure I/O Tags in **E3**, **Elipse Power** and in old **Elipse SCADA** for the most common usages.

Tag Configuration in E3 and in Elipse Power

Configuring I/O Tags in **E3** and in **Elipse Power** can be performed using the new **String configuration** method or using the old **numerical configuration** method, compatible with **Elipse SCADA**. For new projects, it is recommended to use **String** configuration, which improves application's legibility and maintenance.

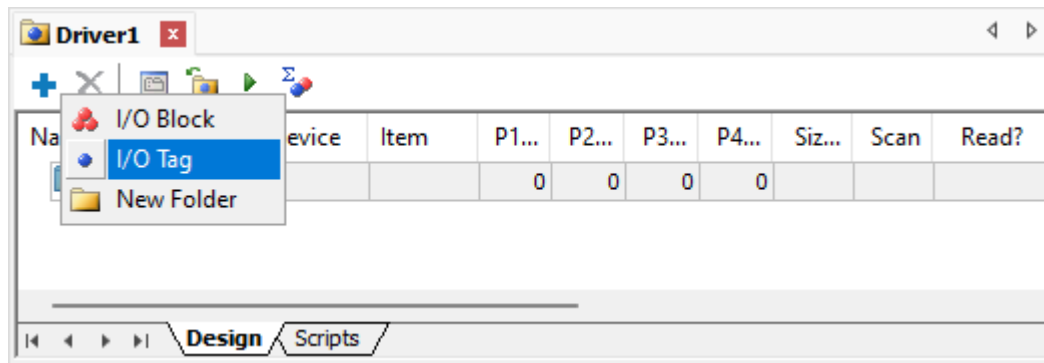
Next there is a description of the recommended procedures for **String** configuration and also for the old numerical configuration, if it is needed for legacy applications.

String Configuration

To add **String**-configured Tags, users have an option to import Tag Browser's pre-defined models, as explained on topic **Adding a Driver to an Elipse Software Application**. To do so, users must keep the **Show Operations in Tag Browser** option disabled on the **Operations** tab and then open Tag Browser by clicking **Tag Browser** .

To add a new Tag to an application without using Tag Browser, follow these steps:

1. On Organizer, double-click this Driver, select the **Design** tab, click **Add** , and then select the **I/O Tag** item, according to the next figure.



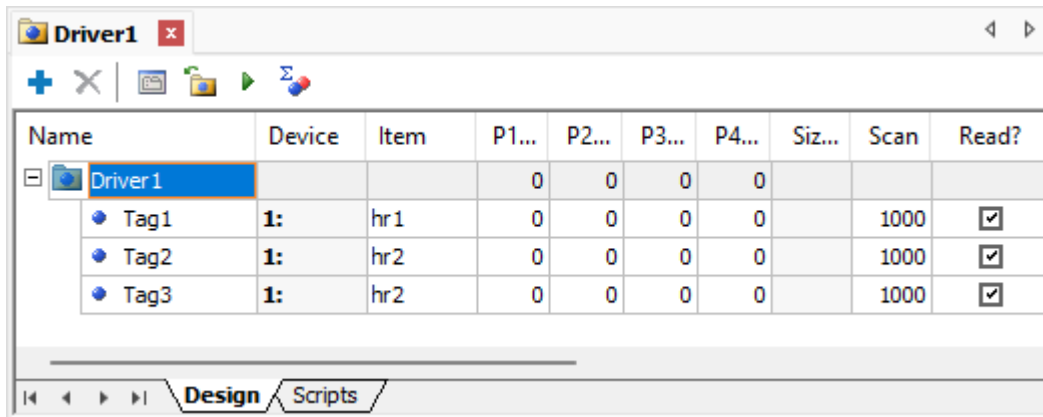
Adding a new I/O Tag

2. On the **Adding IOTag** window, configure the **Quantity** field with value 1 (one) and specify a name for this Tag in the **Name** field. Click **OK** to create a new Tag.
3. On the **Device** column, type the numerical value of device's *Slave Id* to communicate, followed by a colon, such as "1:" for a *Slave Id* equal to 1 (one). Notice that, in an **Ethernet TCP/IP** layer, this value is sometimes ignored, and only the IP address is used and the TCP/IP port configured on the **Ethernet** tab, which must be declared on device's documentation.
4. On the **Item** column, specify a mnemonic for the **address space** (a set of reading and writing Modbus functions) followed by the register's or bit's address. For *Holding Registers*, the address space is "hr" or "shr", and this last one does not allow writing in blocks, because it uses the writing function **06** (*Write Single Register*), while the "hr" address space uses the writing function **16** (*Write Multiple Registers*). Both use the reading function **03** (*Read Holding Registers*). For *Coils* use "cl" or "scl". Again, the difference is that the last one, which uses function **05** (*Force Single Coil*), does not write to blocks. Next, there are some examples of configurations for the **Item** column.
 - a. Reading or writing *Holding Register* 150 using functions **03** and **16** (writing multiple registers): **Item** must be equal to "hr150".
 - b. Reading or writing *Holding Register* 150 using functions **03** and **06** (writing simple registers): **Item** must be equal to "shr150".
 - c. Reading or writing a *Coil* with address FFF0h (65520) using functions **01** and **15** (writing multiple bits): **Item** must be equal to "cl65520" or "cl&hFFF0" (prefix "&h" can be used to provide addresses in hexadecimal format).
 - d. Reading or writing a *Coil* with address FFF0h (65520) using functions **01** and **05** (writing simple bits, one by one): **Item** must be equal to "scl65520" or "scl&hFFF0" (prefix "&h" can be used to provide addresses in hexadecimal format).

5. For more information about other features of **String** configuration, such as other Modbus functions, special functions, and different data types, please check topic **String Configuration**.
6. Tag addressing must correspond to the Modbus address map on the device, which must be declared on manufacturer's documentation. When in doubt, please check topic **Addressing Tips (Modbus Convention)**.

Prefer simple Tags, called PLC Tags in **Elipse SCADA**, rather than Block Tags, keeping the Superblock feature enabled, that is, the **EnableReadGrouping** property set to True, leaving group optimization to the application and to this Driver. For more details, please check topic **Superblock Reading (Grouping)**.

As an example, the next figure shows Tags configured by **Strings**.



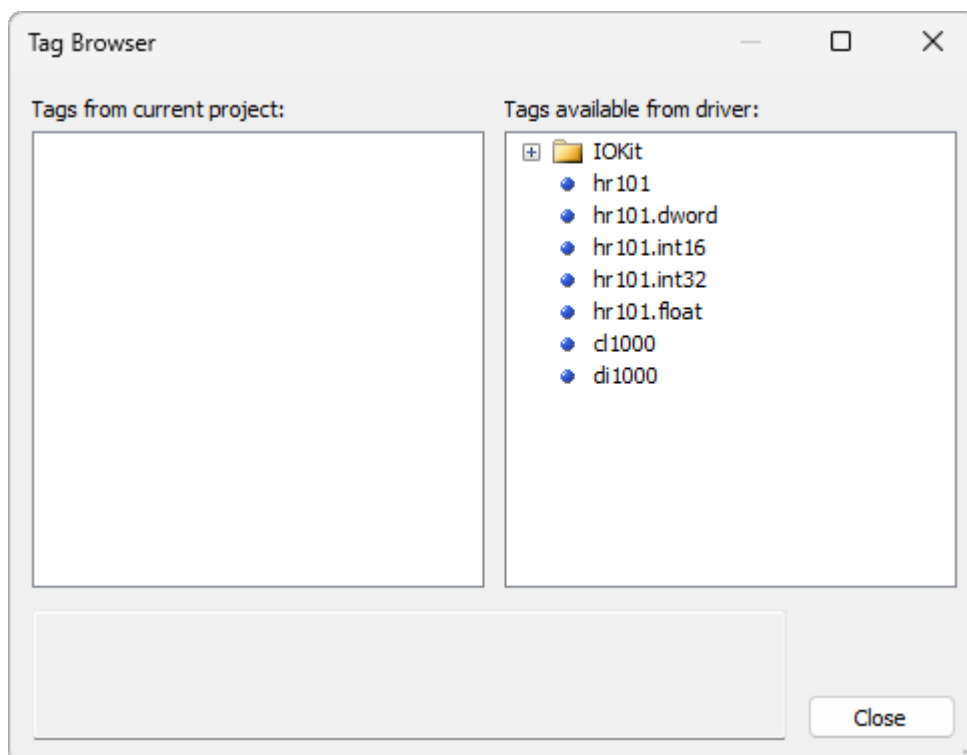
Name	Device	Item	P1...	P2...	P3...	P4...	Siz...	Scan	Read?
Driver1			0	0	0	0			
Tag1	1:	hr1	0	0	0	0		1000	☒
Tag2	1:	hr2	0	0	0	0		1000	☒
Tag3	1:	hr2	0	0	0	0		1000	☒

Example of Tags configured by Strings

Numerical Configuration

For **E3** or **Elipse Power**, users can use Tag Browser to create Tags with pre-defined operations, **numerically configured**. To do so, select the **Show Operations in Tag Browser** option on the **Operations** tab.

Tag Browser's window, shown on the next figure, is opened when clicking **OK** on this Driver's configuration window.



Tag Browser for numerical configuration of Tags

To add a new Tag to an application, follow these steps:

1. Drag Tags from the list **Tags available from driver** to the list **Current project tags**, as described on topic **E3 or Elipse Power**. For many devices, operation **1**, the most common, should be sufficient, so just drag **Op1<word>** Tag to the list of project Tags. Notice that, assuming that several Tags are needed with the same operation, which is a normal situation, users can drag the same Tag several times. Notice that an application adds sequential numbers to the default name. Users can also add a Tag from each operation and then later create other copies in Organizer.
2. Close Tag Browser and configure the *N4/B4* parameter of each Tag with the register or bit address to read or write, according to device's register map. This address map must be described on manufacturer's documentation. When in doubt, please check topic **Addressing Tips (Modbus Convention)**.
3. Also configure the *N1/B1* parameter of each Tag with device's address (*Slave Id*) to access in each case. This parameter is usually configured on the device and, to determine it, please check manufacturer's documentation or technical support when in doubt.
4. Rename all Tags, if needed, with a more significant name for the application.

Configure simple Tags, called PLC Tags in old **Elipse SCADA**, rather than Block Tags, keeping the Superblock feature enabled, that is, the **EnableReadGrouping** property set to True, leaving group optimization to the application and to this Driver. For more information, please check topic **Superblock Reading (Grouping)**.

Tag Configuration in Elipse SCADA

Elipse SCADA does not support Tag Browser, thus it is necessary to configure I/O Tags manually. Users must create Tags with the following configuration:

- **N1/B1**: Device address (Slave Id)
- **N2/B2**: Operation Code
- **N3/B3**: Not used, leave it 0 (zero)
- **N4/B4**: Address of a Modbus register or bit

Notice that, for this Driver, simple Tag's *N* parameters have the same meaning as Block Tag's *B* parameters, so they are described together.

When in doubt about which value to configure in the *N4/B4* parameter, please check topic **Addressing Tips (Modbus Convention)**.

As **Elipse SCADA** does not support Superblocks, it is recommended to create Block Tags, grouping adjacent or close registers, to read the maximum number of registers in the smallest number of protocol requests.

Also notice that, once a device supports **default protocol limits for the size of a communication frame**, due to **Automatic Block Partition** feature, there is no need to worry about exceeding the maximum block size supported by this protocol, because this Driver already creates the appropriate subdivisions during communication.

Final Considerations

If users only want to use this Driver's default operations, and if a device complies with the standard protocol defined by Modbus Organization, the steps presented in this Quick Configuration Guide should be sufficient to configure this Driver.

For larger applications, it is recommended to check topic **Optimization Tips**.

More details on I/O Tag configuration are provided on topic **Configuring an I/O Tag**.

The Modbus Protocol

The **Modbus Protocol** was initially developed by Modicon in 1979, and today it is an open standard, maintained by the Modbus Organization (modbus.org), and implemented by hundreds of manufacturers in thousands of devices. Schneider Electric, current controller of Modicon, transferred protocol rights to the Modbus Organization in April 2004, and committed to keep Modbus as an open protocol. Its specification can be downloaded for free at Organization's website, www.modbus.org, and protocol's usage is free of licensing fees.

This protocol is based on command and response messages, positioned at layer 7 of the OSI model, the application layer, which provides client and server communication among devices connected to different types of networks. It offers services with functions defined by an eight-bit code. The existing categories of function codes are the following:

- **Public function codes:** Protocol's well-defined functions, guaranteed to be unique, validated by the Modbus community, and publicly documented in MB IETF RFC. They can assume values ranging from **1 to 64**, from **73 to 99**, and from **111 to 127**
- **User-defined function codes:** Non-standard functions, which do not need Modbus.org approval, without any guarantee of being unique, and freely implementable. They can assume values ranging from **65 to 72** and from **100 to 110**
- **Reserved function codes:** Codes with values inside the range of public functions, currently used by some manufacturers for legacy products, and not publicly available anymore. Examples are **9, 10, 13, 14, 41, 42, 90, 91, 125, 126**, and **127** codes. For more information, please check **Annex A** of protocol's specification, version 1.1b, which is available at *protocol's official website*

This Driver currently implements 11 of all 19 public functions defined on the current version of protocol's specification, 1.1b, as well as some specific manufacturer's functions or related to this Driver's specific features, known as **Special Functions**. All public functions implemented are described on topic **Supported Functions**. The following protocol's public functions are not yet supported:

- **Function 08:** Diagnostic
- **Function 11:** Get Com event counter
- **Function 12:** Get Com Event Log
- **Function 17:** Report Slave ID
- **Function 22:** Mask Write Register
- **Function 23:** Read/Write Multiple Registers
- **Function 24:** Read FIFO queue
- **Function 43:** Read Device Identification

If users want to implement one of these functions, please contact *Elipse Software's sales department*.

Recommended Websites

Modicon Modbus Driver is available for download, at no cost, at **Elipse Software's Drivers download** area.

More information about the Modbus protocol can be found at www.modbus.org, protocol's official website.

Elipse Modbus Simulator is available for download, at no cost, at **Elipse Software's E3 download** area.

Modsim Modbus Slave Simulator, probably the best known in its category, can be purchased at www.wintech.com/html/modsim32.htm. This software emulates a device, allowing communication with this Driver.

There is also a free alternative called **Free Modbus PLC Simulator**, available for download at www.plcsimulator.org.

Other alternatives for simulators and software tools related to this protocol can be found at [protocol's official website](#).

Supported Functions

Modbus protocol functions supported by this Driver are described next.

Reading functions

- **01: Bit** Reading (*Read Coil Status - 0x*)
- **02: Bit** Reading (*Read Input Status - 1x*)
- **03: Word** Reading (*Read Holding Registers - 4x*)
- **04: Word** Reading (*Read Input Registers - 3x*)
- **07:** Status Reading (*Read Exception Status*)
- **20:** File Record Reading (*Read File Register - 6x*)

Writing functions

- **05: Bit** Writing (*Force Single Coil - 0x*)
- **06:** Simple **Word** Writing (*Preset Single Register - 4x*)
- **15: Bit** Writing (*Force Multiple Coils - 0x*)
- **16: Word** Writing (*Preset Multiple Registers - 4x*)
- **21:** File Record Writing (*Write File Register - 6x*)

Detailed information about each one of these functions can be found on Modbus protocol's specification, available for download at [Modbus Organization's website](#).

In addition to protocol's standard functions, as already stated, this Driver also implements special functions, not defined by the protocol, usually related to mass memory readings. A list of all supported special functions can be checked on topic **Special Functions**. A complete configuration of this Driver is described on topic **Configuration**.

If users want to add support to a new function in this Driver, please contact *Eclipse Software's sales department*.

Special Functions

Special reading and writing functions are this Driver's functions not defined by the standard Modbus protocol. They were developed to meet specific features of certain devices, or else to provide, in a standardized way by this Driver, features not available in the standard protocol. This Driver, in its current version, includes the special functions described next.

Reading Functions

- **65 03:** Mass Memory Reading (*ABB MGE 144*), described in details on topic **Reading Mass Memory Registers from ABB MGE 144 Meters**
- **GE SOE:** Event Reading (*GE PAC RX7 Systems*), described in details on topic **Reading an Event Buffer from GE PAC RX7 Controllers**

- **SP SOE**: Event Reading (*Schneider Electric SEPAM series Relays*), described in details on topic **Reading Events from Schneider Electric Relays from SEPAM 20, 40, and 80 Series**
- **GenSOE**: SOE reading with a generic algorithm, implemented by a resident software in the slave device (PLC), described in details on topic **Eclipse Software's Generic SOE Reading Algorithm**

Writing Functions

- **65 01**: Restarts, that is, performs a reset operation, a power meter (*ABB MGE 144*). This command is sent as a Tag's simple writing command (**Write**). Tag's **Value** field is ignored by this Driver and can be kept in 0 (zero). For more information, please check that device's manual
- **65 02**: Zeroes the maximum and minimum memory (*ABB MGE 144*). This command is sent as a Tag's simple writing command (**Write**). Tag's **Value** field is ignored by this Driver and can be kept in 0 (zero). For more information, please check that device's manual

Notice that this Driver's special functions, except for the writing function **65 01**, are directly or indirectly related to mass memory reading of registers of their respective devices. For more information, please check topic **Mass Memory Reading**. For a description on how to configure operations and Tags using these functions, please check topic **Configuration**.

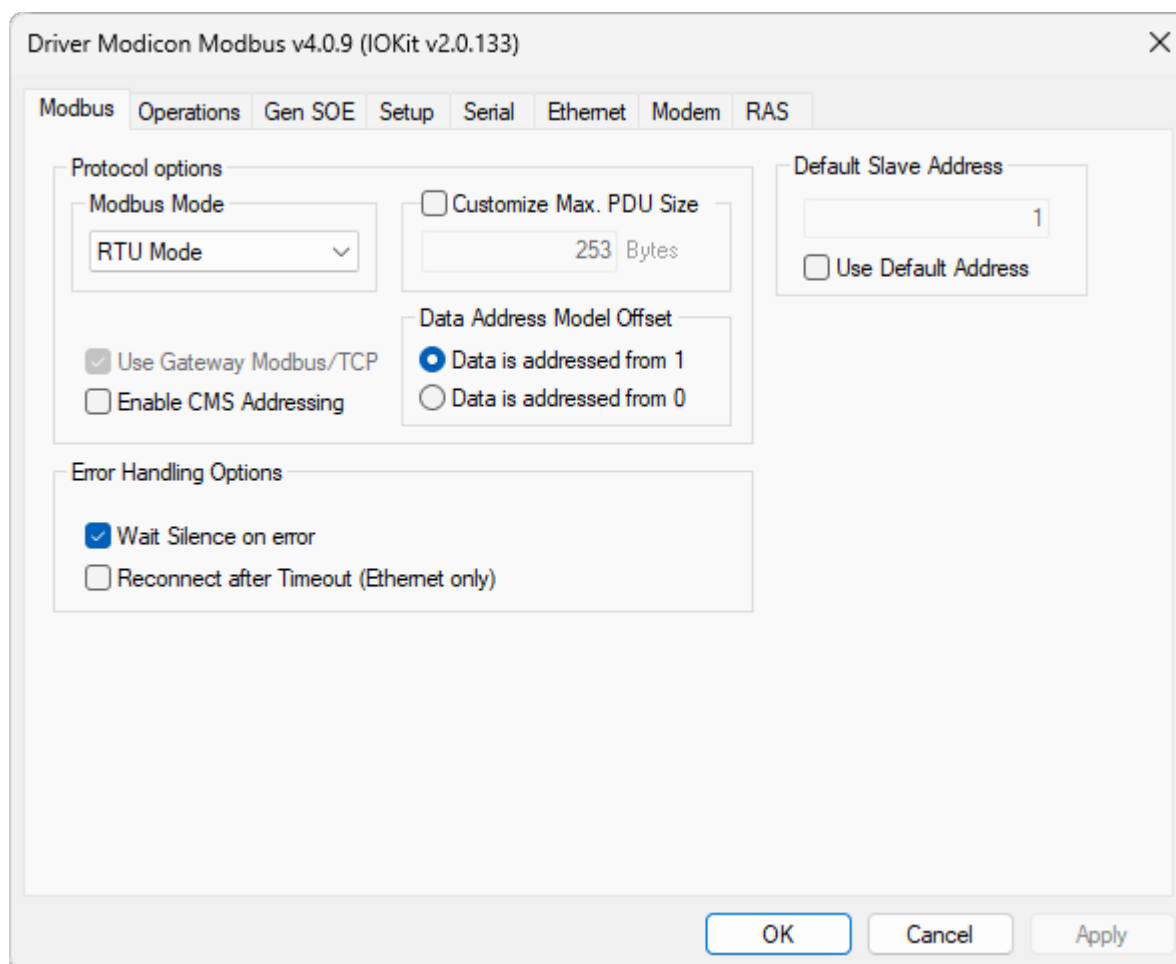
Configuration

This section describes how to configure this Driver. The topics **Properties** , **Configuring Tags**, and **Mass Memory Reading** are discussed.

Properties

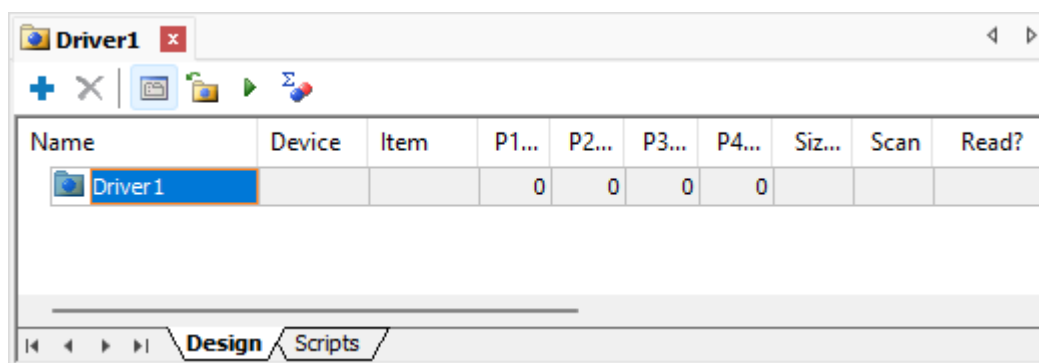
This Driver's properties can be configured at design time or at run time. A runtime configuration is also called an **Offline Mode Configuration**, and it is described on a specific topic.

At design time, this Driver can be configured using its configuration window, displayed on the next figure.



Configuration window of this Driver

To open this Driver's configuration window in **E3** or **Elipse Power**, double-click the Driver object on Organizer and click **Driver settings** , as shown on the next figure.



Driver settings option

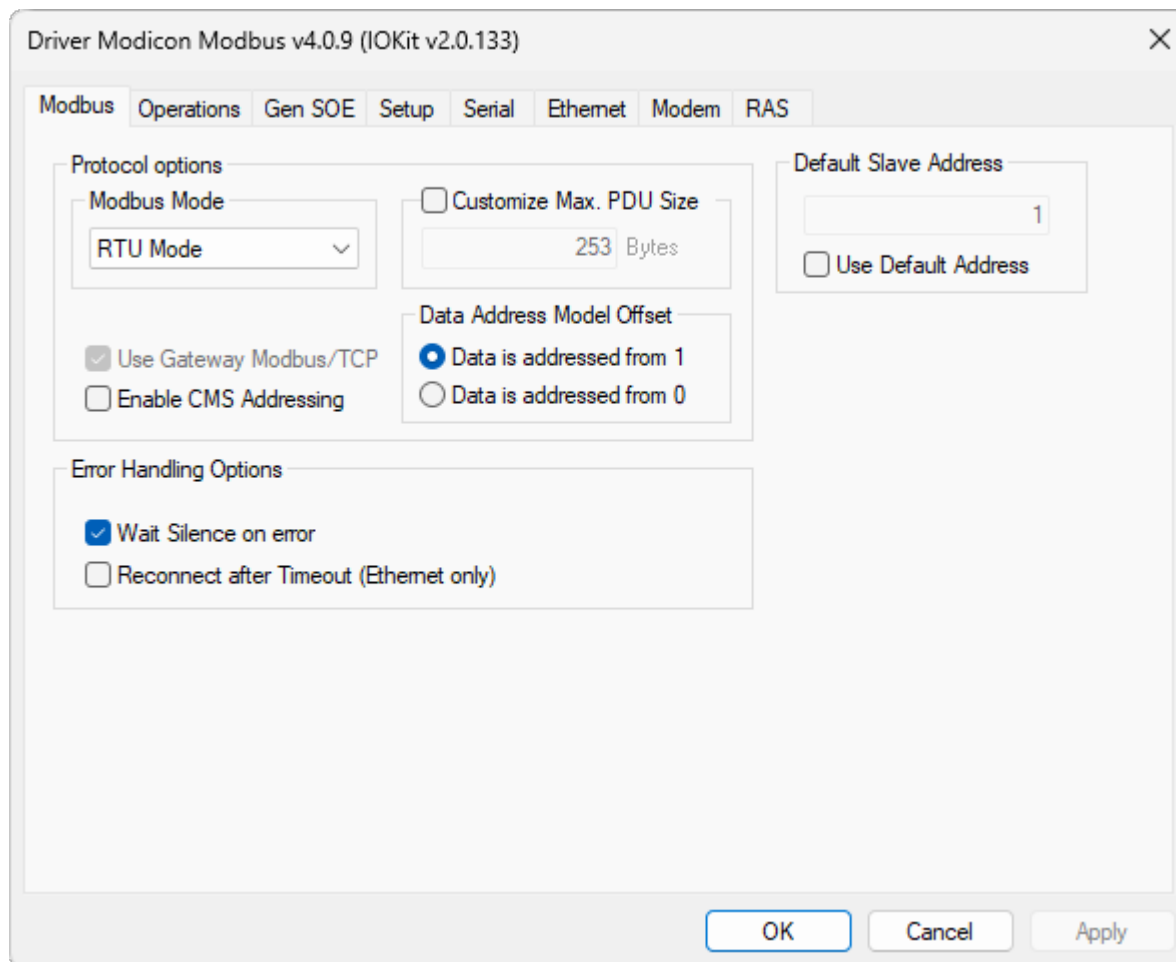
In **Elipse SCADA**, on the other hand, this Driver's configuration window can be opened by clicking **Extra**, on application's Organizer.

This configuration window is divided into several tabs, some of them for **IOKit** library configuration, and others are Driver-specific. For this Driver, the **Modbus**, **Operations**, and **Gen SOE** tabs are specific. All other tabs are for **IOKit** library configuration, and they are not described on this Manual. For more information about **IOKit** library configuration, please check topic **Documentation of I/O Interfaces**.

The topics **Modbus Tab**, **Operations Tab**, **Gen SOE Tab**, and **Offline Mode Configuration** describe this Driver's specific tabs and also its runtime configuration, in **Offline Mode**, using scripts.

Modbus Tab

The **Modbus** tab allows configuring parameters for this Driver and for the protocol, as shown on the next figure.



Modbus tab on this Driver's configuration window

The next sections describe all configuration options available on this tab.

Protocol Options

This group of options gathers options referring to variations on protocol's standards, according to the next table.

Available protocol options on the Modbus tab

OPTION	DESCRIPTION
Modbus Mode	On this option, users can select a protocol mode to use. Protocol modes are variations defined by the standard for a better adaptation to different physical layers, such as Serial, Ethernet TCP/IP, or RAS, among others. The available options are the following: • RTU mode: Default mode for use in serial communications. Includes a 16-bit CRC • ASCII mode: Also used in serial communications, it is used in simpler devices, which do not support RTU mode requirements. It uses ASCII characters for transmission, where each byte contains two ASCII characters, one per nibble, thus it is less efficient than RTU mode, and rarely

OPTION	DESCRIPTION
	found on the market. Uses LRC (<i>Longitudinal Redundancy Checking</i>) for error checking • ModbusTCP mode: Used for communication in TCP/IP mode. Includes a field for transaction check and does not have an error check system. The transaction field allows discarding delayed responses, thus avoiding that this Driver assumes as a valid response for the current command the response frames from previous commands. This situation may occur if previous modes are encapsulated in TCP/IP
Customize Max. PDU Size	If enabled, this option allows defining a maximum custom size for PDU (<i>Protocol Data Unit</i>). A PDU is a part of the protocol that does not vary between modes ModbusTCP, ASCII, and RTU and contains a data area. The number of data bytes supported in each communication is given by this value minus the header bytes, which depend on the Modbus function used. If disabled, the maximum considered size is the default value defined by the Modbus protocol version 1.1b, with 253 bytes. This is the recommended option for most devices
Use Gateway Modbus/TCP	In Ethernet mode, via Modbus TCP, this option indicates the presence of a <i>Modbus Gateway</i> to access a Serial RS-485 network, reserving the <i>Slave ID</i> with a value of 0 (zero) for Broadcast mode. If users want to directly communicate with a device in Ethernet mode, disable this option to use a <i>Slave ID</i> with value 0 (zero)
Enable CMS Addressing	This option must be only used in devices that support the TeleBUS protocol. If enabled, this Driver accepts a 16-bit Word as its slave address, that is, users can define values above 255 and below 65535 as a slave address. In this case, a slave address is then defined in the protocol by three bytes. In addition, the Default Slave Address option stops working
Data Address Model Offset	This option enables or disables the default protocol's data offset, by one unit. Available options are: • Data is addressed from 1 (default): The address provided (address of the Item field in String configuration or the <i>N4/B4</i> parameter in numerical configuration) is decremented by 1 (one) before sending it to a device. This offset is part of protocol's specification, therefore this is the default option • Data is addressed from 0: The user-provided address is used in protocol requests, without changes As a general rule, select the first option if device's register mapping starts at 1 (one) and the second option if it starts at 0 (zero). Also check if the manufacturer uses additional offsets from the old Modbus Convention. For more information, please check the next section.

TIP

Avoid using protocol's **RTU** mode encapsulated in **Ethernet TCP/IP** layer. If there is a need to encapsulate serial communication for devices using **Modbus RTU** in **TCP/IP**, there are gateways available on the market that not only encapsulate serial communication in **Ethernet TCP/IP**, but also convert **Modbus RTU** to **Modbus TCP**. As a last option, if using **Modbus RTU** in an **Ethernet TCP/IP** layer is inevitable, remember to enable the **Reconnect after Timeout** option, described on the next table.

Data Address Model Offset

This configuration option, described on the **previous table**, is a source of frequent doubts when addressing I/O Tags, because there are many variations in how it is implemented by manufacturers. Next there is more information about this addressing.

In protocol's standard data model, four data blocks are defined, or address spaces: *Discrete Inputs*, *Coils*, *Input Registers*, and *Holding Registers*. In each one of these blocks, data elements are addressed starting at 1 (one). On the other hand, the communication frame's specification defines a PDU with addresses that range from 0 (zero) to 65535. The relationship between the address provided by the PDU and the address of data elements, therefore, has an offset of 1 (one), that is, if in a request's PDU there is an address 0 (zero), the data element to access is the address 1 (one).

With this option on the **Modbus** tab, users can select whether this Driver sets that value automatically, thus allowing the use of data element's address on Tags, the default option, or the value sent in the PDU is the same provided on Tag configuration, the N4/B4 parameter in **numerical configuration**. There are devices that comply with Modbus standard in their address maps, starting at one, and other devices that map their values without a default offset, directly using the value of the address on communication's frame, starting at 0 (zero).

In addition to this single offset, there are still devices that use the old offset standard used by Modicon, the company that created the protocol, which is known as **Modbus Convention**, detailed on topic **Addressing Tips (Modbus Convention)**. Please check device's manual for information on the register map used. When in doubt, please check manufacturer's technical support.

NOTE

The **Data Address Model Offset** option used to be named **Use Older Address** on versions earlier than version 2.03, where the **Data is addressed from 1** option is equivalent to the old **Use Older Address** option enabled, and the **Data is addressed from 0** option is equivalent to the **Use Older Address** option disabled.

Other Options

The next table describes all other options on this tab, referring to this Driver's behavior.

Other available options on the Modbus tab

OPTION	DESCRIPTION
Default Slave Address	This feature allows configuring a default address for slaves, so that it is not necessary to configure them in each Tag. To use this feature, configure the Slave Id, the <i>N1/B1</i> parameter on numerical configuration or the Device field on String configuration , as 1000, that is, all Tags with their Slave Id equal to 1000 have this value replaced by the value configured in the Default Slave Address option. Users can also force using a default address in all Tags,

OPTION	DESCRIPTION
	regardless of the value configured in Slave Id, by selecting the Use Default Address option
Wait Silence on error	If this option is enabled, after every communication error this Driver remains in loop, receiving data until a time-out occurs. This clears the reception channel and prevents problems in future communications due to the reception of delayed bytes still in transit at the time of the error, and which may be confused with a response to a new command
Reconnect after Timeout (Ethernet only)	With this option enabled, after any time-out error in device's frame reception, this Driver performs a disconnection and a reconnection to the physical layer, clearing the connection from possible delayed frames still in transit, which may affect future requests. This option must be always enabled if using Modbus RTU in an Ethernet TCP/IP layer is inevitable in legacy systems, as the RTU mode does not have a transaction control, therefore it is not always possible to distinguish a correct response frame from a delayed one resulting from a previous reading, possibly from another address, which failed by a time-out. For new projects, it is strongly recommended NOT using Modbus RTU or Modbus ASC modes in an Ethernet TCP/IP layer. Notice that users must keep the Retry failed connection every option enabled on IOKit's Setup tab, so that this Driver reconnects after a time-out. Otherwise, this time-out only generates a disconnection and the application is responsible for managing this new connection

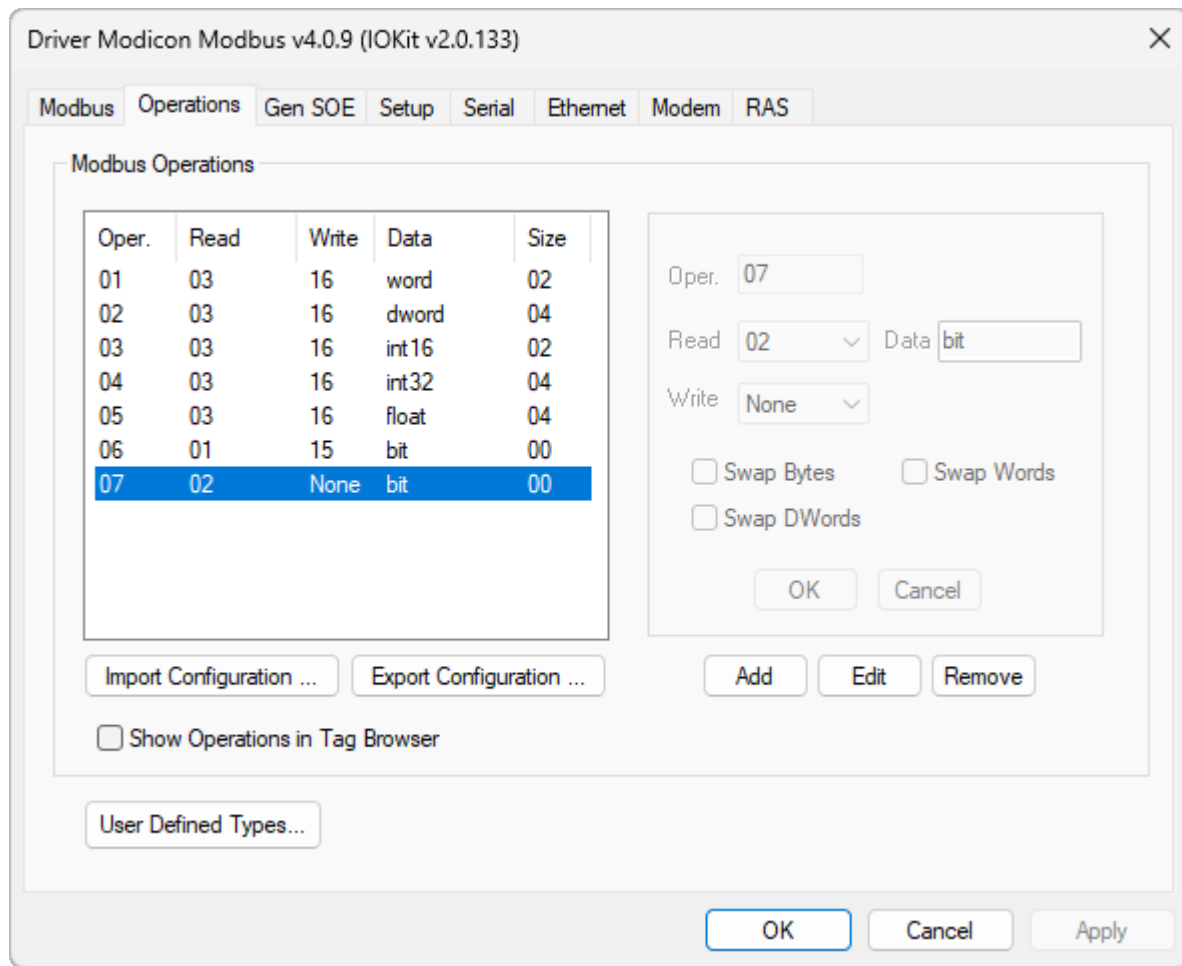
NOTE

The old **Swap Address Delay** option was removed from the configuration window in version 2.08. This Driver still supports it in pre-existing applications and allows enabling it by script. For more information, please check topic **Offline Mode Configuration**. For new applications, it is recommended to use the **Inter-frame Delay** option on **IOKit's Serial** tab, which replaces this old option with benefits.

Operations Tab

This topic describes how to configure the **Operations** tab on this Driver's configuration window, where all operations used on I/O Tags are defined, as shown on the next figure.

Configuring operations is no longer used when **configuring Tags by Strings**, it is only used in the old **numerical configuration**, the *N/B* parameters, on **Elipse SCADA**.



Operations tab on this Driver's configuration window

Operations

For a proper usage of this Driver, users must define which reading and writing Modbus functions are used for each I/O Tag. To do so, if Tag configuration is performed using the old **N/B numerical parameters** in **Elipse SCADA**, select the **Operations** tab on the configuration window.

For this Driver, **Operations** are configurations defining how each I/O Tag performs data writings and readings to and from a device.

An operation is simply a definition of a pair of protocol functions, one for writing and another one for reading, and a specification for additional conversions on the format of data that can be linked to application Tags. In other words, in this Driver the *N* or *B* numerical parameters of I/O Tags do not directly reference protocol functions, but rather pre-configured operations, which by their turn not only inform functions, protocol's **native** or even **special ones**, to use when communicating, as well as the way native protocol data must be interpreted.

Configuration of I/O Tag parameters is described later on topic **Configuring an I/O Tag**. Next, there is a description of the configuration of operations, which must be later linked to each I/O Tag.

NOTE

Operations work only as a template to configure **I/O Tags**, and user can, and usually want to, set a single operation to several Tags, which have in common the same value for their *N2/B2* parameters.

Functions

The Modbus protocol defines reading and writing functions, which can access distinct address spaces on a device, and with specific data types. Functions **03** and **16**, for example, protocol's most used ones, are responsible for reading and writing *Holding Registers*, which are simply 16-bit unsigned integer values (**Words**).

Modbus protocol's default functions provide data only in basic 16-bit **Bit** and **Word** formats. There are no additional data formats in protocol's specification.

A list of all Modbus functions supported by this Driver, which can be set to the configured operations, can be checked on topic **Supported Functions**.

In addition to protocol functions, this Driver also contains some **Special Functions** that are not part of protocol's standard, with a proprietary format and commonly used for reading events (SOE).

Data Formatting

In addition to allow linking functions, from the protocol or special ones, to specific Tags, operations also allow defining an additional formatting to apply to data, providing support to additional data types, not specified by the protocol, such as 32-bit (**Float**) and 64-bit (**Double**) floating point values. Supported data types are described on topic **Supported Data Types**.

It is important to notice that, when 32- and 64-bit data types are defined in operations, users must define protocol functions that work with 16-bit registers. This way, reading data with more than 16 bits results in reading several 16-bit Modbus registers from a device, that is, to read a Tag linked to an operation defining a 32-bit **Float** data type, this Driver must read two consecutive 16-bit registers from a device, concatenate them, and then perform a conversion to a **Float** format.

Users can also define eight-bit data types, **Byte**, **Int8**, or **Char**, in operations. Notice that, as protocol functions do not allow reading and writing isolated bytes, for each two Block Elements of eight-bit data types, this Driver is forced to access a distinct 16-bit register on a device. For that reason, this Driver does not allow writing eight-bit data types to Tags, to isolated Block Elements, or to Blocks with odd or unitary sizes. Writing eight-bit data types must be always performed with even-size Blocks.

User-Defined Data Types

In addition to pre-defined data types, native or built-in data types, described on topic **Supported Data Types**, this Driver also allows user-defined data types. These data types must be declared on a specific window, by clicking **User Defined Types** on the **Operations** tab. Such data types are structures created from pre-defined data types. For more information about user-defined data types, please check topic **User-Defined Data Types**.

Byte Order

In addition to protocol's reading and writing functions and the data type used, each operation also allows setting additional manipulations to bytes, related to a byte order, that is, the order of bytes inside every value. These are called swap options, *Swap Bytes*, *Swap Words*, and *Swap DWords*. Such options only need to be enabled for devices that do not respect protocol's default byte order.

The Modbus protocol defines that its 16-bit values always use a byte order known as *big endian*, also known as *Motorola*, because it is used by that manufacturer. The *big endian* standard always defines a byte order so that the most significant byte of each value comes first. Thus, as an example, when reading the hexadecimal value 1234h, the device first sends the most significant byte 12h and then the least significant one, 34h.

For devices that do not implement protocol's default byte order, and use another one known as *little endian* or *Intel*, data is sent with the least significant bytes first. Users must then enable those swap options to reverse that byte order.

There are also devices that use different byte orders for 32- and 16-bit types. For devices that, for example, use Modbus' default byte order (*big endian*) for 16-bit types, but provide 32-bit data with the least significant **Word** first (*little endian*),

users must only enable the **Swap Words** option, leaving the **Swap Bytes** option deselected. Possible situations are the following:

- For devices that provide data using Modbus' default byte order, *Motorola* or *big endian*, with the most significant bytes first, users must left all swap options disabled. This is the most common situation
- For devices using another byte order standard, with the least significant bytes first, *little endian*, users must enable all swap options referring to the data type used, that is, for 16-bit types, enable the **Swap Bytes** option. For 32-bit data types, enable the **Swap Bytes** and **Swap Words** options. For 64-bit types, all these swap options must be enabled
- In the least common case, devices that use different byte orders for different data sizes, providing, for example, the most significant byte of each **Word** first, but the least significant **Word** of each **DWord** first, then users must evaluate in which case each swap option must be enabled, so that it converts a value returned by a device to protocol's default *big endian* format

NOTE

All mentioned swap options have no effect for **Bit** data types or for data types with an eight-bit size, **Byte**, **Char**, and **Int8**. Swapping occurs inside every data type, that is, the **Swap Words** option has no effect for 16-bit data types, as well as the **Swap DWords** option has no effect for 32-bit data types. **BCD** data types also do not allow swapping.

To check if a device uses some unusual byte order format, check its manufacturer's documentation. In case this information is not found on that documentation, please contact manufacturer's technical support.

The topic **Frequently Asked Questions** contains tips on byte order configurations for some devices that are known to use swap options.

Bit Mask

The **Use Bit Mask** option is an advanced feature, used in specific and unusual cases where users want to read only a bit from the value returned by a device, but they cannot use application's bit mapping.

For most users, application's bit mapping fields are the best alternative to access bit masks, and there is no need to use this Driver's feature.

This feature was initially created to allow reading bits from *Holding Registers* by specialized **E3** libraries, in situations that prevented the usage of application's bit mapping.

In this case, this Driver reads a value from a device normally and then masks it, to return to Tag's **Value** field only the specified bit, **0** (zero) or **1** (one). The definition of a bit number to return is performed on I/O Tag's *N3/B3* parameter.

The **Use Bit Mask** option can only be used with integer data types with 16 bits or more (**Int16**, **Int32**, **Word**, or **DWord**). In addition, operations that enable this option can only be used for reading. The Modbus function for writing operations (**Write**) that use this mask option can be defined as **None**.

Driver's Default Operations

By default, when a new Driver is added to an application, that Driver is already created with default operations, described on the next table.

Default operations

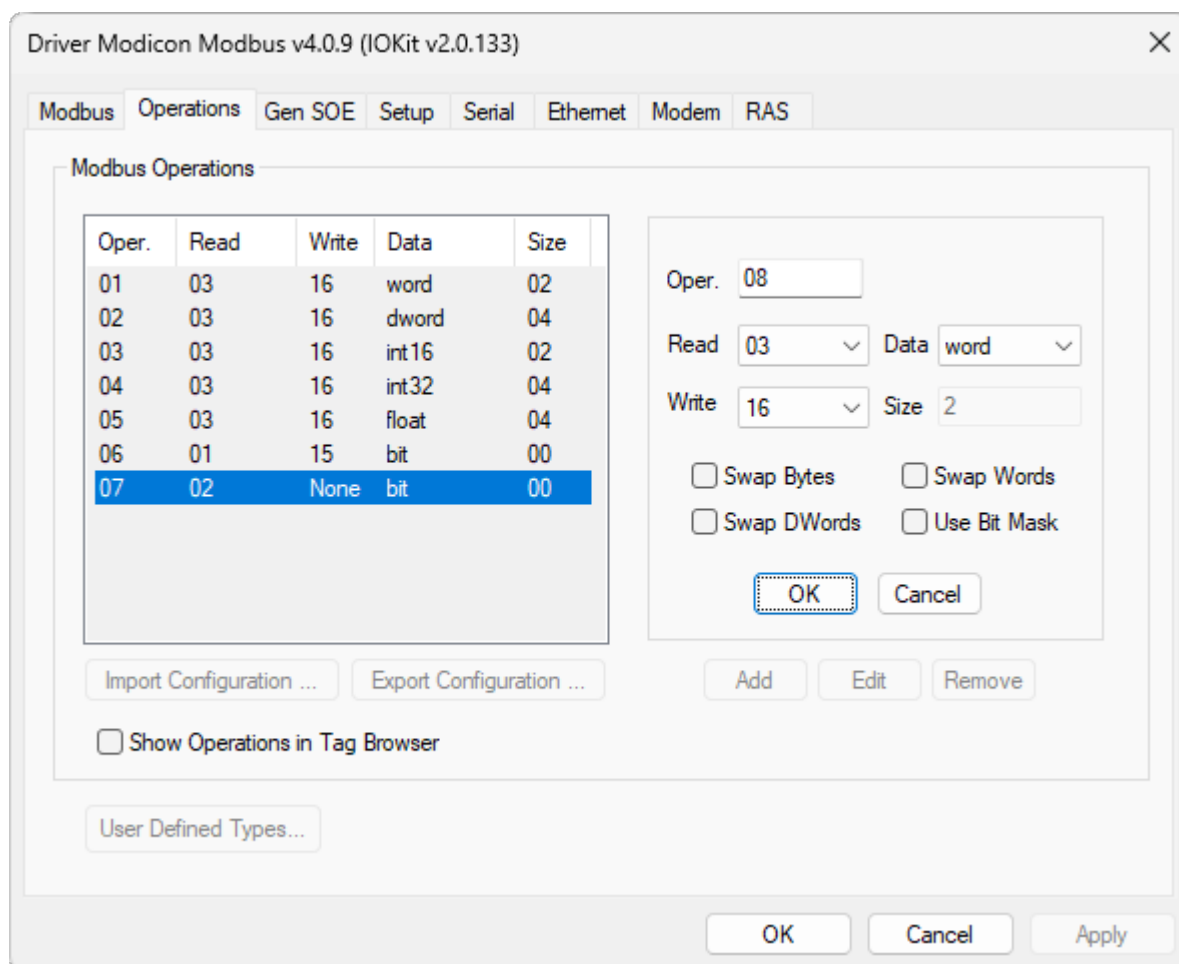
OPERATION	READING	WRITING	DATA TYPE	PURPOSE
1	3 - Read Holding Registers	16 - Write Multiple Registers	Word	Reading and writing unsigned 16-bit

OPERATION	READING	WRITING	DATA TYPE	PURPOSE
				integers
2	3 - Read Holding Registers	16 - Write Multiple Registers	DWord	Reading and writing unsigned 32-bit integers
3	3 - Read Holding Registers	16 - Write Multiple Registers	Int16	Reading and writing signed 16-bit integers
4	3 - Read Holding Registers	16 - Write Multiple Registers	Int32	Reading and writing signed 32-bit integers
5	3 - Read Holding Registers	16 - Write Multiple Registers	Float	Reading an writing 32-bit floating point values
6	1 - Read Coil Status	15 - Write Multiple Coils	Bit	Reading and writing bits
7	2 - Read Discrete Inputs	None	Bit	Reading bits from a Discrete Input data block

These operations are the most commonly used, and operation 1 (one) is the most common one. For most devices, select all operations needed among the ones provided by default, there is no need to create new operations or change the configuration of these default operations.

Defining New Operations

To add a new operation to this Driver, click **Add**.



Adding a new operation

To configure this new operation, select a number for it, used on I/O Tag's *N2/B2* parameter, which function to use for reading, and which function to use for writing, as well as informing a data type to be read or written by this Driver. Notice that, when clicking **Add**, this Driver already suggests a value that is not in use for this new operation.

For more information about supported data types, please check topic **Supported Data Types**. All other fields can be configured as needed. The next table contains a description of these fields.

Field options for operations

OPTION	DESCRIPTION
Size	A size in bytes of each element of the selected data type must be informed. This field is automatically filled in for fixed-size data types, such as Byte , Word , and Int16 , and it must be filled in for String and BCD data types. For Strings , this size defines exactly the number of bytes sent or received for each String value, that is, for each Tag or Block Element. If the String read or written has a shorter size, the remaining bytes are filled in with zeroes to complete its configured size. The String data type in this Driver has no defined maximum limit size, this limit is the maximum allowed by the protocol for frame's data area of a certain function
Swap Bytes	Indicates that this Driver must reverse the byte order, one by one, to retrieve a value
Swap Words	Indicates that this Driver must reverse the byte order, two by two, in Words , to retrieve a value

OPTION	DESCRIPTION
Swap DWords	Indicates that this Driver must reverse the byte order, four by four, in DWords , to retrieve a value
Use Bit Mask	Enables a bit masking of registers, using the <i>N3/B3</i> parameter. This option only affects readings and can only be used with integer data types, signed or unsigned, with at least 16 bits of size, Int16 , Int32 , Word , or DWord . Operations with this option enabled cannot be used for writing. For most users, it is recommended to use application's bit mapping, and leave this option deselected. For more information, please check the specific section

Protocol functions that can be configured in operations' **Read** and **Write** fields are described on topic **Supported Functions**. The next table describes each one of the available options.

Available options on the Operations tab

OPTION	DESCRIPTION
Import Configuration	This option allows importing configurations for operations from versions prior to Modbus Master/Slave Driver version 2.0, which stored these configurations on a modbus.ini file. This Driver does not use INI files anymore to store such configurations, which are now stored on the application file. For more information, please check topic Import and Export Operations
Export Configuration	This option executes the opposite operation of the previous option, generating an INI file containing all operation configurations, in the current format or in the same format of this Driver's previous versions. This way, users can store operation configurations of a certain device on a file, and these configurations can be used by other applications. For more details, please check topic Importing and Exporting Operations
Show Operations in Tag Browser	If this option is not selected (default), templates of Tags configured by Strings (Device and Item fields) are displayed on Tag Browser. If it is selected, templates of Tags numerically configured (<i>N/B</i> parameters) are displayed on Tag Browser. When creating new instances of this Driver, this option is deselected by default. In legacy applications, when this Driver's version is updated from a version previous to 3.1, this option is already selected, keeping the behavior of previous versions
Add	Adds a new operation to the list
Edit	Updates the selected operation on the list, which is equivalent of double-clicking an item
Remove	Removes the selected operation from the list

NOTE

The **Swap Bytes**, **Swap Words**, and **Swap DWords** options, as already explained, were added to provide compatibility with devices that do not comply with Modbus protocol's standard on data encoding (byte order). **If these options remain disabled, this Driver's behavior corresponds to protocol's standard, which is the recommended option for most devices.**

Supported Data Types

The next table lists this Driver's native data types, which can be defined when configuring I/O Tags.

As explained on topics **String Configuration** and **Operations Tab**, the Modbus protocol itself only supports **Bit** and **Word** data types (16 bits) for the **most commonly used functions** implemented by this Driver (the only exception is currently **function 7**). All other data types of this Driver are converted to **Word** at protocol's level, for reading from or writing to a device or slave device.

Also notice that this Driver supports **User-Defined Data Types**, defined as structures with elements composed by the native data types listed on the next table.

On the next table, data types use the same denominations of mnemonics for the **data type** field, when Tags are **configured by Strings**. For the old **numerical configuration**, the same denominations are also used on **Data** column of this Driver's configuration window, on the **Operations** tab. In some cases, frequent alternative denominations are displayed between parentheses.

Available options for data types

DATA TYPE	RANGE	DESCRIPTION
Char	-128 to 127	Eight-bit word, character. Writing must always occur in blocks with even size (Words)
Byte	0 to 255	Unsigned eight-bit word. Writing must always occur in blocks with even size (Words)
Int8	-128 to 127	Signed eight-bit word. Writing must always occur in blocks with even size (Words)
Int16	-32768 to 32767	Signed 16-bit integer
Int32	-2147483648 to 2147483647	Signed 32-bit integer
Word (or UInt)	0 to 65535	Unsigned 16-bit integer
DWord (or UInt)	0 to 4294967295	Unsigned 32-bit integer (Double Word)
Float	-3.4E38 to 3.4E38	32-bit floating point (IEEE 754) (four bytes: EXP F2 F1 0). Handling of non-normalized values: Float Underflow returns 0 (zero), Float Overflow returns NULL, and Float NaN returns NULL
Float_GE	-1.427E+45 to 1.427E+45	32-bit floating point used by GE, not compatible with IEEE 754 . It is used in GE GEDE UPS devices, with an eight-bit exponent $2^{[-128 \dots +127]}$ and 24-bit

DATA TYPE	RANGE	DESCRIPTION
		mantissa $[-2^{23} \dots +(2^{23}-1)]$. (four bytes: EXP F2 F1 F0). For more information, please check device's documentation
Double (or Real)	-1.7E308 to 1.7E308	64-bit floating point (IEEE 754)
String	Does not apply	Text in ANSI format, with a determined number of eight bit ASCII characters (Chars)
BCD	Check description	BCD (<i>Binary-Coded Decimal</i>) numerical value. When using this data type, an application must provide a positive and integer decimal value, sent in BCD format, respecting the specified size. The Size field, for BCD types, refers to the number of bytes sent to represent a value. As in BCD encoding each figure is converted to a nibble, then the allowed values must have a maximum number of figures equal to double the size of the value specified in the Size field, that is, if a value of 2 (two) is selected for the Size field, the maximum value that can be sent is 9999. Likewise, if Size is equal to 4 (four), the maximum value is then 99999999. Allowed values for the Size field for BCD types are two (two, Word) and four (four, Double Word). For more information about the BCD encoding, please check topic BCD Encoding
GE_events	Check description	Data type used when reading an event buffer (SOE) from a GE PAC RX7 PLC. Its definition is only allowed for operations that use the special reading function GE SOE . These events are returned as blocks with two Elements, with timestamps defined by the controller. For more information, please check topic Reading an Event Buffer from GE PAC RX7 Controllers
Bit	0 (zero) or 1 (one)	This data type is automatically selected when a bit-access function is selected. Bit-access functions are 01 , 02 , 05 , and 15 . The Size field is not used for Bit types. When using this data type, each Tag or Block Tag Element represents a bit
SP_events	Check description	Data type used when reading events (SOE) in Schneider Electric relays from SEPAM 20, 40, and 80 series. Its definition is only allowed for operations that use the special reading function SP SOE . These events are

DATA TYPE	RANGE	DESCRIPTION
		returned as a three-Element Block, with a device-provided timestamp. For more information, please check topic Reading Events from Schneider Electric Relays from SEPAM 20, 40, and 80 Series
GenTime	1/1/1970 00:00 to 31/12/2035 23:59:59.999 (please check the next note)	Date and time type composed by an eight-bit structure, originally created for use when reading events using generic SOE algorithm (GenSOE) . This data type can be used with other Modbus protocol functions, in addition to GenSOE . This format is read internally as a structure of Words , and the only valid swap function for this type is Swap Bytes . Representation of this type in PLC memory is described on topic GenTime Data Type . For more information about this type, please check topic Elipse Software's Generic SOE Reading Algorithm
Sp_time	1/1/1970 00:00 to 31/12/2035 23:59:59.999 (please check the next note)	Date and time data type composed by an eight-byte structure, used by Schneider Electric relays from SEPAM 20, 40, and 80 series, usually to represent a timestamp. For more information, please check device's documentation
UTC64d	1/1/1970 00:00 to 31/12/2035 23:59:59.999 (please check the next note)	Date and time data type represented in Double format (64-bit IEEE 754), with seconds since 1/1/1970 00:00
UTC32	1/1/1970 00:00 to 31/12/2035 23:59:59.999 (please check the next note)	Date and time data type represented as an unsigned 32-bit integer, DWord or UInt , with seconds since 1/1/1970 00:00. This format does not represent milliseconds, which are always considered as 0 (zero)
Int16_sm	-32767 to 32767	Signed 16-bit integer, with magnitude signal
Int32_sm	-2147483647 to 2147483647	Signed 32-bit integer, with magnitude signal
Int52 and UInt52	-9007199254740992 to 9007199254740992	52-bit integers converted to Double (floating point)
Str_I64	"-9223372036854775808" to "9223372036854775807"	64-bit signed integers converted to String (text) format
Str_U64	"0" to "18446744073709551615"	64-bit, unsigned integers converted to String (text) format

NOTES

- Although the representation of date and time data types on the previous table can represent dates greater than 12/31/2035, this limit is displayed on the table because **Elipse Software** applications do not currently support ranges of values that exceed this limit for timestamps.
- The **Int52** and **UInt52** data types process data equivalent to 64-bit integers, that is, they process 4 (four) Modbus registers with 8 (eight) bytes per value, limiting the range of values represented by 52-bit integers, due to a conversion to a **Double** data type. This Driver does not support 64-bit integer data types natively.
- The **Str_I64** and **Str_U64** types are handled as text only by SCADA applications. For devices, this data is automatically converted to a number format by this Driver.

GenTime Data Type

GenTime is a date and time data type originally defined and added to this Driver for use with **Elipse Software's Generic SOE Reading Algorithm**. It is, however, a generic data type that can be easily used with almost any PLC.

In an application, that is, in values of Tags and Block Tag Elements, as well as in Tag's **Timestamp** field, this data type, as well as all other date and time data types of this Driver, is represented by an application's native date and time data type. For more information about other date and time data types supported by this Driver, please check topic **Supported Data Types**. For more information about application's date and time data types, please check their respective user's manual, because there are some differences between VBScript used in **Elipse SCADA** and VBScript used in **E3** and **Elipse Power**.

In the PLC or slave device, this data type is represented by a structure composed by four 16-bit registers, or eight bytes, as displayed on the next table.

Structure of registers

OFFSET	CONTENT	BIT MAPPING (16 BITS)	RANGE (DECIMAL)
0	Year	AAAAAAAA AAAAAAAAAA	Between 0 and 65535
1	Day and Month	DDDDDDDD MMMMMMMM	Between 0 and 65535
2	Hour and Minute	HHHHHHHH MMMMMMMM	Between 0 and 65535
3	Second and Millisecond	SSSSSSMM MMMMMMMM	Between 0 and 65535

The base address, offset 0 (zero), for attribution on Tag's *N4/B4* parameter accessing data, contains the year. The next register, offset 1 (one), contains the day as the most significant byte and the month as the least significant byte. The offset 2 (two) contains the hour represented in the most significant byte and the minutes in the least significant byte. The fourth register contains the four most significant bits of the **Word** representing seconds, and the remaining bits, the two least significant of the most significant and the integer's least significant byte, representing milliseconds.

Notice that each Tag referencing this data type forces this Driver to read a block of four Modbus registers in a device to represent the value of each Tag or Block Element to return a valid value.

Advantages of this data type are its simplicity, because it is easily generated in a PLC ladder, its milliseconds precision, and its relative compression, as it does not need native support by the PLC or slave device.

NOTE

Although the **GenTime** data type itself is an eight-byte size, or four **Words**, the only effective swap option is **Swap Bytes**. This happens because, as already explained on this topic, this data type is structured in the PLC memory as having four **Words**, and it is not a device's native data type, rather a Driver's data type. More information about swap options (byte order) can be found on topic **Operations Tab**.

User-Defined Data Types

User-defined data types, or structures, after configured on the **User Defined Types** configuration window, can be used by this Driver's operations the same way as pre-defined data types.

These data types are in fact structures whose elements may have different **native data types**, that is, a user-defined data type is merely a structure defined based on this Driver's pre-defined data types, native or built-in data types, allowing users to configure Block Tags where each Element may have a different native data type.

Users can use almost all data types pre-defined by this Driver in their structures. The only data types not allowed are **Bit** data types, eight-bit data types, variable-size data types, such as **String** and **BCD**, and event data types linked to specific SOE functions.

Once a data type is defined, users can link it to any Tag, as long as it uses Modbus functions supporting **Words**, that is, it is not allowed to link a user-defined data type to an operation that defines as its reading function (**Read**) the **01** function, for example, because that function only reads bits.

In addition to the definition of structure elements, whose values are returned in Block Elements, users can also define the type of a Tag's timestamp, as well as the default address for that structure, address that is used for the **B4** parameter of Tags available using **E3**'s or **Eclipse Power**'s Tag Browser.

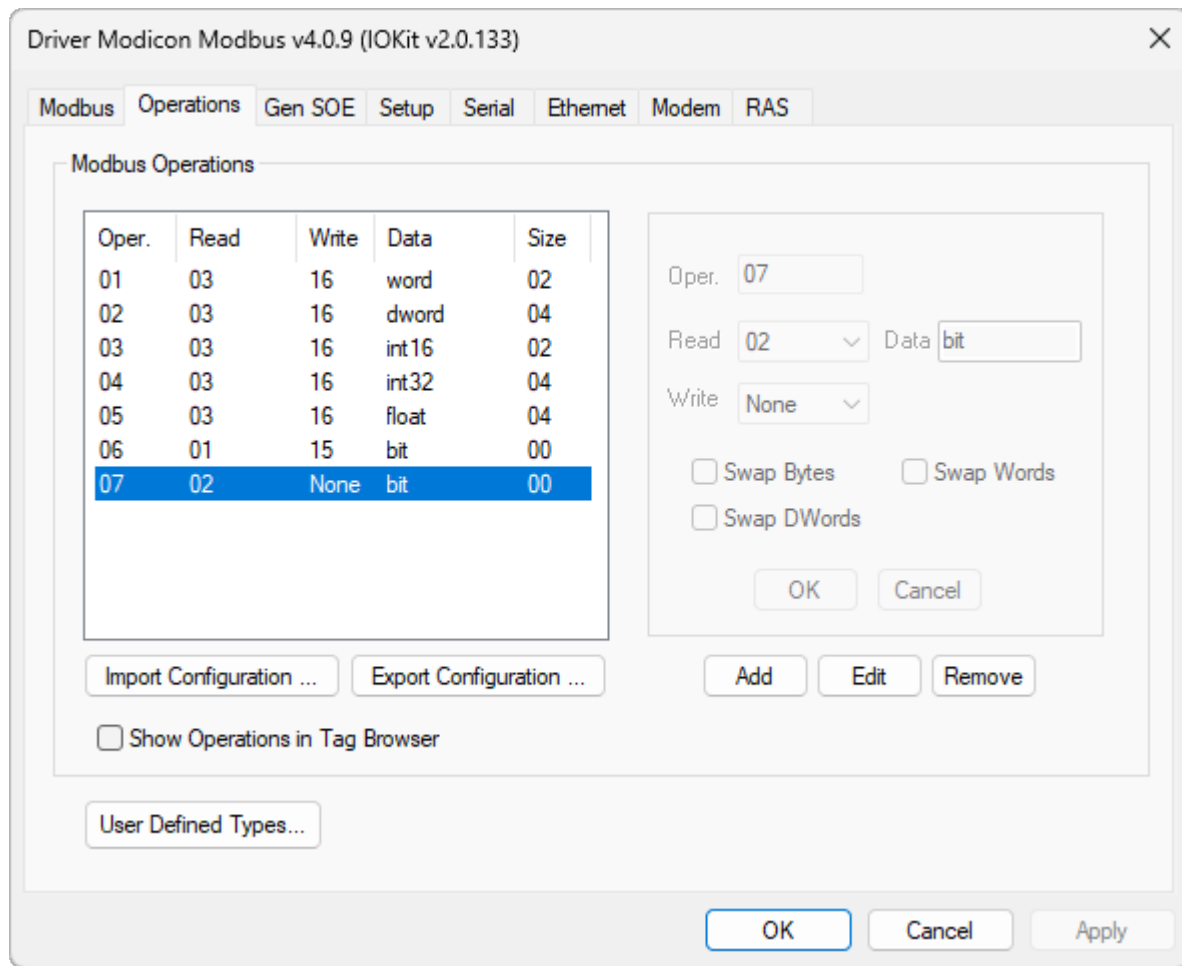
Applications

User-defined data types were originally implemented for use with **Eclipse Software's Generic SOE Reading Algorithm (Gen SOE)**, as this algorithm performs a reading on data structure tables.

In addition to using with SOE's generic algorithm, this feature can also be used to group different data types on a single Block Tag, optimizing communication in applications without Superblocks, such as **Eclipse SCADA**, or if the device in use for some reason does not allow using Superblocks. For more information, please check topic **Superblock Reading (Grouping)**.

Configuring User-Defined Data Types

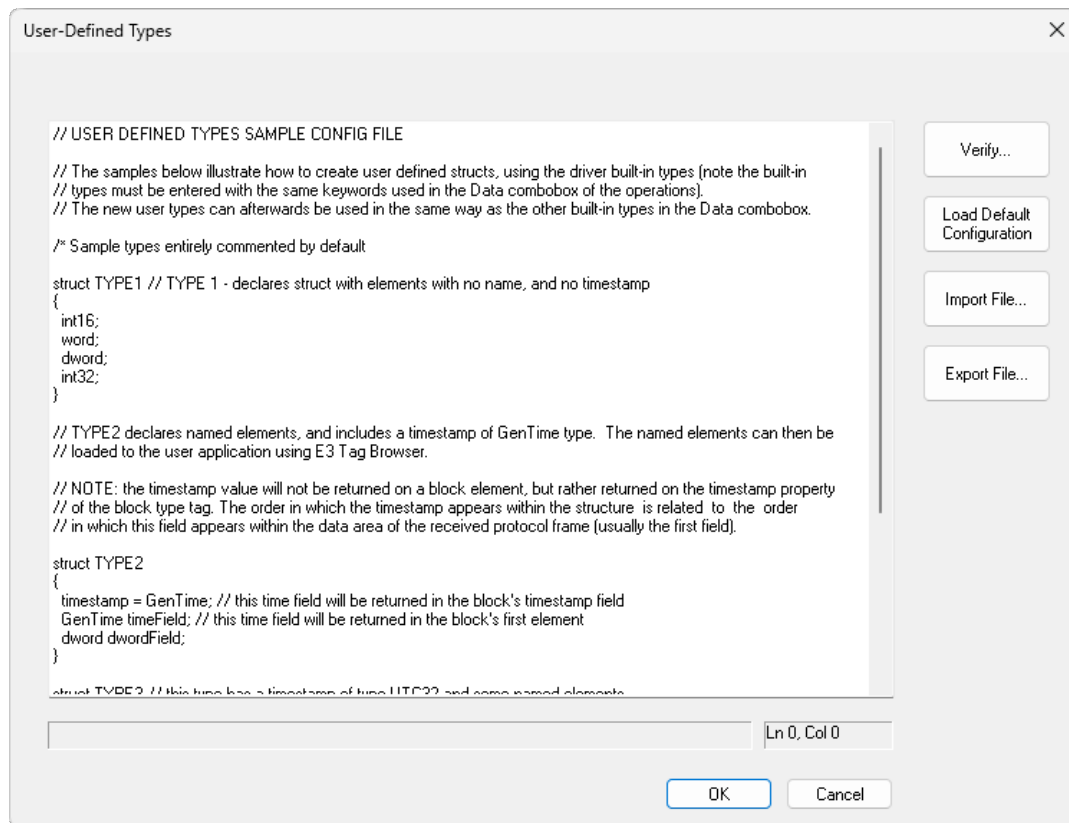
Configuration of user-defined data types is performed on a specific window, by clicking **User Defined Types** on the **Operations** tab of this Driver's configuration window, as shown on the next figure.



Operations tab of this Driver's configuration window

This window to configure user-defined data types allows editing files with structure configurations. When opening this window for the first time, it displays the default configuration file, with comments, which defines three example types that appear commented with multiple-line comments `/*` and `*/`, as explained later.

The next figure displays the configuration window for user-defined data types, with a small file defining these three example data types.



Configuration of user-defined data types

Notice that line comments always start with "//", identifying what is on the right, on the same line, as a comment, following the pattern for line comments of the C++ programming language, which is also used for other languages such as Java and C#.

Comments with multiple lines are also supported, still following the same C++ language syntax, starting by "/*" and finishing by "*/". Notice that the example file that accompanies this Driver already applies that comment format to its example types, leaving them commented by default. Remove the lines indicated by "/* Sample types entirely commented by default" and "*/", without quotation marks, so that all three example data types are ready to use.

As the text of the configuration file changes, the status bar displays the result of this file's syntactical analysis, in real-time. This status bar displays a "Status: OK!" message if no errors are detected in this file.

The line and column of cursor position in the edit box are always displayed on the right side of the status bar. Errors displayed on the status bar always reference the line and column number where it was detected.

This check can be also fully performed by clicking **Verify** and, in case of any error, cursor is then automatically placed on the error line.

The definition of each type has the following syntax, in which the elements inside brackets are optional:

```
struct <Type Name>
{
    [timestamp = <date and time type>;]
    [DefaultAddress = <address>;]
    <type> [name of element 1];
    <type> [name of element 2];
    <type> [name of element 3];
    [...]
    <type> [name of element n];
}
```

Where:

- **struct**: Keyword, lower case, starting the definition of a user-defined data type.

- **<Type Name>**: Name by which this new data type is identified by this Driver. This is the name displayed on the **Data** option, when configuring operations. Its maximum size is six characters
- **timestamp**: Optional field indicating that this structure contains a device-defined timestamp, which must be returned in Tag's **Timestamp** property. Each structure can have only one timestamp. The order in which it appears on this structure affects the position in which this field is read in the frame returned by a device. Notice that in Tags this value is returned only in the **Timestamp** property. Any date and time data types supported by this Driver can be defined. In the current version, this Driver supports date and time data types **GenTime**, **Sp_time**, **UTC64d**, and **UTC32**. For more information about data types, please check topic **Supported Data Types**
- **DefaultAddress**: Optional field specifying a default address value, used to fill the *B4* parameter of Tags in Tag Browser referencing operations containing this structure. Address values can be provided in decimal or hexadecimal format. To use the later format, users must precede the number with the "0x" prefix, such as using "0x10" to encode the decimal value 16 to hexadecimal
- **<date and time type>**: Pre-defined date and time data types for this Driver, which can be used as a timestamp by a slave device. In the current version of this Driver, native data types **GenTime**, **Sp_time**, **UTC32**, and **UTC64d** are accepted
- **<type>**: Element's data type. It must be defined as one of the pre-defined data types of this Driver, and written as it appears on the **Data** option, on the configuration window for operation parameters, respecting lower and upper case. **Bit** data types, eight-bit data types, and variable-size data types, such as **BCD** and **String**, are not allowed
- **[name of element]**: Optional parameter defining a name for each Block Element. If defined, determines a name for Block Elements in Tags displayed in **E3's** or **Elipse Power's** Tag Browser. If not defined in structure's declaration, this Driver then sets default names to Elements in Tag Browser, with the keyword "Element" followed by the index of that Element in the Block, such as "Element1", "Element2", and so on

Importing and Exporting

The **Import File** and **Export File** options allow importing and exporting a configuration file with user-defined data types to text files on disk. These options can be used to create backup copies of a file, or to share it among several Drivers. This file is always saved and read in Windows default ANSI format, using the **Windows-1252** character set. Future versions of this Driver may include support for other formats.

In addition to copying a file to disk, users can also use shortcut keys CTRL + A (**Select All**), CTRL + C (**Copy**), and CTRL + V (**Paste**) to copy and paste this file's content to another Text Editor.

The **Load Default Configuration** option loads the default configuration file again in the editor, the same file loaded when the configuration window is opened for the first time.

NOTE

When clicking **Cancel**, all changes performed on the file are discarded by this Driver. By clicking **OK**, this file is then stored in the application. This operation performs a full check on this file and if any error is identified, this error is then displayed and this window is not closed. If users want to save these changes with pending errors, export this file or copy and paste it to another Text Editor.

Using User-Defined Data Types with Tags Configured using Strings

Names for user-defined data types can be used as mnemonics for the **Type** field in the **Item** field, such as with Driver's native data types, as long as this name has been previously declared, as defined earlier in this topic.

IMPORTANT

As in **E3** the **Item** field is not case-sensitive, to use user-defined data types in this field, the user-defined data types must not be case-insensitive, that is, users must not define, for example, a data type named as "type1" and another one as "TYPE1". If this happens, users cannot use that user-defined data type in the **Item** field until that name is fixed.

For more information on Tag configuration using **Strings**, please check topic **String Configuration**. Example:

1. Read or write Holding Registers (functions **03** and **06**) of address 100 from a device with *Id* 5, interpreted as a user-defined data type named "mytype", with *Slave Id* in the **Item** field:
 - a. **Device**: "" (empty **String**)
 - b. **Item**: "5:shr100.mytype"

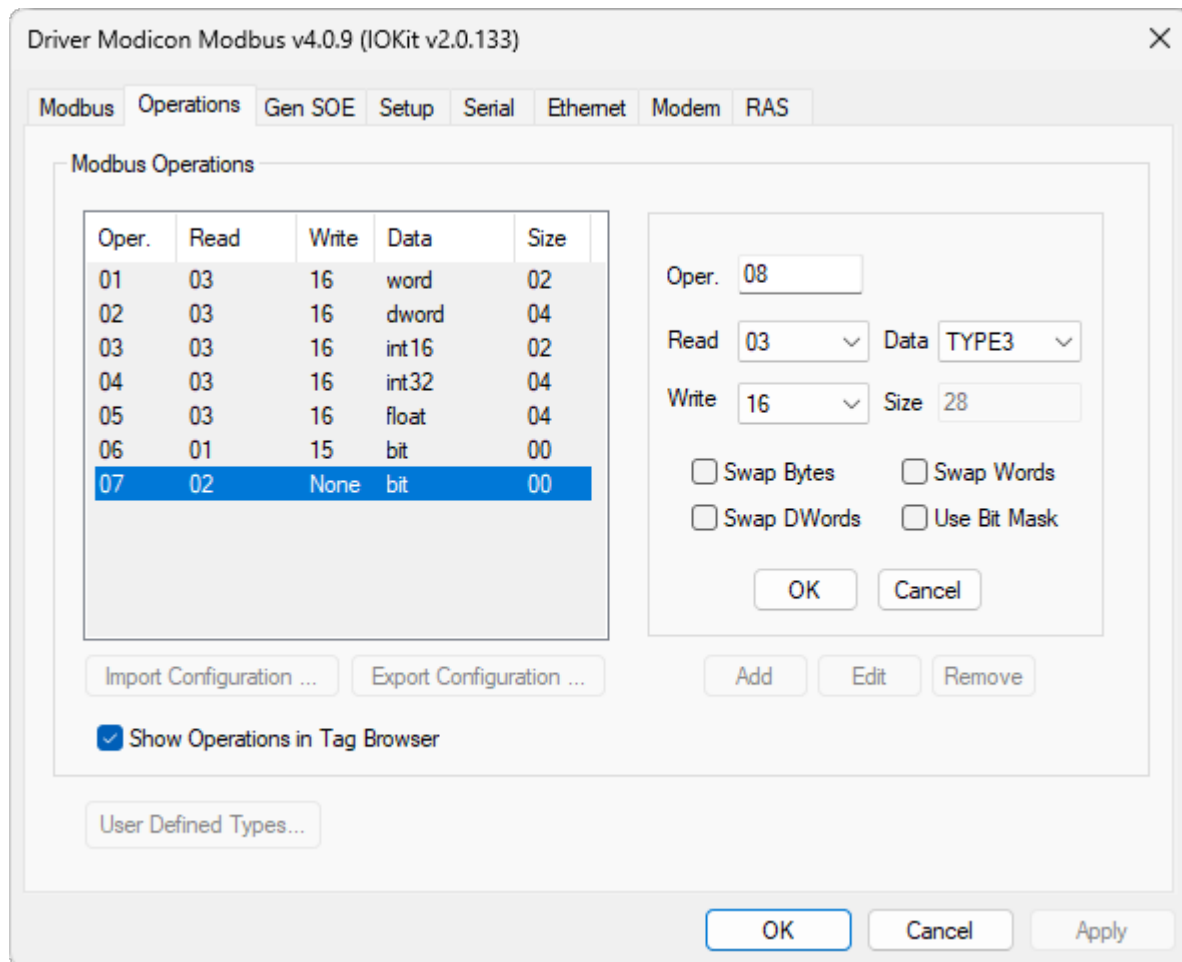
NOTE

Swap options, or byte order, for user-defined data types are only effective on the elements of the defined structure, not on the entire structure, that is, if the **Swap Words** option is enabled, all elements with more than 16 bits have their **Words** swapped. 16-bit elements, however, are not changed.

Using User-Defined Data Types on Numerical Configuration

After defining new data types in the configuration file on the **User-Defined Types** window, these data types are available for use in this Driver operations. Remember that only operations that use Modbus functions for access to 16-bit registers, such as functions **03**, **04**, **06**, and **16**, allow user-defined data types.

The next figure displays the configuration of a new operation that uses a user-defined data type, or structure, named **TYPE3**, showed on the previous example, after clicking **Add**.

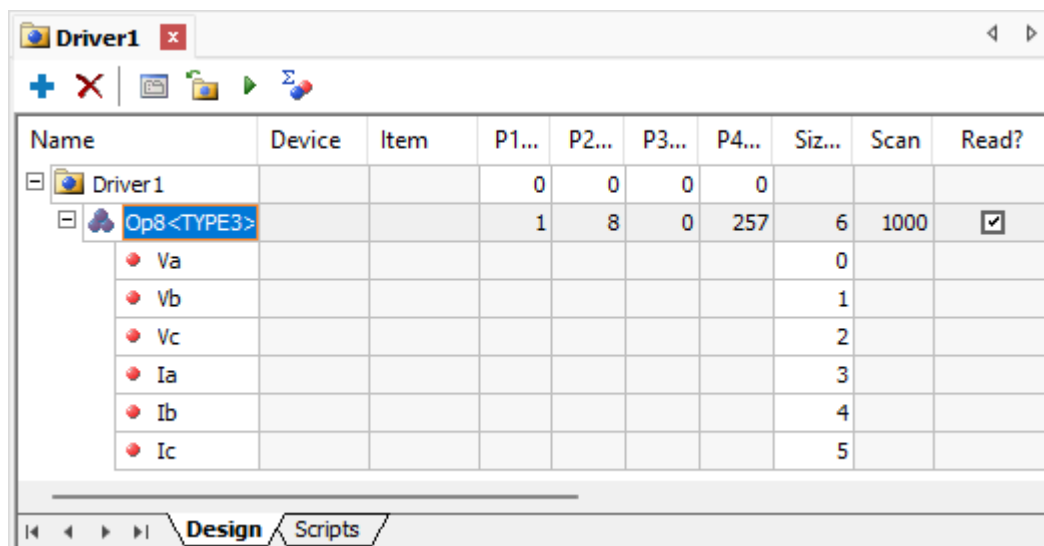


Adding a user-defined data type

NOTE

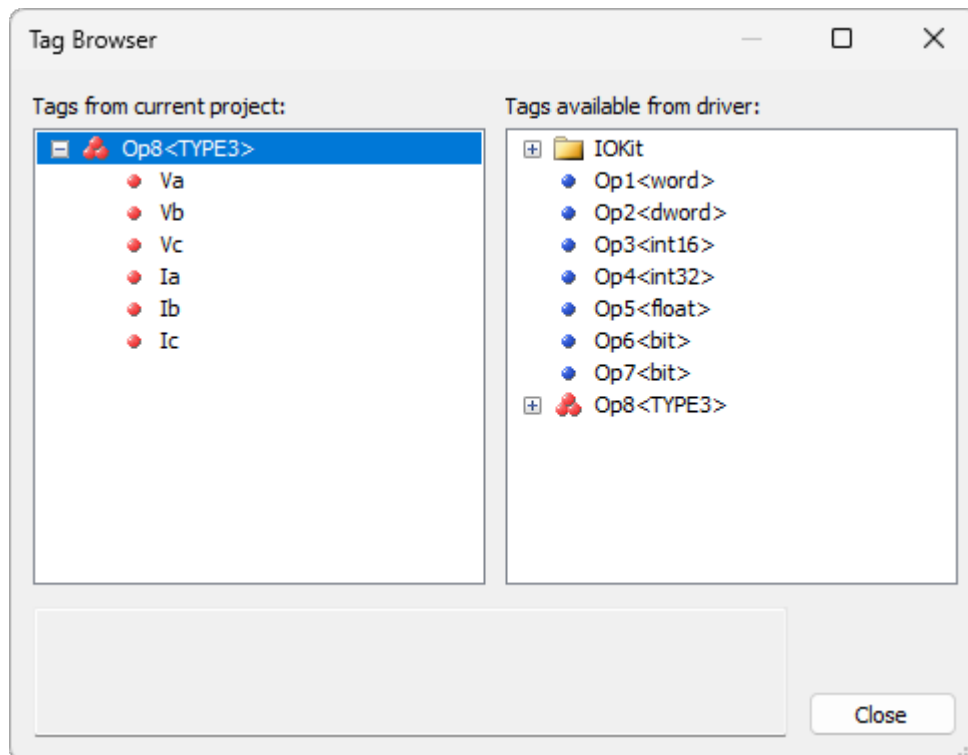
Swap options for user-defined types are only effective on the elements of the defined structure and not on the entire structure, that is, if the **Swap Words** option is enabled, all elements with more than 16 bits have their **Words** swapped. 16-bit elements, however, are not changed.

After defining a new operation, by using the new type **TYPE3**, define a Block Tag with that same data type and size equal to the number of elements of that structure, as shown on the next figure.



Declaring Tags using structures in E3 or Elipse Power

If a name was defined for each element of this structure, then users can use **E3's** or **Elipse Power's** Tag Browser to include a Block Tag referring the desired data type in an application, without typing it again. To use this feature, users must select the **Show Operations in Tag Browser** option on the **Operations** tab. The next figure shows how to perform this procedure.



Using Tag Browser to define Tags using structures

As this figure suggests, click  on this Driver's **Design** tab to open Tag Browser and drag the desired data type from the list **Tags available from driver** to the list **Tags from current project**.

Event-Reported Reading

User-defined data types or structures are generally used to define events in a PLC's memory, and can be used with **Elipse Software's Generic SOE Reading Algorithm**. However, if users want to read events organized in a PLC's memory, such as a sequence of structures, in an operation that only uses **protocol's public reading function**, that is, without using special functions with the SOE algorithm, such procedure can be performed in these ways:

- **Block Reading:** Create a Block with a number of Elements that is a multiple of the number of elements of user's data structure. For example, a user-defined data type or structure with two elements that represent events collected on an arrangement in a PLC's memory. If users want to read a block with five events, they must define a Block Tag with 10 Elements. Thus, a single reading from this Tag retrieves all events at once
- **Event-Reported Reading:** Uses a sequence of Tag's **OnRead** events to read a data block. With it, considering the example on the previous item, instead of creating a Tag with 10 Elements, users only need to create a single Block Tag with two Elements, configuring its *B3* parameter as "5". This way, when performing a Tag reading, **E3** calls Tag's **OnRead** event five times, and at each call the Elements and properties of this Block Tag contain data related to a specific event. The most common usage for Tags reported by events is storing events read directly from a historical database. This is easily performed using Historic's **WriteRecord** method previously linked to this Tag, on the **OnRead** event of the event-reported Tag. For more information, please check topic about Event-Reported Tags on *E3 User's Manual*

In other words, every I/O Tag using structures and using a **protocol's public reading function**, because this resource does not work for **special SOE functions**, becomes an Event-Reported Tag if its *B3* parameter is configured with a non-null value.

In case of special SOE functions, such as the **Gen SOE function**, the event-reported return is defined by the function's proprietary algorithm itself.

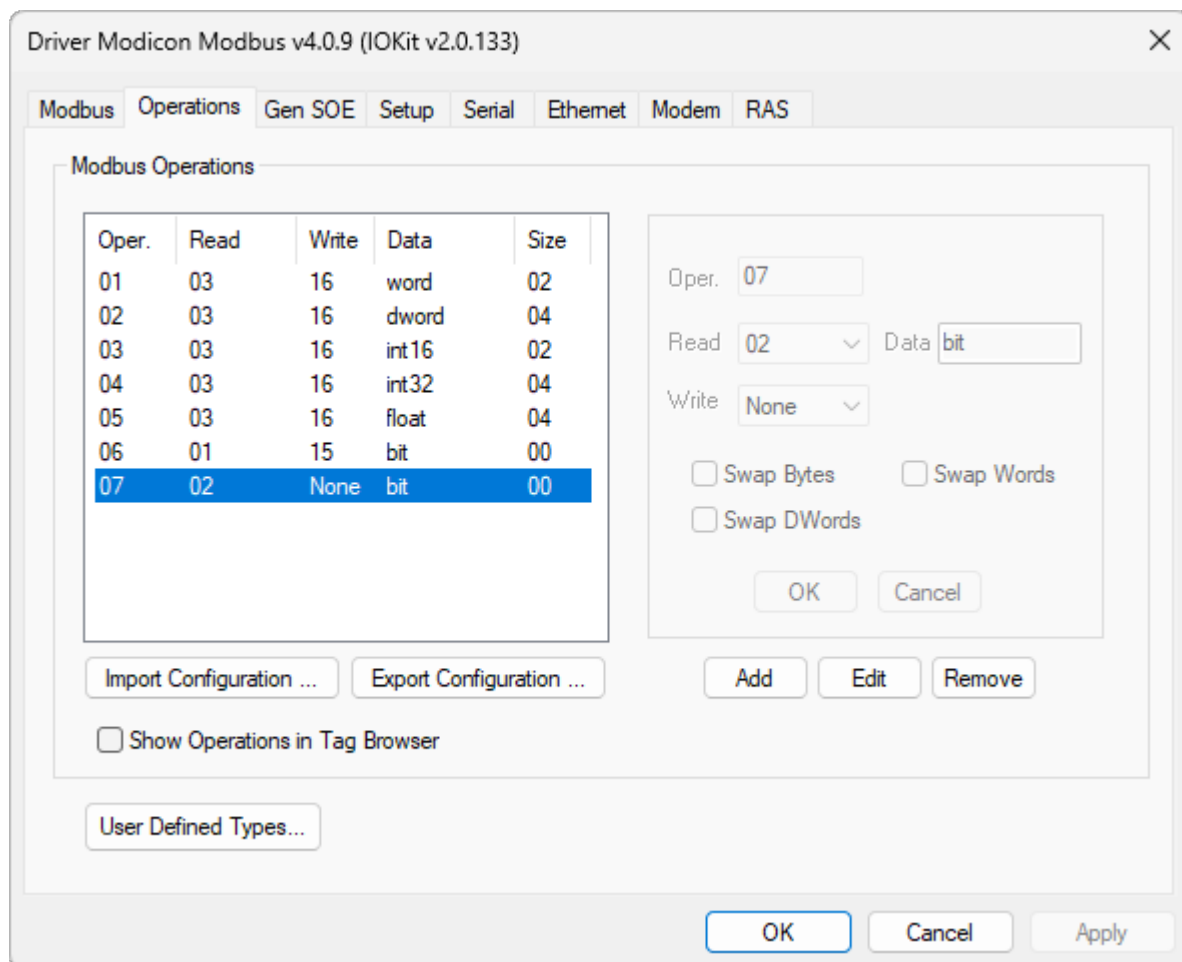
For more information about I/O Tag configurations, please check topic **Configuring an I/O Tag**.

IMPORTANT

When reading mass memory events in event-reported Tags in **E3** or **Elipse Power**, disable Tag's dead band, that is, the **EnableDeadBand** property configured as False, and also the linked Historic object, that is, the **DeadBand** property equal to 0 (zero), to avoid losing events with close values. It is also important to disable the historic by scan, that is, configure the **ScanTime** property as 0 (zero) in **E3** or **Elipse Power**. This ensures that new events are only stored using the **WriteRecord** method, executed in Tag's **OnRead** event, thus avoiding duplicated events.

Importing and Exporting Operations

Importing and exporting operations can be performed on this Driver's **Operations** tab, by clicking **Import Configuration** or **Export Configuration**, as shown on the next figure.



Options for importing and exporting operations

These options allow importing and exporting operation configurations displayed on the **Modbus Operations** frame to INI files.

On versions previous to 2.00 of this Driver, the configuration of operations were performed on a modbus.ini file, which was loaded during the initialization of a Driver object. modbus.ini files of these old versions still can be loaded on this Driver's current version, by using the import option.

NOTE

This Driver's operations used to be called **Driver Functions** in initial versions. This name was then changed to **Driver Operations** due to some cases when users confused it with **Protocol Functions**.

Importing

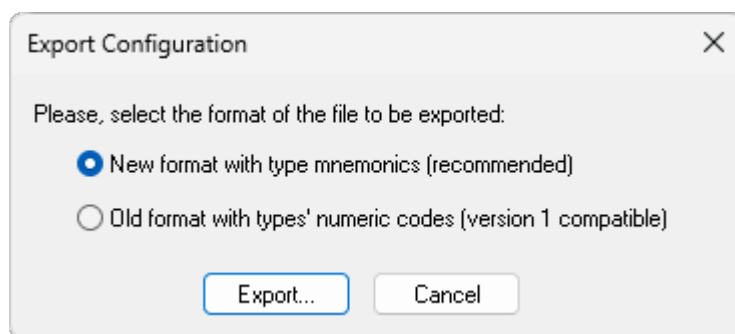
Importing configuration files is very simple. Click **Import Configuration** and select an INI file. This Driver must load operation configurations, which immediately appear on the **Modbus Operations** frame. This Driver allows importing files generated by previous versions.

Exporting

Exporting files with operation configurations can be performed to share the same operation configurations among different Driver objects, as well as performing backup copies of operation configurations of a certain device.

Another possible use is exporting configurations to a modbus.ini file compatible with previous versions of this Driver, allowing to load these configurations on previous versions. This is not advisable but, if inevitable in case of legacy applications, users must notice the next considerations.

When clicking **Export Configuration**, a window with the options shown on the next figure is opened.



Export options

On this window, users must select between exporting based on the new format, the **New format with type mnemonics** option, with displayed data types defined as **Strings** (mnemonics), or based on the old format, the **Old format with types' numeric codes** option, in which data types were identified by a numerical value, corresponding to the position where they appeared on the **Data** option of the **Operations** tab.

This new format is more legible, making it easy to debug, and is used on the most recent versions of this Driver, and it is the most recommended option.

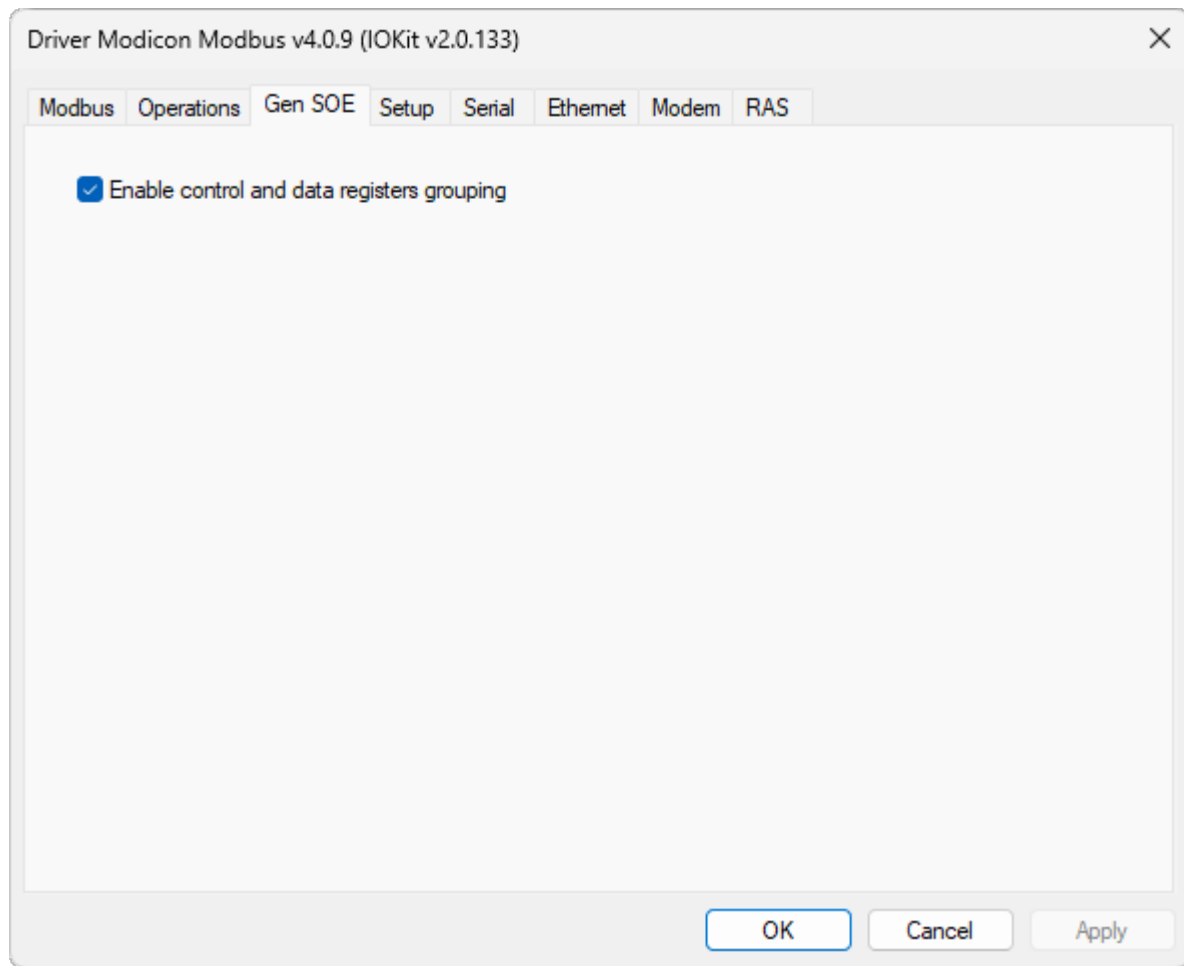
The old format, on the other hand, must be selected only if it is indispensable to export for versions previous to version 2.08 of this Driver.

Notice that, to export modbus.ini files successfully to load them to versions previous to 2.00, operations cannot define any new data type not implemented on the destination version, nor define operations that use the **Use bit mask** parameter, or the import operation may fail.

Generally, it is recommended to avoid exporting configurations to previous versions, preferring updated versions of this Driver.

Gen SOE Tab

The goal of this tab is to concentrate configuration options for **Elipse Software's Generic SOE Reading Algorithm**. The next figure displays all options on this tab.



Gen SOE tab

On the current version, there is only one configuration option:

- **Enable control and data registers grouping (default True):** Enables grouping control and data registers, to perform the minimum possible number of readings. If this option is enabled, this Driver starts reading tables already trying to read the maximum number of protocol-allowed registers, be them control or data registers, and possibly reading the entire table in a single reading. Such procedure usually optimizes Tag scanning the same way as Superblocks, because the time spent for reading large blocks is usually less than the time needed to perform several readings of the same amount of data, although this may depend on the PLC. ATOS PLCs do not allow grouped reading of control registers and data structures, thus requiring this option to be disabled

Offline Mode Configuration

This Driver's configurations can also be accessed at run time if it starts in **Offline** mode, as explained on topic **Documentation of I/O Interfaces**, by using the **String**-type parameters described on the next table.

Available parameters

PARAMETER	TYPE
ModiconModbus.ModbusMode	Integer. Possible values are 0 : Modbus RTU, 1 : Modbus ASCII, or 2 : Modbus TCP
ModiconModbus.Olderaddr	Boolean. Possible values are 0 : Data is addressed from 0 (zero) or 1 : Data is addressed from 1 (one)
ModiconModbus.UseDefaultSlaveAddress	Boolean. Possible values are 0 (zero) or 1 (one)
ModiconModbus.DefaultSlaveAddress	Unsigned integer
ModiconModbus.UseSwapAddressDelay	Boolean. Possible values are 0 (zero) or 1 (one)

PARAMETER	TYPE
ModiconModbus.SwapAddressDelay	Integer, with a delay interval in milliseconds. NOTE: Obsolete option kept for compatibility reasons. For new applications, please use the Inter-frame delay option on the Serial tab of IOLib library
ModiconModbus.WaitSilenceOnError	Boolean. Possible values are 0 (zero) or 1 (one)
ModiconModbus.UseGatewayTCP	Boolean. Possible values are 0 (zero) or 1 (one)
ModiconModbus.EnableCMSAddressing	Boolean. Possible values are 0 (zero) or 1 (one)
ModiconModbus.ConfigFile	String containing a configuration file with this Driver's operations. This file can be exported and imported on the Operations tab on this Driver's configuration window
ModiconModbus.EnableReconnectAfterTimeout	Boolean. Possible values are 0 : Time-out does not generate a disconnection from physical layer or 1 : In case of a time-out, when in an Ethernet physical layer, this Driver performs a physical layer disconnection and a reconnection. For Modbus RTU/ASC serial protocol communications over Ethernet, the reconnection is always performed after a time-out
ModiconModbus.UserTypesConfigFile	Configuration String for user-defined data types , or structures. This is the same configuration file available on this Driver's configuration window, in the User-Defined Types option
ModiconModbus.EnableGenSOERegGrouping	Boolean. Possible values are 0 : Event-reading algorithm first reads control registers, and then event data or 1 : Generic SOE reading is grouped to its maximum, not only reading control registers first, but also the maximum possible events
ModiconModbus.ShowOperationsInTagBrowser	Boolean. Possible values are 0 : Tag Browser displays Tag models configured by Strings , which is the default behavior, or 1 : Tag Browser displays Tag models configured numerically, which is the legacy behavior

For more information about **Offline** configurations at run time, please check topic **Documentation of I/O Interfaces**.

Configuring Tags

This topic describes configurations for several types of Tags supported by this Driver. Tags are divided into categories **Configuring an I/O Tag** and **Configuring Special Tags**.

Configuring an I/O Tag

This Driver's I/O Tags are Tags that allow communication with devices.

I/O Tags allow reading and writing Modbus registers from and to slave devices, by using **Modbus protocol functions**, or even **special functions**. This Driver does not differentiate between Block and simple Tags, in case of I/O Tags, that is, I/O Tags work the same way as a Block Tag with a single Element.

Data is read from a device using protocol-supported formats, that is, registers with integer values of 16 bits, bytes, or sets of bits, depending on the protocol's function used. For more information about protocol functions, please check specifications at *protocol's official website*.

I/O Tags can be configured in these ways:

- **String Configuration:** This is the newest method that can be used with **Elipse E3**, **Elipse Power**, and **Elipse OPC Server**, and recommended for new projects. It is not supported by **Elipse SCADA**
- **Numerical Configuration:** Old method used in **Elipse SCADA**

String Configuration

The **String** configuration of **I/O Tags** is performed using the **Device** and **Item** fields of each Tag.

That new configuration method does not work with **Elipse SCADA**, which still uses the old **numerical configuration**, with *N* and *B* parameters.

N and *B* parameters are not used when configuring by **Strings** and they must be kept in 0 (zero).

Configuration by **Strings** makes Tag configuration more readable, making it easy to configure and maintain applications.

Block Reading

Tags configured by **Strings** can be simple Tags or Block Tags, with their **Device** and **Item** fields with the same syntax.

Device Field

In the **Device** field, the *Slave Id*, or device's identification address, must be provided as a number between 0 (zero) and 255 followed by a colon, such as "1:", "101:", or "225:".

Please remember that, in Modbus RTU Serial protocol, the *Slave Id* "0" (zero) is reserved for broadcasting, which only makes sense if used by writing operations, because there is no return from devices, or else there would be conflicts.

Optionally, the *Slave Id* may appear at the beginning of the **Item** field, explained later, and in this case the **Device** field must be kept empty. This option is detailed next.

Item Field

The **Item** field must provide all addressing and operation information to be performed by a Tag, using the following syntax, in which parameters between brackets are optional.

```
<address space><address>[.<type>[<type size>]][.<byte order>][/bit]
```

As mentioned earlier, users can alternatively provide a *Slave Id* at the beginning of the **Item** field, such as the next example.

```
<slave id>:<address space><address>[.<type>[<type size>]][.<byte order>][/bit]
```

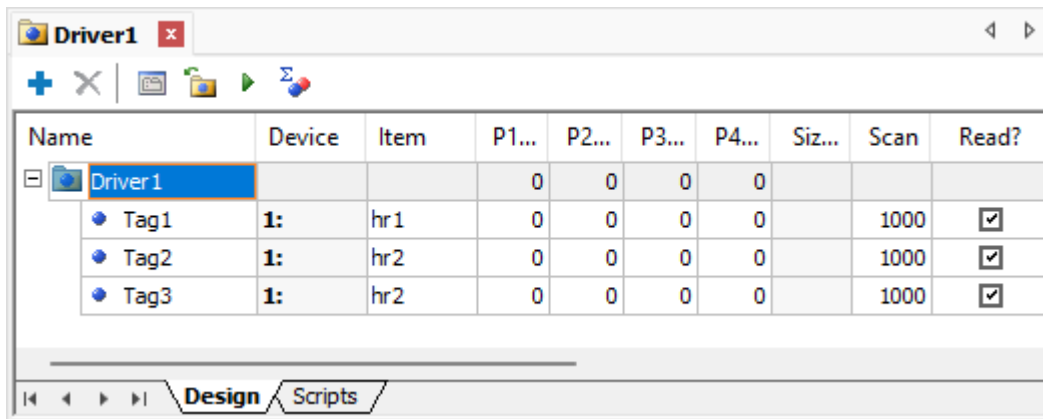
In this case, as already explained, users must keep the **Device** field empty.

The next examples show the most common usage cases. Notice that quotation marks must not be added to the application.

1. Reading or writing a Holding Register, functions **03** and **16**, on address 100 of a device with *Id* 1 (one), and *Slave Id* provided in the **Device** field:
 - a. **Device:** "1:"
 - b. **Item:** "hr100"

2. Reading or writing a Holding Register, functions **03** and **16**, on address 120 of a device with *Id* 3 (three), and *Slave Id* provided in the **Item** field:
 - a. **Device:** "" (empty **String**)
 - b. **Item:** "3:hr120"
3. Reading or writing a Coil, functions **01** and **15**, on address 65535 (FFFFh) of a device with *Slave Id* 4 (four), here provided in the **Item** field:
 - a. **Device:** "" (empty **String**)
 - b. **Item:** "4:cl&hFFFF" (or optionally "4:cl65535")

The next figure shows examples of **String**-configured Tags in **E3** or **Elipse Power**.



String-configured Tags

Mandatory and Optional Fields

Mandatory fields for all Tags are described next and individually detailed later in this topic:

1. **Address space:** A mnemonic defining a set of protocol's reading and writing functions to use. Please check the **table** with all supported mnemonics later on this topic.
2. **Address:** A numerical value identifying the item's address, register or bit, to read or write within the defined address space. These values can be provided in decimal, hexadecimal, or octal. For decimal values, no prefix is needed, or optionally a "&d" prefix can be used. For hexadecimal values, use the "&h" prefix, such as "&hFFFF". For octal values, use the "&o" prefix, such as "&o843". This address can have an offset relative to the address sent in the communication frame, which depends on the convention used by the manufacturer. When in doubt about addressing conventions, please check topic **Addressing Tips (Modbus Convention)**. Particularly, check whether the device implements the default offset of protocol's Data Model. Please check the **Data Address Model Offset** option on the **Modbus** tab.

The next fields are **optional**, used for extensions to the default protocol or for compatibility with devices not fully compliant with the protocol, and they are also individually detailed later in this topic:

1. **Type:** Defines how bytes from the data area of the communication frame must be interpreted. If omitted, the default types of the protocol for the respective functions are used, that is, **Word** for functions to access registers and **Bit** for functions to access digital data, Coils and Discrete Inputs. Please check the **mnemonics table** for all supported types later in this topic.
2. **Type size:** Users must specify this field only for variable-size types, such as **BCD** and **String**. Its numerical value indicates the size of the type in bytes.

3. **Byte order:** Mnemonic indicating the byte order of numerical values. If omitted, protocol's default order is then used, with the most significant bytes coming first in the communication frame, which is called *big endian*. Please check more information on the **specific section** about this option, later in this topic.
4. **Bit:** Allows returning a specific bit of an integer value, which obviously only makes sense in Modbus functions returning integer values or **Words**. Usually, users are advised to not use this feature, preferring application's bit mapping. Bit 1 (one) is the least significant and, the greater the value, the most significant is the bit. The maximum allowed value obviously depends on the type size, which is currently 64 for **Double** types. This field corresponds to the old **Use bit mask** option on **numerical configuration**. Please check more information on this option on topic **Operations Tab**.

Exceptions

Modbus protocol's **functions 20 and 21**, which access files, use a slightly different syntax from the one described previously:

```
fr<file>.<register>[.<type><type size>][.<byte order>[/bit]
```

Example:

- **Device:** "" (empty **String**)
- **Item:** "1:fr4.101" (reading register 101 from file 4 on *Slave Id* 1)

Specifically for the **GenSOE special function** (*Elipse Generic SOE*):

```
elsoe<N>.<initial addr.>[.<type><type size>][.<byte order>[/bit]
```

Specifically for the **SP SOE special function** (*Sepam SOE*), to read all events:

```
spsoe<event type>.<initial addr.>[.<type>][.<byte order>[/bit]
```

Specifically for the **SP SOE special function** (*Sepam SOE*), to read events from specific points:

```
ptspsoe<event type>.<event addr.>
```

Specifically for the **GE SOE special function** (*GE PAC RX7 SOE*):

```
gesoe<tag type + point index>.<queue base addr.>
```

Please check specific topics about the special functions mentioned previously for more information about configuring the respective Tags using **Strings**.

Addressing Space

Instead of explicitly defining Modbus functions or special reading and writing functions to use, as performed on the old **numerical configuration** and its concept of **operations**, when configuring using **Strings**, users define an addressing space through mnemonics listed on the next table, already linked to protocol's pre-defined functions and their respective native data types.

Addressing spaces and mnemonics

ADDRESSING SPACE	MNEMONIC	NATIVE TYPE	FUNCTION	COMMENTS
Holding Register	hr	Word (16-bit)	Functions 03 and 16	Functions 03 and 16 are protocol's most used ones (Modbus class 0)

ADDRESSING SPACE	MNEMONIC	NATIVE TYPE	FUNCTION	COMMENTS
Single Holding Register	shr	Word (16-bit)	Functions 03 and 06	Function 06 writes to the same registers of function 16 , the difference is that function 16 can only write in Blocks, while function 06 writes to a single register at a time, but with less overhead
Coil	cl	Bit	Functions 01 and 15	
Single Coil	scl	Bit	Functions 01 and 05	Function 05 writes to the same registers of function 15 , but cannot write in Blocks, therefore with less overhead
Discrete Input	di	Bit	Functions 02 and None (read-only)	
Input Register	ir	Word (16-bit)	Functions 04 and None (read-only)	
Exception Status	es	Byte	Functions 07 and None (read-only)	
File Register	fr	Word (16-bit)	Functions 20 and 21	
ABB MGE 144 - Mass Memory Reading	abbmge	Word (16-bit)	Functions 65 03 and None (read-only)	
ABB MGE 144 - Reset	abbmge.rst	Not used	Functions None for reading and 65 01 for writing	
ABB MGE 144 - Zeroes Maximum and Minimum Memory	abbmge.rstmxm	Not used	Functions None for reading and 65 02 for writing	
GE PAC RX7 SOE - Reading	gesoe	GE_events	Functions GE SOE for reading and None for writing	
SEPAM SOE Events	spsoe	SP_events	Functions SP SOE for reading and None for writing	Collects from the meter and returns a structure, with SP_events type, to the Tag with events from any points. For more information, please check topic SEPAM SOE
SEPAM SOE Single Point Events	ptpsoe	Int32	Functions SP SOE for reading and None for writing	Collects from the meter and returns to the Tag an integer value from the Edge field, relative to events

ADDRESSING SPACE	MNEMONIC	NATIVE TYPE	FUNCTION	COMMENTS
				from a specific point. For more information, please check topic SEPAM SOE
Elipse Generic SOE	elsoe	Word (16-bit)	Functions GEN SOE for reading, Modbus function 03 with additional algorithms, and 16 for writing	

Data Types

The table on the previous section lists all protocol's native data types, according to the Modbus functions used, as well as some specific data types used in protocol's non-standard **special functions**. For Tags returning these native data types, the *Data Type* parameter can be omitted from the **Item** field's **String**.

If users must interpret native data in a different way, which is common among devices using Modbus, they must specify a data type to use, as explained in this section.

A list with all native data types supported by this Driver, as well as their description, can be checked on topic **Supported Data Types**.

The next table lists all mnemonics used in the *<type>* parameter of the **Item** field for each supported data type, Driver-native, and also an alias or an alternative name.

Supported data types

TYPE	MNEMONIC	ALIAS
Char	char	ch
Byte	byte	by or u8
Int8	int8	i8
Int16	int16	i16
Int32	int32	i32
Word or UInt	word	u16
DWord or ULong	dword	u32
Float	float	f
Float_GE	float_GE	fge
Double or Real	double	d
Int52	int52	i52
UInt52	uint52	u52
String	string	s
Str_I64	str_i64	s_i64
Str_U64	str_u64	s_u64
BCD	BCD	bcd
GenTime	GenTime	gtm

TYPE	MNEMONIC	ALIAS
Sp_time	Sp_time	sptm
UTC64d	UTC64d	-
UTC32	UTC32	-

User-Defined Data Types

In addition to the data types listed on the previous table, users can also provide mnemonics for user-defined data types or structures. For more information, please check topic **User-Defined Data Types**.

To use user-defined data types in the **Item** field, however, the names of these data types must not be case-insensitive, as the **Item** field does not differentiate upper and lower case. If that happens, this Driver does not allow using these data types in the **Item** field. For more information, please check topic **User-Defined Data Types**.

Examples

- Reading or writing Holding Registers, functions **03** and **16**, to or from address 100 of a device with *Id* 1 (one), interpreted as a **DWord**, with *Slave Id* in the **Device** field:
 - Device**: "1:"
 - Item**: "hr100.u32" or "hr100.dword", or if it is convenient to use a hexadecimal, "hr&h64.u32"
- Reading or writing Holding Registers, functions **03** and **16**, to or from address 150 of a device with *Id* 3 (three), interpreted as a **Float**, with *Slave Id* in the **Item** field:
 - Device**: "" (empty **String**)
 - Item**: "3:hr150.f" or "3:hr150.float", or if it is convenient to use a hexadecimal, "3:hr&h96.f"
- Reading or writing Holding Registers, functions **03** and **16**, to or from address 1500 of a device with *Id* 5 (five), interpreted as a **Double**, with *Slave Id* in the **Item** field:
 - Device**: "" (empty **String**)
 - Item**: "5:hr1500.d" or "5:hr1500.double", or if it is convenient to use a hexadecimal, "5:hr&h5DC.d"
- Reading or writing Holding Registers, functions **03** and **06**, to or from address 100 of a device with *Id* 5 (five), interpreted as a **Word**, with *Slave Id* in the **Item** field:
 - Device**: "" (empty **String**)
 - Item**: "5:shr100" or "5:shr100.u16", or "5:shr100.word". Notice that, because it is a **Word**, Modbus protocol's native data type for Holding Registers, the data type can be omitted
- Reading or writing Holding Registers, functions **03** and **06**, to or from address 100 of a device with *Id* 5 (five), interpreted as a **user-defined data type** named "mytype", with *Slave Id* in the **Item** field:
 - Device**: "" (empty **String**)
 - Item**: "5:shr100.mytype"

NOTE

The addressing space of Holding Registers in Modbus protocol contains 16-bit registers. Therefore, to read 32-bit data types, such as a **DWord** or a **Float**, users must read two "hr" addresses for each Tag accessed. Likewise, reading a **Double**-type Tag demands the reading of four Holding Register addresses. For the same reasons, reading and writing "hr" Tags with a **Byte** data type can only be performed in pairs. On the device, each Holding Register address always contains two bytes.

Size of Data Types

BCD and **String** data types, as they have a variable size, demand the specification of a type size, in bytes, right after their data type.

Notice that **only data types 2 (two) and 4 (four) are valid**, that is, 2 (two) and 4 (four) bytes or 4 (four) and 8 (eight) digits for **BCD** data types. Examples:

1. Reading or writing Holding Registers, functions **03** and **16**, to or from address 100 of a device with *Id* 1 (one), interpreted as a **10-byte-String**, five "hr" registers, with *Slave Id* in the **Device** field:
 - a. **Device**: "1:"
 - b. **Item**: "hr100.s10"
2. Reading or writing Holding Registers, functions **03** and **16**, to or from address 100 of a device with *Id* 1 (one), interpreted as an **eight-digit-BCD**, four bytes or two "hr" registers, with *Slave Id* in the **Item** field:
 - a. **Device**: "" (empty **String**)
 - b. **Item**: "1:hr100.bcd4"

Byte Order

As explained on the syntax of the previous section, users can add an optional byte order parameter in Tag's **Item** field to specify a byte ordering for devices that do not comply with protocol's standard. If a device complies with Modbus protocol's default ordering, this field can be omitted.

If distorted values are read, which can be observed on early tests with a device, and if these values, converted to hexadecimal, are correct after inverting the position of some bytes, please read this section carefully.

The Modbus protocol uses the big endian format by default, where values are formatted with their most significant bytes coming first in communication frames. This is the default format used by this Driver. There is, however, a large amount of devices in the market that use values with other combinations for byte ordering.

As an example, if this Driver reads a value equal to "1234h", or "4660" in decimal, by default this Driver waits that data to be sent with a byte sequence equal to 12h and 34h. If a device uses an inverted default, which is called little indian, then the byte 34h is sent first and then the byte 12h, and this Driver may interpret it as 3412h, or 13330 in decimal, unless these two bytes were inverted before interpreting.

For 32-bit values, there are cases when **Word** values are swapped, but with bytes inside **Word** values keeping their default ordering. For example, the value 12345678h can be received as 56781234h. There are also other situations, with several different combinations for ordering.

To allow communication with these devices that do not follow protocol's standard byte order, this Driver allows users to configure Tags by specifying the standard to use.

The byte order parameter corresponds to the **swap options** from the old **numerical configuration**, and it may have values "b0", "b1", "b2", "b3", "b4", "b5", "b6", "b7", "alias", or else "alias2". For more information, please check the next table.

If the byte order parameter is omitted, data is interpreted as protocol's default, which is equivalent to the "b0" code.

The next table indicates that swap operations, *Swap Bytes*, *Swap Words*, and *Swap DWords*, are performed for each ordering mnemonic, from "b0" to "b7".

Swapping operations

	SWAP BYTES	SWAP WORDS	SWAP DWORDS	ALIAS	ALIAS 2 (SWAPS)	ORDERING *
b0				msb	-	by7 by6 by5 by4 by3 by2 by1 by0
b1	X			-	sb	by6 by7 by4 by5 by2 by3 by0 by1
b2		X		-	sw	by5 by4 by7 by6 by1 by0 by3 by2
b3	X	X		-	sb.sw	by4 by5 by6 by7 by0 by1 by2 by3
b4			X	-	sdw	by3 by2 by1 by0 by7 by6 by5 by4
b5	X		X	-	sb.sdw	by2 by3 by0 by1 by6 by7 by4 by5
b6		X	X	-	sw.sdw	by1 by0 by3 by2 by5 by4 by7 by6
b7	X	X	X	lsb	sb.sw.sdw	by0 by1 by2 by3 by4 by5 by6 by7

* 64-bit, where "by0" is "lsb" and "b7" is "msb"

That is, "b0" does not perform any swap operation on data bytes, keeping the original ordering of bytes received from a device, which is equivalent to deselecting the **swap options on the Operations tab** from the old **numerical configuration**.

"b1", on the other hand, performs a byte swapping, two by two, that is, when receiving a **Word**, an unsigned 16-bit integer, with the hexadecimal value 0102h, the value returned to the Tag is 0201h, with its bytes swapped. It is equivalent to the old **Swap Bytes** option.

"b2" performs a **Word** swapping, that is, bytes two by two, which obviously affects only 32-bit data or larger. This is the same as selecting the **Swap Words** option from the old numerical configuration. As an example, if the value 01020304h is received from a device, the value used for application Tags is 03040102h.

"b3" performs byte and **Word** swapping, which is equivalent to the old **Swap Bytes** and **Swap Words** options enabled simultaneously. In this case, the value 01020304h becomes 04030201h.

Likewise, "b4" performs a **DWord** swapping for 64-bit values, which corresponds to the **Swap DWords** option from the old **numerical configuration**, that is, the value 1122334455667788h is interpreted as 5566778811223344h, and so on for all other codes.

The last two table columns specify aliases that users can use for readability, that is, instead of using a "b0" code, users can use an "sw.sdw" alias, and so on.

How to Select the Correct Byte Order?

In most cases, a device's documentation specifies the byte order used, or how to configure it, because there are devices that allow that configuration.

In cases where device's documentation does not contain a configuration, users must contact manufacturer's technical support.

If there is no reliable information, users must perform empirical tests, analyzing the returned values, in hexadecimal, comparing them to the expected values, and then observing if there are byte order inversions that may explain the differences.

There are basically the following situations:

1. For devices providing data using Modbus protocol's default byte order, big endian or Motorola, with the most significant bytes coming first, users must omit this parameter or define it as "b0". This is the most common situation.
2. For devices using another standard for byte order, with the least significant bytes coming first, little endian, users must enable all swap options referring to the data type used, which corresponds to the "b7" mnemonic.
3. In the least common case, there are devices that use different byte orders for different data sizes, providing for instance the most significant byte of each **Word** first, but the least significant **Word** of each **DWord** first. Therefore, users must evaluate in which case each swap option must be enabled, thus converting the value returned by the device to protocol's default big endian format.

NOTE

All mentioned swap options have no effect for **Bit** data types or eight-bit-size types, such as **Byte**, **Char**, and **Int8**. Swapping occurs inside each data type, that is, the **Swap Words** option has no effect for 16-bit data types, as well as the **Swap DWords** option has no effect for 32-bit data types. **BCD** data types do not allow swapping operations either.

The topic **Frequently Asked Questions** lists a few known cases, already observed on technical supports. Examples:

1. Reading or writing Holding Registers, functions **03** and **16**, to or from address 1500 of a device with *Id* 5 (five), interpreted as a **Double** without byte inversion, with *Slave Id* in the **Item** field:
 - a. **Device:** "" (empty **String**)
 - b. **Item:** "5:hr1500.d" or "5:hr1500.double", or else "5:hr1500.d.b0"
2. Reading or writing Holding Registers, functions **03** and **16**, to or from address 1500 of a device with *Id* 5 (five), interpreted as a **Double** with the least significant byte of each **Word** coming first, and with *Slave Id* in the **Item** field:
 - a. **Device:** "" (empty **String**)
 - b. **Item:** "5:hr1500.d.b1" or "5:hr1500.double.b1", or else "5:hr1500.double.sb"
3. Reading or writing Holding Registers, functions **03** and **16**, to or from address 1500 of a device with *Id* 5 (five), interpreted as a **Double** with the least significant byte coming first, or little endian, and with *Slave Id* in the **Item** field:
 - a. **Device:** "" (empty **String**)
 - b. **Item:** "5:hr1500.d.b7" or other variations, such as "5:hr1500.d.lsb" and "5:hr1500.d.sb.sw.sdw"

Special Tags of this Driver

In addition to all Tags described previously, users can configure this Driver's **Special Tags** using **Strings**, which are described in details on their specific topics. Please click an item for more information.

Special Tags

DEVICE	ITEM	OPERATION
	ForceWaitSilence	Writing
<slave id>:	LastExceptionCode	Reading or writing

Numerical Configuration

Numerical configuration is performed using *N* and *B* parameters of **I/O Tags**, not using the **Device** and **Item** fields available in **Elipse E3** or **Power**, which must be kept empty.

This configuration method must be used in **Elipse SCADA** and in legacy applications. In applications using newer products, such as **Elipse E3**, **Elipse Power**, or **Elipse OPC Server**, it is recommended to use **String configuration**.

I/O Tags configured numerically reference **operations** previously configured on configuration window.

Operations

As already explained on topic **Operations Tab**, this Driver supports other data types in addition to protocol's native data types. For this reason, the concept of **Operation** was created on this Driver.

On operations using Modbus functions that read and write bits, such as protocol functions **1**, **5**, and **15**, this Driver always map binary values of each bit to Block Elements, where each Element represents the value of a specific bit.

Operations with eight-bit data types, such as the **Byte** data type, always imply, obviously, on reading at least two bytes, a 16-bit Modbus register. To prevent surprises, this Driver requires that eight-bit data writings be performed in pairs, that is, writing Blocks with an even number of Elements. These operations must be referenced using I/O Tag's *N2/B2* parameters, as described later.

Configuration Parameters of I/O Tags

The following configuration applies to I/O Tag's *N* parameters, as well as to I/O Block Tag's *B* parameters.

- **N1/B1**: Address of a slave device (PLC) on the network (*Slave Id*). This address can vary from 1 (one) to 247 for Serial networks via Modbus RTU or vary from 0 (zero) to 255 for direct communication with Ethernet devices via Modbus TCP. For **Serial** mode, the ID with a value of 0 (zero) is reserved for **Broadcast** mode, sending a message to all slave devices (PLC) on the network. In **Ethernet**, in **Modbus TCP** mode, the address generally used is the IP address, but the *Slave Id* can still be used when the IP address references a gateway connected to a device network, usually an RS485 network, with Modbus RTU, using a gateway capable of performing a conversion from **Modbus TCP** to **Modbus RTU**

NOTE

In **Serial** Modbus RTU, the **Broadcast** mode with *N1* equal to 0 (zero) cannot perform readings, only writings. In this mode all devices on the network are addressed, receiving the written value and not returning any response, to avoid network conflicts.

- **N2/B2**: Operation code. References an operation added on this Driver's configuration window. For more information, please check topic **Operations Tab**

- **N3/B3:** Additional parameter. This parameter is not generally used and can be kept in 0 (zero). It is only used in these situations:
 - **Modbus functions 20 and 21:** For operations that use these functions for file access, the functions **20 and 21**, the *N3/B3* parameters specify a file to access.
 - **Use Bit Mask:** For Tags referencing operations with the **Use Bit Mask** option enabled, the *N3/B3* parameter specifies the number of the bit to access. For more information, please check topic **Operations Tab**.
 - **User-Defined Data Types:** For operations that use structures, if the *B3* parameter is greater than 0 (zero), it defines the return of an event-reported block array, by using a sequence of Tag's **OnRead** events. For more information, please check topic **User-Defined Data Types**.
 - **Gen SOE Special Function:** For operations that use the **Gen SOE** special reading function, the *N3/B3* parameter indicates the size of the linked table in the PLC or slave device memory, as the maximum number of supported events. For more information, please check topic **Eclipse Software's Generic SOE Reading Algorithm**.
- **N4/B4:** Register, variable, or bit address on the slave device (PLC) to read or write, according to device's register map. For more information, please check device's documentation. It is important to correctly configure the **Data Address Model Offset** option. For more information, please check topic **Modbus Tab** and check if manufacturer's documentation does not use offsets used by old Modbus devices, known as **Modbus Convention**.
- **Size/Index (Block Tags only):** Each Block Element represents a data value of a type defined in the operation used, in the *N2/B2* parameter. Notice that this protocol only supports **Bit** or **Word** types. Thus, if this operation selects the **DWord** type, 32-bit, for each Block Element, this Driver must read two consecutive registers from a device.

Special Tags

In addition to I/O Tags, that is, Tags referring to operations, there are also special Tags to execute specific functions of this Driver. These Tags are described on topic **Configuring Special Tags**.

Addressing Tips (Modbus Convention)

On topic **Configuring an I/O Tag**, Tag addressing, that is, *N4/B4* parameters on **numerical configuration**, is described based on the most recent Modbus protocol specification, version 1.1b. However, there are devices that still use the old offset addressing convention known as **Modbus Convention**, which adds offsets to an address. This topic explains how to address Tags if a device's register mapping still follows that old convention, originated from initial Modicon specification, not included on the current specification.

The address provided in the Tag is sent together with protocol's request frame, with or without the default offset of 1 (one), required by the Modbus Data Model specified by the protocol, according to the configuration in the **Data Model Offset** option, on the **Modbus** tab of this Driver's configuration window.

In addition to this default offset of 1 (one), defined on the current Modbus standard, version 1.1b, some manufacturers still use the old Modicon standard, known as **Modbus Convention**, with an offset that can be added to the address, and whose value depends on the Modbus function used, or more specifically, depends on which addressing space this function accessed originally. Such additional offset must be ignored when defining Tag addresses on this Driver. Later on this Manual there are more examples. The next table lists all offsets used by the **Modbus Convention** standard.

Modbus Convention standard offsets

DATA TYPE (STANDARD DATA MODEL)	MODBUS FUNCTION	OFFSET
Coils	01: Read Coils (0x), 05: Write Single Coil (0x), and 15: Write Multiple Coils (0x)	000000
Discrete Inputs	02: Read Discrete Inputs (1x)	10000

DATA TYPE (STANDARD DATA MODEL)	MODBUS FUNCTION	OFFSET
Input Registers	04: Read Input Registers (3x)	30000
Holding Registers	03: Read Holding Registers (4x), 06: Write Single Register (4x), and 16: Write Multiple Registers (4x)	40000
File Register (old Extended Memory file)	20: Read General Reference (6x) and 21: Write General Reference (6x)	60000

If a device's register map uses this convention, users must follow this procedure to determine the addresses to attribute to Tags, in the **Item** field when **configuring by Strings** or in *N4* or *B4* parameters for **numerical configuration**:

1. On the **Modbus** tab, select the **Data is addressed from 1** option.
2. Subtract from the address displayed on device's manual the offset shown on the previous table for the Modbus function used. **TIP:** Remove the fifth digit from right to the left.

Notice that, in devices that use this old convention, users can determine which Modbus functions can be used to access each register or bit using the offset used in its address.

Examples

ADDRESS WITH OFFSET (DEVICE)	ADDRESS ON I/O TAG (ITEM OR N4/B4)	MODBUS FUNCTION
01234	1234	01: Read Coils, 05: Write Single Coil, and 15: Write Multiple Coils
11234	1234	02: Read Discrete Inputs
31234	1234	04: Read Input Registers
41234	1234	03: Read Holding Registers, 06: Write Single Register, and 16: Write Multiple Registers
45789	5789	03: Read Holding Registers, 06: Write Single Register, and 16: Write Multiple Registers
65789	5789	20: Read General Reference and 21: Write General Reference

Modbus functions **20** and **21** are intended to manipulate reading or writing of multiple subgroups of data, allocated in the equipment's extended memory, or 6XXXXX references, according to the next table.

Extended memory or 6XXXXX references

MEMORY FILE NUMBER 1		MEMORY FILE NUMBER 2		MEMORY FILE NUMBER 3	
Register 600100	Data 1	Register 600200	Data 1	Register 600300	Data 1
Register 600101	Data 2	Register 600201	Data 2	Register 600301	Data 2
Register 600102	Data 3	Register 600202	Data 3	Register 600302	Data 3
Register 600103	Data 4	Register 600203	Data 4	Register 600303	Data 4

MEMORY FILE NUMBER 1		MEMORY FILE NUMBER 2		MEMORY FILE NUMBER 3	
Register 600104	Data 5	Register 600204	Data 5	Register 600304	Data 5

To use a PLC Tag to read the second register of Memory File Number 2 of Slave ID 1 (one), use the following parameters:

- **Numerical Configuration:** $N1 = 1$, $N2 = \text{Oper}$, $N3 = 2$, $N4 = 201$
- **String Configuration:** $\text{Item} = 1:\text{fr}2.201$

To use a Block Tag with 3 (three) Elements to read the last three registers of Memory File Number 3 of Slave ID 2 (two), use the following parameters:

- **Numerical Configuration:** $N1 = 2$, $N2 = \text{Oper}$, $N3 = 3$, $N4 = 302$
- **String Configuration:** $\text{Item} = 2:\text{fr}3.302$

To use a PLC Tag for writing in the fourth register of Memory File Number 1 of Slave ID 3 (three), use the following parameters:

- **Numerical Configuration:** $N1 = 3$, $N2 = \text{Oper}$, $N3 = 1$, $N4 = 103$
- **String Configuration:** $\text{Item} = 3:\text{fr}1.103$

NOTE

The Modbus protocol allows users to read or write multiple subgroups (file number) of data through a single request via functions **20** or **21** respectively. However, this Driver only allows users to manipulate one subgroup of data at a time in each PLC or Block Tag.

Automatic Block Partition

Starting with version 2.00, Modbus Driver now has a feature called **Automatic Block Partition**. With this feature, this Driver manages the division of blocks larger than **protocol limits**. Thus, users do not need to worry about exceeding the maximum limit for block size, because this Driver divides blocks in the correct sizes during communication with a device, if any Block Tag exceeds the maximum allowed size.

Starting with version 2.01, this Driver also supports **Superblock Readings**. With this feature enabled, users do not need to group variables into Block Tags aiming to improve performance, it is possible to use only Tags without degrading performance. And as the Superblock algorithm already considers the maximum block size allowed by the protocol, when this feature is used this Driver also does not need to use the Automatic Partition feature.

In cases when, due to device's specificity, as described on topic **Superblock Reading (Grouping)**, it is not possible to enable the **EnableReadGrouping** property in **E3** or **Eclipse Power**, that is, the property that enables Superblocks, or if users are using the old **Eclipse SCADA**, which does not support grouping (Superblocks, then they must use Automatic Block Partition to ignore protocol limits.

IMPORTANT

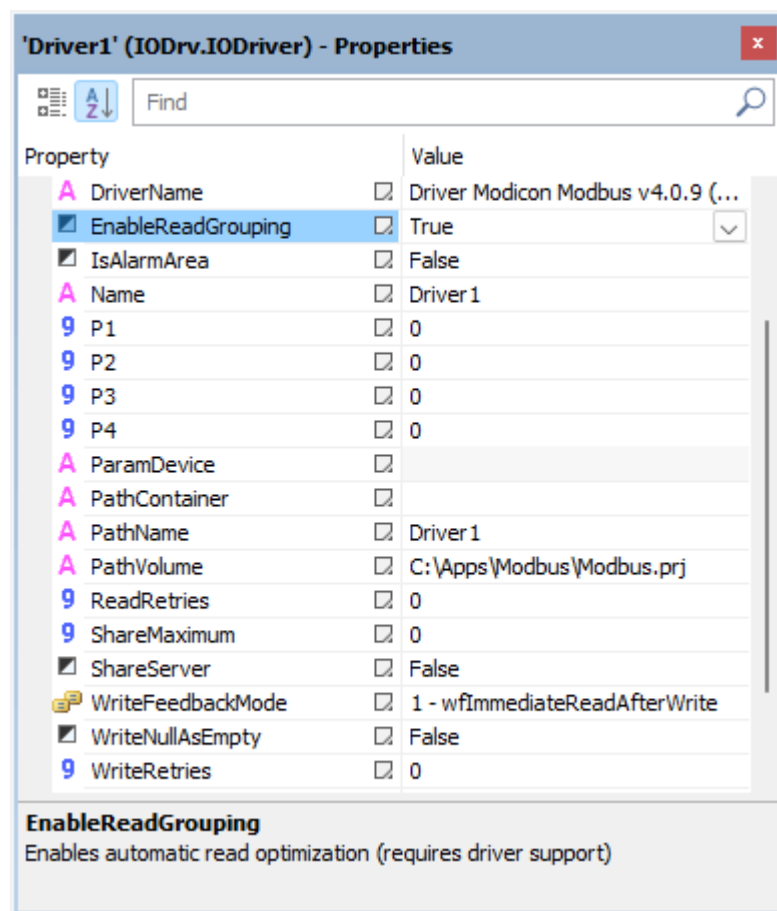
Superblock grouping in **E3** and **Elipse Power**, as well as this Driver's Automatic Block Partition, require that a device supports all limits established by standard Modbus. For more information, please check topic **Maximum Limit for the Size of Blocks Supported by the Modbus Protocol**. There are devices, however, that support lower limits. For this automatic block partition to work in these cases, as well as Superblock grouping, starting with version 2.03 this Driver allows customizing the maximum supported limit for PDU (*Protocol Data Unit*). To do so, on this Driver's configuration window, on the **Modbus** tab, enable the **Customize Max. PDU Size** option and configure the maximum size of bytes supported by this device for PDU. If a device has different limits for each function, it may be necessary to perform that grouping manually. For more information, please check topic **Superblock Reading (Grouping)**.

The article *KB-23112* on **Elipse Knowledgebase** presents a summary of questions related to Tag grouping and Block resizing in this Driver, discussed here and in other topics. For more information, please check topics **Superblock Reading (Grouping)** and **Optimization Tips**.

Superblock Reading (Grouping)

Starting with version 2.01, this Driver supports a **Superblock Reading** feature. This feature is supported by **E3** and **Elipse Power**, and it can be enabled via the **EnableReadGrouping** property of this Driver on Organizer. When this property is configured to True, users do not need to worry about block resizing.

With this feature, it is possible, and usually recommended, to create applications with only simple Tags, or PLC Tags in **Elipse SCADA**, without performance issues, because group optimization on readings is automatically performed during communication. The next figure displays the configuration of the **EnableReadGrouping** property on **E3** or **Elipse Power**.



EnableReadGrouping property

Elipse SCADA does not support Superblocks. The behavior when reading Tags on **Elipse SCADA** is identical to **E3** and **Elipse Power** when the **EnableReadGrouping** property is configured as False. In both cases, this Driver relies on **Automatic Block Partition**, and it can divide blocks with sizes larger than **protocol limits** into smaller blocks during

communication. In these cases, users must consider that grouping when defining application Tags, as described later on this topic.

NOTES

Automatic grouping is performed based on application Tags in advise. Whenever new Tags enter or leave advise, the Superblock algorithm redefines this grouping, that is, Superblocks to be read automatically, at run time, including only Tags in advise.

IMPORTANT

Superblock grouping in **E3**, as well as Driver's **Automatic Block Partition**, require that a device supports the limits established by standard Modbus. For more information, please check topic **Maximum Limit for the Size of Blocks Supported by the Modbus Protocol**. There are devices, however, that support lower limits. For automatic block partition and Superblock grouping to work on these cases, starting with version 2.03, this Driver allows customizing the maximum limit supported for PDU (*Protocol Data Unit*). To do so, on this Driver's configuration window, **Modbus** tab, enable the **Customize Max. PDU Size** option and configure the maximum size of bytes supported for PDU on this device. If this device supports different limits for each type of function, users must perform a manual grouping, as described further on this topic, observing all limits described on manufacturer's documentation.

Identifying Devices that do not Support Automatic Grouping (Superblocks)

The Superblock algorithm considers all limits and addressing spaces defined by standard Modbus protocol. For devices that implement Modbus protocol with small variations, some additional advanced configurations may be necessary to use this Superblock feature, if its usage appears viable. In these cases, it is necessary to disable automatic grouping, that is, the **EnableReadGrouping** property configured as False, and then perform a manual grouping. The following conditions may prevent using Superblocks, or may require additional advanced configurations:

- Devices that define maximum limits for block sizes lower than protocol's standard limit, that is, **a limit of 253 bytes for PDU**. **Solution:** Configure the **Customize Max. PDU Size** option on the **Modbus** tab

NOTE

There are devices whose PDU limits vary according to the Modbus function used. In these cases, if it is necessary to use functions with different limits, it is also necessary to disable Superblocks, that is, the **EnableReadGrouping** property configured as False, by manually grouping Tags, as described later on this topic.

- Devices with discontinuities, or undefined address intervals inserted between valid intervals, on the register map. **Solution:** Once it is impossible to inform to the Superblock algorithm which intervals cannot be inserted in blocks, usually it is not possible to use Superblocks. Disable Superblocks, that is, configure the **EnableReadGrouping** property as False, and manually group all Tags
- Devices that do not support block readings. **Solution:** Disable Superblocks, that is, configure the **EnableReadGrouping** property as False, and define simple Tags
- Devices that only allow defining blocks in pre-determined addresses and with fixed sizes. **Solution:** Disable Superblocks, that is, configure the **EnableReadGrouping** property as False, and define simple Tags, or PLC Tags in **Elipse SCADA**, or Blocks according to a device's specification

Manual Grouping

Usually, the larger the grouping of variables in blocks, the less reading requests are needed to complete a scan cycle of application Tags, thus increasing Tag's update speed. For this reason, if it is not possible to use automatic grouping, or

Superblocks, it is preferable to create Block Tags containing as many variables as possible, instead of creating simple Tags, or PLC Tags in **Eclipse SCADA**.

Notice that, due to the **Automatic Block Partition** feature, there is no need to prevent exceeding protocol's maximum limits, as long as a device supports **protocol's default maximum limits**. If this device does not support these limits, but defines fixed limits, valid for all supported Modbus functions, users must configure the **Customize Max. PDU Size** option on the **Modbus** tab.

If a device supports different limits for each supported function, automatic partitioning can be also unfeasible. In these cases, an application developer must also consider device's limits, and define blocks respecting these limits.

For a manual grouping, using **user-defined data types** can increase possibilities of grouping, by allowing to gather on a single Block Tag variables from the same addressing space, that is, a single Modbus function, but with different data types. In this case, the defined structure may have elements with different data types.

For more tips, please check topic **Optimization Tips**. The article *KB-23112* in **Eclipse Knowledgebase** presents a summary of questions related to Tag grouping and block resizing in this Driver, discussed here and on other topics.

Configuring Special Tags

In addition to **I/O Tags**, this Driver also supports a few Special Tags that allow an application to trigger features not related to data reading or writing. Unlike I/O Tags, Special Tags do not reference **Driver operations** in **numerical configuration**. The topics **Forcing a Wait Silence** and **Reading the Last Exception Code** describe in details all supported Special Tags.

Forcing a Wait Silence

Special Tag used to discard all pending data from communication until it finds a time-out, indicating that there is no more data to receive.

This service can be configured on the **Modbus** tab, to occur whenever detecting a communication error. With this Tag, however, users can execute this service at any time by the application.

This Special Tag is executed using a Tag's writing command. Its value, written to the Tag, is ignored.

String Configuration

- **Device:** Not used, this field must be kept blank
- **Item:** "ForceWaitSilence"

Numerical Configuration

- **N1:** Not used, can be left in 0 (zero)
- **N2:** 9001
- **N3:** Not used, can be left in 0 (zero)
- **N4:** Not used, can be left in 0 (zero)
- **Value:** Not used, can be left in 0 (zero)

Reading the Last Exception Code

As already mentioned on this Manual, Special Tags for reading the last exception code are used to read the last exception code sent from a certain slave device.

Such codes are automatically stored by this Driver in internal registers, which can then be accessed using this Tag. In addition, at each successful communication with a certain device where no exception was returned, this Driver automatically zeroes the associated register.

Exception Codes

Exception codes are used by a slave device (PLC) to report a failure when executing a certain function. Slave devices do not return exceptions in case of communication failures, a situation where these devices simply do not respond. Exception codes are returned by slaves in situations where a master request, such as the case of this Driver, was successfully received, but could not be executed for any reason, such as trying to read or write to a non-existent register. In this case, the returned exception code indicates the type of error occurred, that is, the reason that this Driver's request, although correctly received, could not be completed.

The specification of Modbus protocol defines nine exception codes. The list of protocol's default exceptions can be checked on topic **List of Default Exceptions of the Modbus Protocol**. In addition to these codes, some manufacturers define additional codes, specific to their devices. Such codes must be documented on a device's manual. If they are not, please check with manufacturer's technical support.

String Configuration

- **Device:** Numerical value of device's Id (*Slave Id*) followed by a colon, such as "1:", "2:", "3:", and so on
- **Item:** "LastExceptionCode"

Numerical Configuration

- **B1:** Slave device's address (*Slave Id*)
- **B2:** 9999
- **B3:** Not used, can be kept in 0 (zero)
- **B4:** Not used, can be kept in 0 (zero)

Values returned on Block Elements:

- **Element 1 (index 0):** Exception code returned by a device. For more information, please check topic **List of Default Exceptions of the Modbus Protocol**
- **Element 2 (index 1):** *N2/B2* parameter of the I/O Tag generating this exception
- **Element 3 (index 2):** *N3/B3* parameter of the I/O Tag generating this exception
- **Element 4 (index 3):** *N4/B4* parameter of the I/O Tag generating this exception
- **Element 5 (index 4):** *Size* parameter of the I/O Tag generating this exception
- **Element 6 (index 5):** *Device* parameter of the I/O Tag generating this exception
- **Element 7 (index 6):** *Item* parameter of the I/O Tag generating this exception

Using a Special Tag

The most common usage for this Tag during a normal scan of function Tags is via an exception Tag's **OnRead** event. In this case, a script must first reject null values, because these values indicate that exceptions were not received. Next, users

can handle that exception by executing the adequate procedures, according to the received code. It is advisable to zero the exception register when leaving a script, to indicate that this exception was already handled. The next example is written in Elipse Basic (**Elipse SCADA**).

```
// TagExc Tag's OnRead event
// Note: For this example, consider TagExc
// with automatic reading and writing enabled

If TagExc == 0
    Return
EndIf

If TagExc == 1
    ... // Handles exception 1
ElseIf TagExc == 2
    ... // Handles exception 2
Else
    ... // Handles all other exceptions
EndIf

TagExc = 0 // Zeroes the exception register
```

The next example is written in VBScript (**Elipse E3** or **Elipse Power**).

```
' TagExc Tag's OnRead event
' Note: For this example, consider TagExc
' with automatic reading and writing enabled

Sub TagExc_OnRead()
    If Value = 0 Then
        Exit Sub
    End If

    If Value = 1 Then
        ... ' Handles exception 1
    ElseIf Value = 2 Then
        ... ' Handles exception 2
    Else
        ... ' Handles all other exceptions
    End If

    Value = 0 ' Zeroes the exception register
End Sub
```

In writing operations by script, on the other hand, where users must test for returned exceptions right after sending a command, users must first zero the exception register. That avoids an eventual exception provoked by a writing command to be confused with another pre-existing one. Execute the writing operation and test a Special Tag's value, which must return 0 (zero) if no exception was received. In case it returns a value different from 0 (zero), then users can properly handle that received exception. The next example is written in Elipse Basic (**Elipse SCADA**).

```
// Note: For this example, consider TagExc
// with automatic reading and writing enabled
// and TagVal with automatic writing disabled

TagExc = 0 // Zeroes the exception register

TagVal.WriteEx(10) // Writes the value 10

If TagExc <> 0
    ... // Handles this exception
EndIf
```

The next example is written in VBScript (**Elipse E3** or **Elipse Power**).

```
' Note: For this example, consider TagExc
' with automatic reading and writing enabled
' and TagVal with automatic writing disabled

' Zeroes the exception register
Application.GetObject("Tags.TagExc").Value = 0

' Writes the value 10
Application.GetObject("Tags.TagVal").WriteEx(10)

If Application.GetObject("Tags.TagExc").Value <> 0 Then
    ... ' Handles the exception
End If
```

NOTE

This Special Tag returns, in addition to an exception code returned on Element 0 (zero), also Tag parameters whose communication provoked that exception. If this information is not needed, users can read the same register using a simple Tag, or a PLC Tag in **Eclipse SCADA**, without using a Block Tag. In this case, all recommended procedures remain the same.

Mass Memory Reading

This Driver allows defining, on operations, special reading functions for collecting mass memory from slave devices. Such functions do not exist in the protocol, and imply in using specific algorithms for reading events from devices, which may read or write in several registers, by using one or more protocol functions.

Callback Readings

Starting with version 2.08, this Driver implements callback readings, a feature available in **E3**, starting with version 3.0, and in **Eclipse Power**, which optimizes performance of mass memory readings. With this feature, an application delegates to this Driver a Tag scan for reading mass memory events. In other words, an application does not need to keep asking this Driver at each scan period. Instead, this Driver performs a verification of new events on a device and collects events as soon as they become available, and sends them to the application.

Special Functions for Mass Memory Readings

On the current version of this Driver, the following functions for reading sequences of events (SOE) are supported:

- **GE SOE:** Performs event collecting from GE PAC RX7 PLCs. For more information, please check topic **Reading an Event Buffer from GE PAC RX7 Controllers**
- **SP SOE:** Collects events from Schneider Electric SEPAM series 20, 40, and 80 relays. For more information, please check topic **Reading Events from Schneider Electric Relays from SEPAM 20, 40, and 80 Series**
- **GenSOE:** This function uses a generic Algorithm created by **Eclipse Software**, **Eclipse Modbus SOE**, which can be used by most Programmable Controllers. It requires the creation of an analogous programming procedure on PLC's programming (*ladder*). For more information, please check topic **Eclipse Software's Generic SOE Reading Algorithm**
- **65 03:** Special function for reading mass memory events from ABB MGE 144 meters. For more information, please check topic **Reading Mass Memory Registers from ABB MGE 144 Meters**

Reading an Event Buffer from GE PAC RX7 Controllers

An event buffer can be read using **Event-reported Tags**, **Event-reported Tags by point**, and **Real-time Tags**.

Event-reported Tags

Event-reported Tags return, at each reading operation, all events stored in this Driver's internal buffer, and they can be **configured by Strings** or **numerically**.

String Configuration

- **Device:** "<slave id>:"
- **Item:** "gesoe0.<Base address of an event stack>"

Numerical Configuration

To read an event buffer from GE PAC RX7 using numerical configuration, users must define, on this Driver's configuration window, an operation that uses as its reading function the **special function GE SOE**. Its data type must be defined as **GE_events**.

- **B1:** Slave ID
- **B2:** Code of the operation defined with **GE SOE** function
- **B3:** 0 (zero)
- **B4:** Base address of PLC's event stack

At each scan on this Tag, this Driver checks if there are events on controller's buffer. If there are events, this Driver starts an event-reading thread, which is executed in background, not blocking the scan of all other Tags. After finishing the reading of this Driver's buffer, this event-reported Tag returns the set of events read on that scan.

Returned events generate a sequence of **OnRead** events on this Tag. For each read event, **E3** or **Elipse Power** updates Tag fields, the Element values and timestamps, with values from a certain event, and calls the **OnRead** event once. This event's script must be defined by users, and it is generally used to insert Tag's data in a Historic.

Every event is represented by a Block with two Elements, with its **Timestamp** field read from a device. Fields from the respective reading Block Tag are displayed on the next table.

Block Tag fields

OFFSET	MEANING	DATA TYPE	RANGE OF VALUES
0	Point identification	Byte	Between 0 (zero) and 15
1	Point status	Byte	Between 0 (zero) and 1 (one)

For more information about event-reported Tags, please check the specific topic on *E3 User's Manual*.

IMPORTANT

When reading mass memory events in event-reported Tags in **E3** or **Elipse Power**, disable Tag's dead band, that is, the **EnableDeadBand** property configured as False, and also the linked Historic object, that is, the **DeadBand** property equal to zero, to avoid losing events with near values. It is also important to disable historic by scan, that is, in **E3** or **Elipse Power** configure the **ScanTime** property as 0 (zero). This ensures that new events are only stored using the **WriteRecord** method, executed on Tag's **OnRead** event, avoiding duplication of events.

Event-Reported Tags by Point

Starting with version 2.5 of this Driver, users can use a new Tag to download events from a specific point.

This Tag works as the previous one, except that it returns only events from a specific point.

Different from the previous event, the returned value only contains a single Element with a status value of a point, so that only one Tag can be used. This Tag must be configured as follows:

String Configuration

- **Device:** "<Slave Id>:"
- **Item:** "gesoe<200 + Point's index>.<base address of event queue>"

Numerical Configuration

- **N1:** Slave ID
- **N2:** Code of the operation defined with **GE SOE** function
- **N3:** 200 + Point's index, such as configuring the *N3* parameter as 202 for point 2 (two)
- **N4:** Base address of PLC's event stack

For more information about event-reported Tags, please check the specific topic on *E3 User's Manual*.

Real-time Tags

These Tags return the most recent event already read for a specific point. These events are stored on this Driver's internal memory for each PLC's event reading, with their respective timestamps read from the device. This Tag uses the following parameters:

String Configuration

- **Device:** "<Slave Id>:"
- **Item:** "gesoe<100 + Point's index>.<base address of event queue>"

Numerical Configuration

- **N1:** Slave ID
- **N2:** Operation Code
- **N3:** 100 + Point's index
- **N4:** Base address of PLC's event stack
- **Value:** Point's status

Reading Events from Schneider Electric Relays from SEPAM 20, 40, and 80 Series

To read SEPAM relays, the offset model of addressing must be configured as **Data is addressed from 0**, on the **Modbus** tab. Reading these events is performed using the following basic types of Tags:

- **Tag for collecting all table events (mandatory):** Performs a collect of all table events from a certain memory zone on a device. This Tag, in addition to returning all events read to an event-reported application, still stores events read in this Driver's internal buffer, to remove them using readings on Tags for reading a single event, described next
- **Tag for reading a single event (optional):** Returns events received from a specified relay, with a certain address, type, and zone. This Tag does not perform a direct reading from a device, but returns events from this Driver's internal buffer, stored during the reading of a Tag for collecting all table events, described previously, that is, to be able to read events with this type of Tag, a **Collecting all events**-type Tag must be already active, with its scan enabled. This Tag is useful when users need to get events from a specific type and source, such as relay, zone, address, and type. An example of usage is an association to Screen objects, displaying the status of a certain event address. Although this Tag returns the same information returned by the previous Tag, its usage prevents users from creating filters, **Select Case** clauses in VBScript, or any other method to separate several types of events returned by a **Tag for collecting all events by script** in an application

An application must implement a Tag for collecting all events for each table or event zone to collect on each relay, because it is during the reading of this Tag that actual events are collected from a device. Configuration for these two Tags is presented next.

Tag for Collecting all Table Events (Event Zone)

This is an event-reported Tag. Its typical usage is inserting events in a linked Historic, by using Historic's **WriteRecord** method, called on Tag's **OnRead** event. At each reading, that is, at each Tag's scan period, this Driver can collect up to 4 (four) new events from a device. This is the maximum number of events that each relay's event zone contains at each reading request.

As it is during the reading of this Tag that events are effectively collected from a device, even if its data is not used directly, that is, even if **there is no need to store all events in a Historic, its implementation is mandatory** for single-event Tags to return data. A Tag for collecting all events must be configured as a Block Tag with 3 (three) Elements, and with the following parameters:

String Configuration

- **Device:** "<slave id>:"
- **Item:** "spsoe<Zone or Event Table (1 or 2)>"

Example: To read Zone 1 (one) of Slave 1 (one), **Device** is equal to "1:" and **Item** is equal to "spsoe1". Alternatively, **Device** can be equal to "" and **Item** equal to "1:spsoe1". For more information, please check the topic **String Configuration**.

Numerical Configuration

To use numerical configuration, users must define, on the **Operations** tab, an operation that uses as its reading function the **special function SP SOE**. Its data type is automatically defined as **SP_events**, as soon as the **SP SOE** reading function is selected.

- **B1:** 1000 + Slave address (relay) on the network, between 1 (one) and 247

- **B2: Code of the operation** configured with **special reading function SP SOE**
- **B3:** 0 (zero)
- **B4:** Zone or event table (1 or 2)

The next table describes the meaning of the Block Elements, which have their values returned as reported by events.

Meaning of Block Elements (SP_events data type)

OFFSET	MEANING	DATA TYPE	RANGE OF VALUES
0	Type of event	Word	From 0 (zero) to 65535 (800H for Remote Annunciation, Internal Data , and Logic Input)
1	Address of the event	Word	References bit addresses from 1000H to 105FH
2	Ramp up or down	Word	00: Ramp down or 01: Ramp up

For more information about event-reported Tags, please check topic **Tags Reported by Events** on *E3 User's Manual*.

IMPORTANT

When reading mass memory events in event-reported Tags in **E3** or **Elipse Power**, disable Tag's dead band, that is, the **EnableDeadBand** property configured as False, and also the associated Historic object, that is, the **DeadBand** property equal to 0 (zero), to avoid losing events with close values. It is also important to disable historic by scan, that is, in **E3** or **Elipse Power** configure the **ScanTime** property as 0 (zero). This ensures that new events are only stored using the **WriteRecord** method, executed on Tag's **OnRead** event, avoiding duplication of events.

Tag for Reading a Single Event

This is also an event-reported Tag, and users can use its **OnRead** event for storage in a Historic. Notice that this does not prevent it to be treated as a normal Tag, or a real-time Tag, in case only its most recent value is relevant. As this Driver only reads an internal buffer, it is advisable to define a very low scan time, even lower than the one from the other type of Tag. CPU consumption at each scan can be considered as not significant. It is suggested to configure it as half the scan period for a Tag for collecting all events.

As already mentioned, this Tag is used to get a status for a certain event address, without parsing or performing filters on events that arrive by the previous Tag, by script, or by any other means. A typical application would be linking it to Screen objects.

A Tag for reading a single event, as already mentioned, does not perform a reading of events from a device, but from this Driver's internal buffer, previously filled during the reading of a Tag for collecting all events. This Tag returns a single Element, an event-reported one, and it can be configured as a simple Tag, which does not need to be a Block Tag. This Driver accepts up to eight events accumulated by event point, that is, for each combination of relay, zone, type, and event address in its internal buffer. If there is an overflow, that is, if more than eight events from a single point are returned without any single-event Tag collecting them, this Driver starts to discard older events. The correct configuration of a scan time may prevent data loss.

TIP

It is recommended to configure a scan for single-event Tags with a value equivalent to half the configured value for the Tag for collecting all associated events, thus avoiding the loss of events by overflow of this Driver's internal buffer.

This Tag must be configured with the following parameters:

String Configuration

- **Device:** "<slave id>:"
- **Item:** "ptspsoe<Event type (800H by default)>.<Event bit address> + Event zone offset* (please check the next table)>"

Example: To read **800H**-type events at address 1 (one) of zone 2 (two), **Device** must be equal to "1:" and **Item** must be equal to "ptspsoe&h800.&h8001". Alternatively, **Device** can be equal to "" and **Item** equal to "1:ptspsoe&h800.&h8001". For more information, please check topic **String Configuration**.

Numerical Configuration

- **N1:** Slave address (relay) on the network, between 1 (one) and 247
- **N2: Code of the operation** configured with **special reading function SP SOE**
- **N3:** Event type (0800H by default, according to manufacturer's documentation)
- **N4:** Event address (Event bit address) + Event zone offset, as described on the next table

Options for event address (Events Zone Offset)

EVENT ZONE	EVENT ZONE OFFSET
1	0 (zero)
2	8000H (8000 in hexadecimal)

Examples:

- **Event Address:** 102FH, Event Zone = 1 ® N4 = 102FH + 0 = 102FH
- **Event Address:** 102FH, Event Zone = 2 ® N4 = 102FH + 8000H = 902FH

NOTE

To represent values in hexadecimal in **Elipse E3** and in **Elipse Power**, users must use the prefix "&H", such as &H10 = 16. In **Elipse SCADA**, use the suffix "h", such as 10h = 16. On this Manual, however, the uppercase suffix "H" is used to indicate values in hexadecimal format.

- **Value:** Returns ramp up or down, as described on the next table

Available options for Value

VALUE	MEANING
00	Ramp down
01	Ramp up

- **Timestamp:** The **Timestamp** property represents the date and time an event was actually read from a relay, during the reading of a Tag for collecting all events previously described

For more information about relay events, their meanings and addressing, please check manufacturer's documentation. For more information about event-reported Tags, please check topic **Tags Reported by Events** on *E3 User's Manual*.

Eclipse Software's Generic SOE Reading Algorithm

The Modbus protocol does not define a default method for reading events from a device. For this reason, it is a common procedure that manufacturers create their own algorithms for reading events from devices that support Modbus protocol.

The generic algorithm for Sequencing of Events (SOE) of this Driver (**Eclipse Modbus SOE**) was developed by **Eclipse Software** to provide a default alternative for reading events from programmable controllers that do not have a native version of this feature, provided that these controllers meet some basic requirements of memory space and programming features, and also allow the creation of tables and control registers described later.

This algorithm allows storing and reading events from almost all programmable controllers, in an optimized way, by using features already implemented and validated in this Driver.

Event reading on this Driver follows a standard procedure, defined by **Eclipse Software**, reading events from **tables created in PLC's memory** or in slave devices by its resident application (*ladder*).

To use this algorithm, users must define Tags using this Driver's **special function Gen SOE**, which can be performed both using the old **numerical configuration**, the *N* and *B* parameters, or the new **String configuration**, the **Device** and **Item** fields. Tag configuration is described later on topic **Acquisition Procedure in an Application**.

During the process of reading events, the **special function GenSOE** always uses the Modbus function **03** (*Read Holding Register*) to read registers from a device. For writing during the update of control registers, the default function used is Modbus function **16** (*Write Multiple Registers*). By using the numerical configuration, users can select the writing function **06** (*Write Single Registers*), in the rare case of devices that do not support function **16**, because the opposite is more common, by using operation's **Write** field on the **Operations** tab.

PLC's resident software, ladder or equivalent, must keep updated all control registers that provide information to this Driver, such as the number of events available for reading and the address of the last register to read.

A device can keep more than one **event table**, in different memory addresses, containing different data types. Each table must be preceded by their respective **control registers**, in adjacent addresses. This table is formed by a **circular buffer** in contiguous addresses, accumulating events or data for collecting by this Driver at each collect procedure or download of events.

Users can define distinct data formats, or events, for each defined table, which are usually defined as a **data structure**, and may contain event's timestamp field. Events can also be defined using a **native data type** of this Driver. In this case, users cannot define a **Timestamp** field in the PLC, because the timestamp is sent with the date of reading, and the event contains a single field, which can be represented by a simple Tag or a PLC Tag in **Eclipse SCADA**.

NOTE

The SOE algorithm always uses Modbus protocol's function **03** (*Read Holding Registers*) to read registers from a device. For writing registers, the default Modbus function used is **16** (*Write Multiple Registers*). Users can also select function **06** (*Write Single Register*) only on **numerical configuration**, by using the **Write** field of the respective operation, on the **Operations** tab.

The topics **Event Table**, **Acquisition Procedure in a PLC**, and **Acquisition Procedure in an Application** describe in details this algorithm, its implementation in a PLC software (ladder), and how to perform its reading by using this Driver's Tags, respectively.

Event Table

As already mentioned on topic **Elipse Software's Generic SOE Reading Algorithm**, each event table keeps events in a circular buffer. The circular buffer of each table is defined by its initial address, or base address, contiguous to control registers, and by its maximum number of registers, which defines its final limit. The following table shows a layout of events inside a table's circular buffer.

Layout of events in a circular buffer

EVENT	TIMESTAMP	ELEMENT1	ELEMENT2	ELEMENT3	...	ELEMENTN
1						
2						
3						
4						
5						
...						
N						

Each row on the previous table represents a stored event, usually represented by a structure, or by **User-Defined Data Types**.

Notice that, in the example of the previous table, the first element of that event structure is a timestamp. This field, whose presence is not mandatory and that does not need to appear in the first position, defines Tag's **Timestamp** property and it is not returned in its Elements. For more information, please check topic **User-Defined Data Types**.

Users can also define events with **native data types of this Driver**, and in this case there is only one data Element in each event, without timestamp.

Events must be inserted in this circular buffer in ascending order, returning to the base address after reaching the upper limit of this circular buffer. The following control registers must be defined for each table:

- **Table status:** Must be kept by the PLC, indicating the number of events available for reading in the circular buffer. It must be updated by the device whenever new events are added to the circular buffer, or after finishing the collecting of events by an application, which can be detected via **Acquisition status**
- **Recording pointer:** This value indicates an index, starting at 0 (zero), of the position where a device must insert the next event. It must be incremented by the device after each new event insertion in the circular buffer, moving back to the base address after reaching buffer's upper limit. Notice that this value must not be provided in units of Modbus registers, but in event positions, and it must be incremented by one unit at every new event inserted, regardless of the number of Modbus registers occupied for each event in the circular buffer. With this, the maximum allowed value for this pointer is given by the formula **MaxWritePtr = (Size of the circular buffer / Size of event structure) - 1**

- **Acquisition status:** Indicates the number of registers already read by this Driver at each individual event reading. After each reading, this Driver writes in this register the number of registers it successfully read. Slave's resident application, or ladder, must immediately subtract the value written by this Driver from **Table status** and then zeroes **Acquisition status**
- **Reserved:** This register is currently not used. It may be used on future versions of this Driver, and it can be kept in 0 (zero) in its current version

As already mentioned, the base address of this circular buffer, that is, the address in which an event table starts, must be contiguous to control registers.

Control registers, on the other hand, must be also set on contiguous addresses, on the same order presented previously, allowing their reading on a single operation, that is, assuming that the base address of control registers for a certain table is 100, the next table contains the addresses for the other registers.

Register addresses

REGISTER	ADDRESS
Table Status	100
Recording Pointer	101
Acquisition Status	102
Reserved	103
Circular Buffer's Base Address	104

The topic **Acquisition Procedure in a PLC** describes a step-by-step acquisition procedure or algorithm based on a slave device (PLC). On the next topic, **Acquisition Procedure in an Application**, there is a discussion on how to configure an application for acquiring table events.

Acquisition Procedure in a PLC

This topic presents a discussion about an algorithm for collecting events from the point of view of a PLC or slave device. Its goal is to clarify to developers what must be implemented in PLC's resident application, or ladder.

A device must start inserting events in ascending order, starting at table's base address, that is, starting at the circular buffer. For each new event inserted, the Recording Pointer must be incremented, pointing to the next available position in that buffer.

This Driver performs a reading starting from the oldest to the newest event. The starting address of this reading is calculated by this Driver using the value of Recording Pointer and Table Status.

If the number of available events is greater than protocol's **maximum allowed into a single communication frame**, this Driver performs multiple block readings, updating the value of Acquisition Status after finishing this process with the total amount of events read.

NOTE

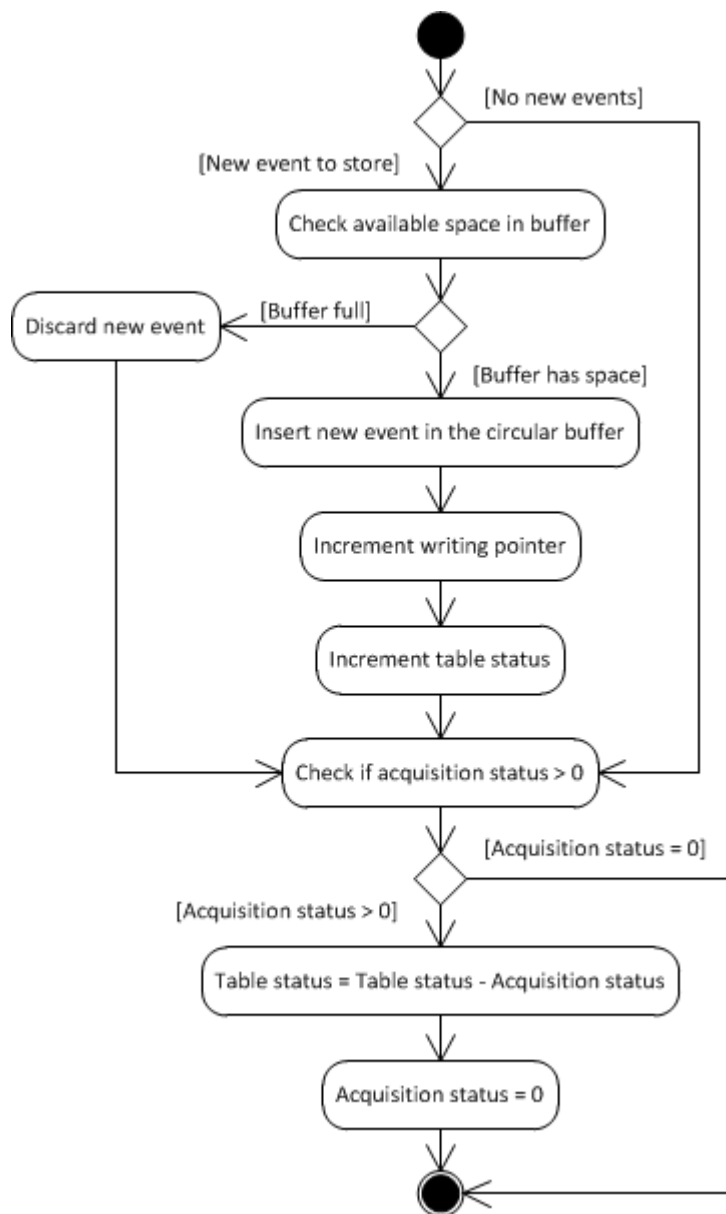
If a device does not respect the default limit of 253 bytes for PDU, then users must configure the **Customize Max. PDU Size** option, on the **Modbus** tab, according to supported limits, which must be described on manufacturer's documentation.

When detecting a non-null value written by this Driver to Acquisition Status, a PLC's or device's application must immediately subtract Acquisition Status' value from Table Status' value and then zeroes Acquisition Status. With Acquisition Status zeroed again, this Driver can start a new acquisition at any time.

A PLC can insert new events to a table during PLC's acquisition process, as long as there is no overflow on circular buffer, that is, as long as its writing pointer does not exceed its reading pointer, by incrementing Table Status.

An event collecting or downloading procedure is finished when Table Status is zeroed. All collected events are then provided to an application via event-reported Tags, as described on the next topic.

The next figure shows a flow chart, as a UML Activity Diagram, with a suggestion of implementation for a PLC logic. Notice that some variations are possible, for example discarding the oldest event in case of overflow, which can be evaluated by developers, depending on the context.



UML Activity Diagram (PLC)

Timestamp

As already mentioned, every event is composed by a structure containing one or more data elements, usually, but not necessarily, represented by **User-Defined Data Types**.

If structures or user-defined data types are used, then users can associate a timestamp to each PLC-provided event. In this case, the value of the **Timestamp** field must be provided in a structure field, in PLC memory, in the order it was

declared in the configuration file, and its value is not displayed in any Block Element, it is only returned in the **Timestamp** property of the linked Tag.

As explained on topic **User-Defined Data Types**, any date and time type supported by this Driver can be used. The **GenTime** data type, however, was specially created for use with **Eclipse Modbus SOE**, due to an easy definition in PLC's resident application, or ladder.

If a millisecond precision is needed, another option is consider this Driver's **UTC32** data type, represented as an integer with only 32 bits, or 4 (four) bytes, with seconds starting at 1/1/1970, without a milliseconds representation, considered as 0 (zero).

The next topic, **Acquisition Procedure in an Application**, describes how to configure an application for collecting events accumulated in a PLC or programmable slave device.

Acquisition Procedure in an Application

This topic contains a detailed explanation on the configuration of an application to acquire events accumulated in a PLC or programmable slave device.

Reading events in an application is performed using Tags that reference the special reading function **Gen SOE**. Tag's data type defines the structure of events stored in the device's event table. If a **native data type of this Driver** is defined, or built-in type, each event contains only one element of this data type, without a PLC-provided timestamp, because the timestamp represents the instant events were collected. On the other hand, if **user-defined data types** are used, then users can define structures for events, including timestamps, as explained later on this topic.

Next, there is a description of the configuration of Tags using the new methodology of configuring by **Strings**, the **Device** and **Item** fields, as well as the old numerical configuration used by **Eclipse SCADA**, the *N* and *B* parameters.

String Configuration

- **Device:** "<Slave Id>:"
- **Item:** "elsoe<N>.<start add.>[.<type>[<type size>]][.<byte order>][<bit>]"

Where:

- **N:** "<Slave Id>:"
- **start add.:** Address of the first control register, using the value defined on the example table of topic **Event Table**
- **type:** Native or user-defined data type used for each event. For more information, please check topic **String Configuration**
- **type size:** Used only for variable-size data types. For more information, please check topic **String Configuration**
- **byte order:** Byte ordering. It can be omitted for devices fully compliant with protocol's standard. For more information, please check the **Byte Order** item on topic **String Configuration**. When structures are used, it only affects their individual elements. For more information, please check topic **User-Defined Data Types**
- **bit:** Bit masking. Usually can be omitted, it would hardly be used here. For more information, please check the **Bit** field on topic **String Configuration**

Example:

- **Device:** "1:"
- **Item:** "elsoe150.&h101.TYPE3"

The **TYPE3** data type is defined as follows in this Driver's default example file. For more information, please check topic **User-Defined Data Types**.

```
// This type has an UTC32-type timestamp
// and a few named elements
struct TYPE3
{
    DefaultAddress = 0x101;
    timestamp = UTC32;
    float Va;
    float Vb;
    float Vc;
    float Ia;
    float Ib;
    float Ic;
}
```

This is a **Structure** data type containing 6 (six) data fields and a timestamp. Therefore, this Tag must be a Block with 6 (six) Elements to represent that structure.

Notice that, as already explained, the value of a timestamp, although it occupies registers in the PLC, does not need Block Tag Elements, because its value is returned in the Tag's **Timestamp** property.

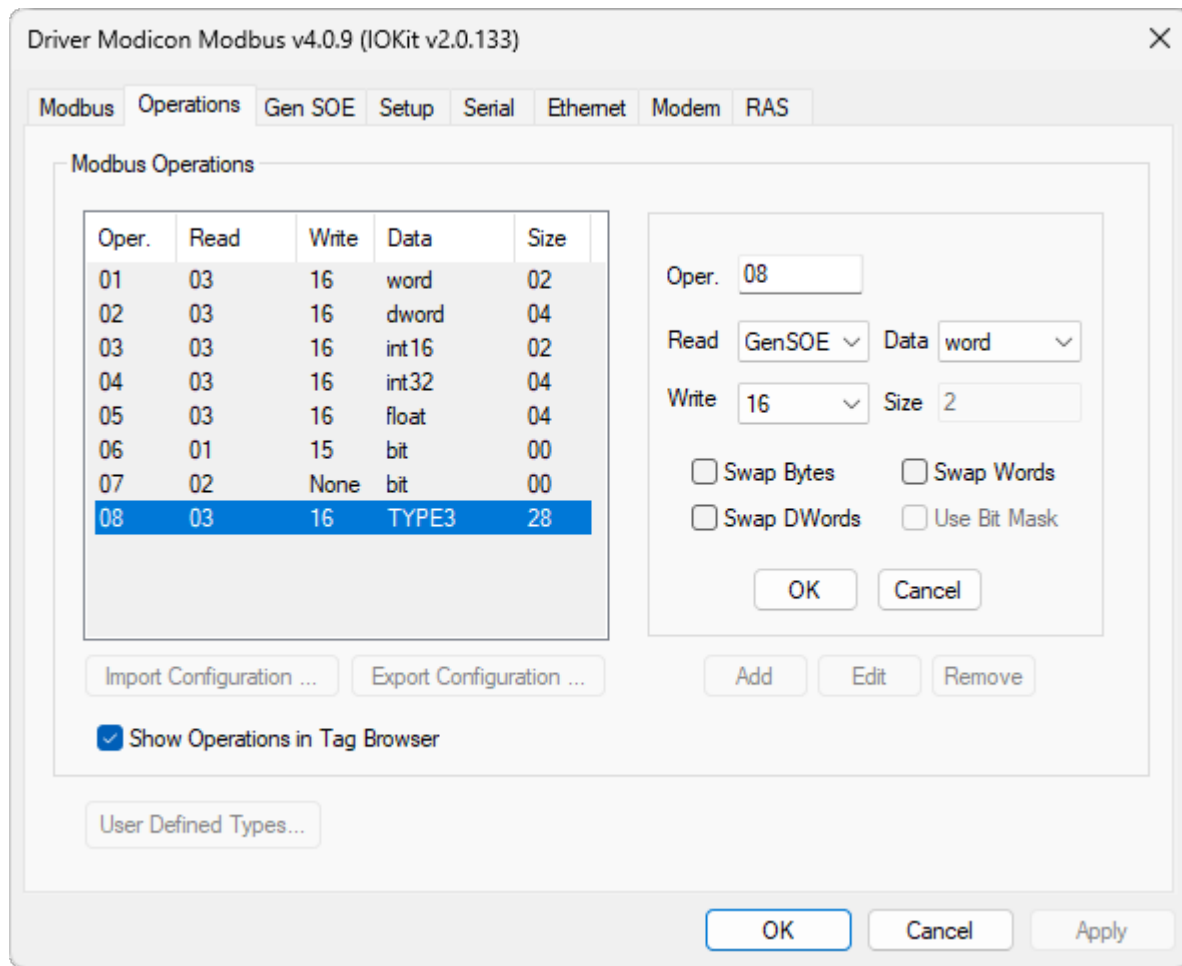
NOTE

The *N* parameter informs the size of a table as the maximum number of events, not as Modbus records. Together with the *Start Address* parameter, it indirectly informs the final address or upper limit of that table. The size of a table's data area, therefore, in number of Modbus registers, is the product of *N* by the size of each event in number of Modbus registers, that is, in 16-bit **Words**.

Numerical Configuration (N and B Parameters)

To configure Tags for reading Elipse SOE using a numerical configuration, users must configure an operation on the **Operations** tab using the **special function GenSOE**.

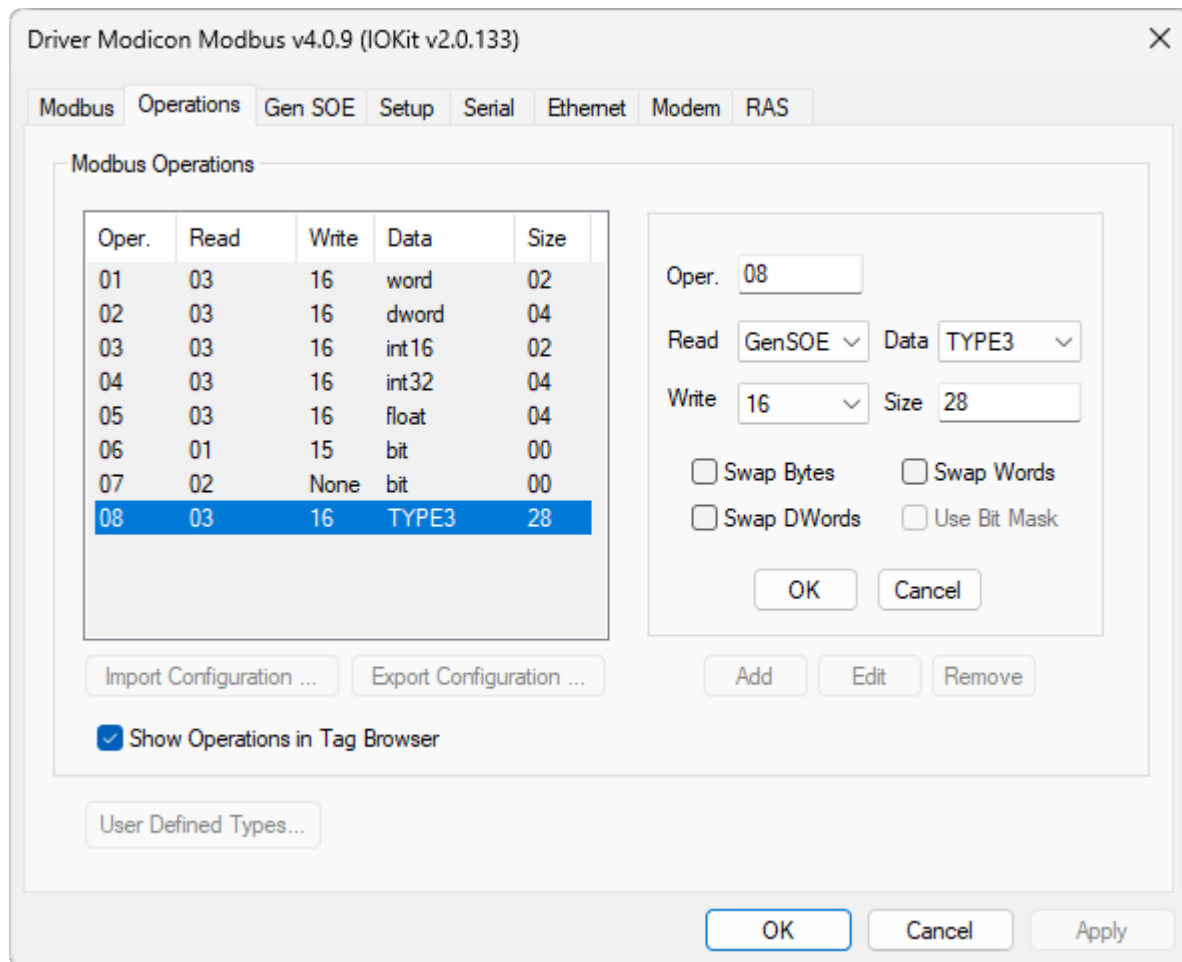
The next figure shows an example of a new operation added using special function **GenSOE** with a **Word** data type.



Special function GenSOE

Notice that function **16** (*Write Multiple Registers*) was selected as its writing function, which is the most commonly used function. However, users are encouraged to use function **06** (*Write Single Register*) whenever supported by a device.

The next figure shows the same operation with a user-defined data type **TYPE3**, which is a data type defined in the example configuration file, available with this Driver and used as an example on this topic. For more information, please check topic **User-Defined Data Types**.



Configuration using the GenSOE function and a user-defined data type

Data type **TYPE3** is defined in this Driver's example file as follows:

```
// This type has an UTC32-type timestamp
// and a few named elements
struct TYPE3
{
    DefaultAddress = 0x101;
    timestamp = UTC32;
    float Va;
    float Vb;
    float Vc;
    float Ia;
    float Ib;
    float Ic;
}
```

This is, therefore, a **Structure** data type with 6 (six) data fields and a timestamp, and the default address, the Tag's *B4* parameter, equal to "101H", or 257 in decimal. To read it, users must define a Block Tag with 6 (six) Elements and the following configuration:

- **B1**: Slave device's address (PLC) on the network (*Slave Id*)
- **B2**: 8 (a previously defined operation with special function **Gen SOE**)
- **B3**: N (size of a device's table, as the maximum number of events that table can contain)
- **B4**: 100 (address of the first control register, using a value defined on the example table of topic **Event Table**)
- **Size**: 6 (six)

NOTE

The *B3* parameter informs the size of a table as the maximum number of events, not as Modbus records. Together with the *B4* parameter, it indirectly informs the final address or upper limit of that table. The size of a table's data area, therefore, in number of Modbus records, is the product of *B3* by the size of each event in number of Modbus records, that is, in 16-bit **Words**.

Notice that, if **E3's** or **Elipse Power's** Tag Browser is used to insert a Tag in an application, as explained on topic **User-Defined Data Types**, Tag Elements are already named according to the name given to structure elements when they were declared. Tag Browser can be opened by clicking  on this Driver's **Design** tab.

Usage

Once defined an appropriate Tag, or Tags, enable its scan and let this Driver collect events from their respective table, whenever new events are detected.

Tags linked to the **GenSOE** function, or **elsoe** when configuring by **Strings**, are always **event-reported**. This means, as already explained on topic **User-Defined Data Types**, that this Driver can return several events on a single reading operation, that is, on a single interval of a Tag's scan.

This means that this Driver returns a set of events, or for the previous example sets of blocks with 6 (six) data fields and a timestamp, at once, which produces a sequence of **OnRead** events on a Tag, one for each event, or a block with 6 (six) data fields and a timestamp, returned by this Driver.

For detailed instructions on the right way to handle event-reported Tags, please check topic **Tags Reported by Events** on *E3 User's Manual*. **Elipse SCADA User's Manual** also contains an analogous topic.

In short, the usual way of handling event-reported Tags is by adding a call to the **WriteRecord** method of a previously linked Historic object on Tag's **OnRead** event, ensuring that all events reaching this Historic are registered. In this case, this Historic must be configured without a dead band, that is, the **DeadBand** property set to 0 (zero), and disabling historic by scan, that is, in **E3** or **Elipse Power** the **ScanTime** property set to 0 (zero). Tag's **EnableDeadBand** property must also be configured to False.

IMPORTANT

When reading mass memory events in **E3's** or **Elipse Power's** event-reported Tags, disable Tag's dead band, that is, the **EnableDeadBand** property configured as False, and also in the linked Historic object, that is, the **DeadBand** property configured as 0 (zero), to avoid losing events with close values. It is also important to disable historic by scan, that is, in **E3** or **Elipse Power** the **ScanTime** property configured as 0 (zero). This ensures that new events are only stored using the **WriteRecord** method, executed on Tag's **OnRead** event, avoiding duplicated events.

Optimization and Compatibility

Some devices, such as PLCs by ATOS, do not support block readings using data types with different structures. In practice, this prevents this Driver to read data from control and event registers as a single block. To collect PLC events with these restrictions, users must disable the **Enable Control and Data Registers Grouping** option on the **Gen SOE** tab.

Reading Mass Memory Registers from ABB MGE 144 Meters

To read mass memory registers from ABB MGE 144 meters, users must configure Tags using the special reading function **65 03**, as described on this topic.

The special function **65 03** is ABB's proprietary and it is practically identical to protocol's standard function **03** (*Read Holding Registers*), differing only on returned data, referring to ABB meter's mass memory.

Data is returned as a **Word**, as in function **03**, with protocol's default byte order, or big endian. Therefore, there is no need to enable any swap function, *Swap Bytes*, *Swap Words*, or *Swap DWords*.

Meter's register map, specifying data to read as well as its correct configuration, must be checked on meter's manufacturer-provided documentation.

This Driver also contains special writing functions specific for this meter, functions **65 01** and **65 02**. For more information about those special writing functions, please check topic **Special Functions** and also device's documentation.

String Configuration

- **Device:** "<slave id>:"
- **Item:** "abbmge<address>[.<type>[<type size>]][.<byte order>][[/bit]]"

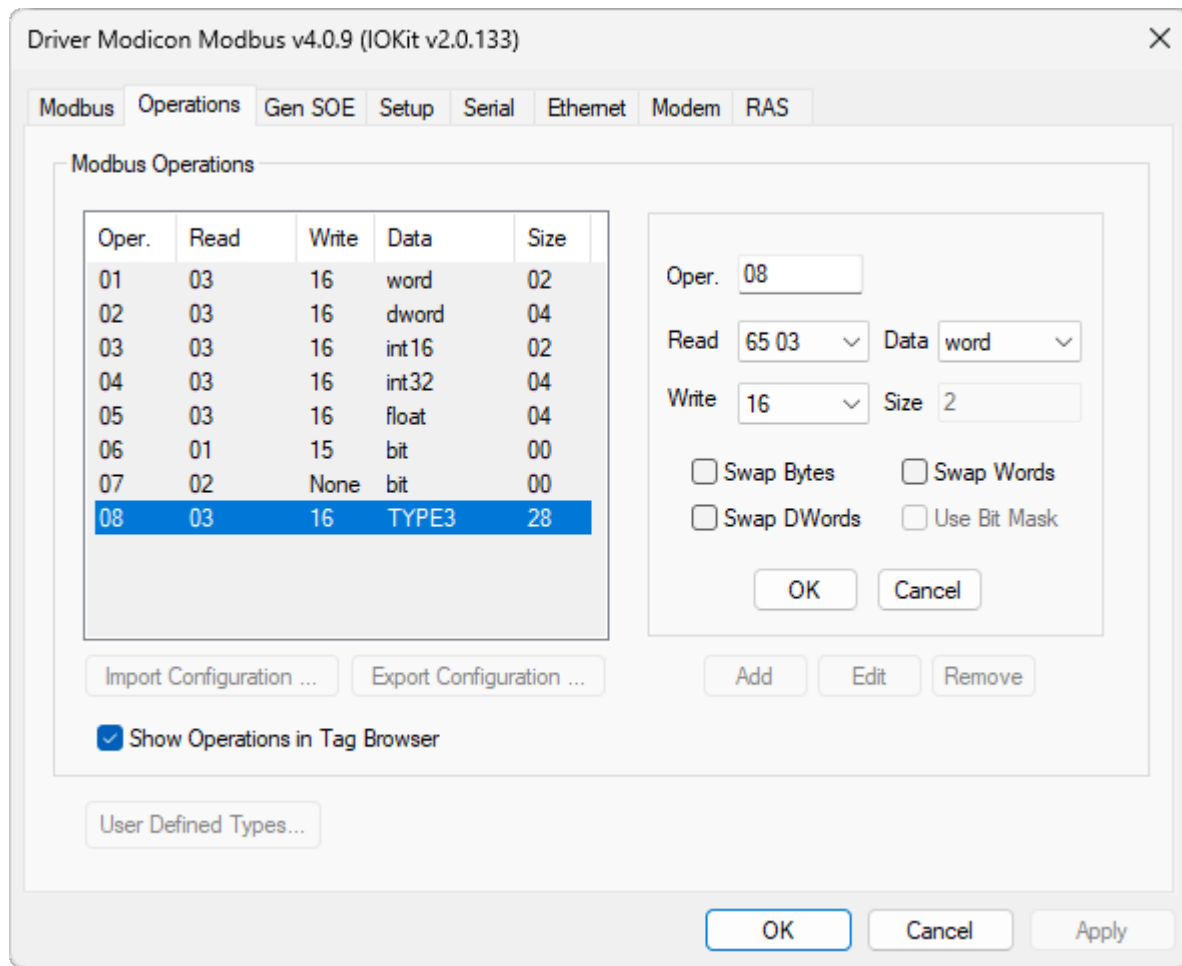
Where:

- **Address:** Address of the Modbus register to read
- **Type:** Data type. If omitted, assumes a default **Word**. For more information, please check topic **String Configuration**
- **Type size:** Used only for variable-sized data types. For more information, please check topic **String Configuration**
- **Byte order:** Byte ordering. If omitted, assumes protocol's default. For more information, please check topic **String Configuration**
- **Bit:** Bit masking. Usually omitted, so prefer application's bit masks. For more information, please check topic **String Configuration**

Numerical Configuration (N or B Parameters)

- **N1/B1:** Slave Id
- **N2/B2:** Code of the **operation** configured with function **65 03**. Please check more information next
- **N3/B3:** Not used, leave in 0 (zero)
- **N4/B4:** Register's address

To configure Tags numerically, users must first add an operation with function **65 03** on the **Operations** tab of this Driver's configuration window, as shown on the next figure.



Creating an operation with special function 65 03

Appendix

This section contains additional information about the Modbus protocol.

Optimization Tips

This topic enumerates some optimization tips to communicate with slave devices.

Tips for Configuring this Driver for E3 and Elipse Power

- Use Superblocks whenever possible, preferring to create simple Tags, or PLC Tags in **Elipse SCADA**, instead of Block Tags. For more information, please check topic **Superblock Reading (Grouping)**.
- If Superblocks are not an option, prefer to create Block Tags, grouping the largest possible number of variables in the smallest number of blocks. For more information, please check the text about manual grouping on topic **Superblock Reading (Grouping)**.
- Consider all recommendations of article *Performance tips for E3*, on **Elipse Knowledgebase**.
- In case of high-latency networks, reduced bandwidth, or packet loss, please also read article *E3's Network Settings for networks with high latency, low bandwidth and/or packet loss*.
- In high-latency networks, configure higher time-outs, considering the expected latency. Remember that a time-out is only effective on delays, it does not interfere with performance on normal usage situations.

Tips for Configuring this Driver for Elipse SCADA

- Prefer to create Block Tags, by grouping the largest possible number of variables in the smallest number of blocks. For more information, please check the text about manual grouping on topic **Superblock Reading (Grouping)**.
- Consider all recommendations of article *Developing applications with optimum performances*, also available on **Eclipse Knowledgebase**.
- In high-latency networks, configure higher time-outs, considering the expected latency. Remember that a time-out is only effective on delays, it does not interfere with performance on normal usage situations.

Tips for Configuring or Programming a Device

- If possible, group all variables defined by a resident application, or ladder, that have the smallest scan time, in contiguous addresses in PLC's memory. The total scan time of Tags highly depends on the capacity of grouping variables in communication blocks.

Frequently Asked Questions

Known peculiarities of Twido devices by Schneider

- The application is trying to read a **Float**-type value, but it is failing. The value displayed by this PLC is completely different from the one displayed by the application for the same address.
- **Answer:** This device does not use protocol's default byte order, or big endian. Users must configure a byte order by executing a swap of **Words**, which corresponds to the **b2** option on the **configuration by Strings**., or select the **Swap Words** option on the configuration of operations on **numerical configuration**. For more information, please check topic **Operations Tab**.
- The application is trying to read inputs and outputs of this PLC, but it is failing.
- **Answer:** This device does not allow reading or writing input and output variables, and users must use PLC's internal variables to perform this reading, that is, create a mirror of inputs and outputs in an area where this Driver has access. Users must also be careful to create a PLC routine to check when an application changes an output value, so that it is effectively activated or deactivated in this PLC.

Known peculiarities of MPC 6006 devices by Atos - Schneider

- The application is trying to read a **DWord**-type value, but the correct value is not displayed. The application displays values different from the ones in this PLC.
- **Answer:** Please check the article *Using Modbus Master (ASC/RTU/TCP) drivers with ATOS's controllers* on **Eclipse Knowledgebase**. If using the new **String** configuration, the **Device** and **Item** fields, also check section **Byte Order** on topic **String Configuration**. If using the old numerical configuration, the *N/B* parameters, also check topic **Operations Tab**, specially the **Byte Order** section.

How to join two Int16-type values in this PLC into an Int32-type value in the application?

- There is a 32-bit number stored in two 16-bit registers in this PLC. How to display this number on an application's screen as a single 32-bit register?
- **Answer:** Users must create Tags using 32-bit data types, such as **Float**, **DWord**, or **Int32** data types. When configuring an I/O Tag, users must inform the first address of each PLC variable. For more information, please check topic **Configuring an I/O Tag**. This Driver then joins two 16-bit registers from the device into a single 32-bit value, which is returned on Tag's **Value** field or on Block Tag's Element. If using a configuration by **Strings**, the **Device** and **Item** fields, inform a data type right after register's address. For more information, please check topic **String Configuration**. If using the old **numerical configuration**, the *N/B* parameters, users must define operations

with 32-bit data types. Notice that, on the **Operations** tab of the configuration window, 32-bit data types are always displayed with a four-byte size, the **Size** field. For more information, please check topic **Supported Data Types**.

- The application is already developed, but how to join values from two **Words** into a single Tag?
 - **Answer:** Users can execute that join operation using scripts, by creating an unsigned 32-bit integer. To do so, users must multiply the **Word** that contains the highest part of that **Word** by 65536 and then sum the **Word** that contains the lowest part of that **Word**. For example, **UInt32 = (HighWord × 65536) + LowWord**.
- The application wants to read **Float**-type values. A reading function **03** and a writing function **16** with a **Float** data type were configured. However, the application displays a value that is different from the one on this device.
 - **Answer:** The official Modbus protocol uses a big endian byte order, with the most significant bytes of every value coming first. If this Driver is reading nonsense values, even with its address correctly configured, probably the device uses a non-standard byte order. In this case, users must configure its swap options. If users are using a configuration by **Strings**, the **Device** and **Item** fields, please check the **Byte Order** section on topic **String Configuration**. If users are using the old **numerical configuration**, the *N/B* parameters, please check the **Byte Order** section on topic **Operations Tab** for more information on using these swap options.

How to communicate with more than one device on a Serial communication network?

- There is more than one device on a serial network, each one with a unique address. How to communicate with every one of them?
 - **Answer:** Users must be careful with the *Slave Id* of each **I/O Tag**, because this field indicates the device to communicate. On an RS485 serial network, all devices listen simultaneously to all driver requests, that is, there is a single bus, although only the one with the corresponding *Slave Id* responds to the request, because multiple devices with the same *Id* are not allowed. When configuring by **Strings**, this value can be provided in the **Device** field, or at the beginning of the **Item** field. For more information, please check topic **String Configuration**. If users are using the **numerical configuration**, this value is provided in the *N1/B1* parameter of each Tag. Users can use the same **operations** for Tags from several devices. A good reference for information and tips regarding the installation and maintenance of serial networks is the book *Serial Port Complete*, by Jan Axelson.
- There is more than one serial port on the computer. How to configure this Driver to communicate with devices connected to each one of these ports?
 - **Answer:** In this case, as there is more than one different physical layer, such as Serial 1, Serial 2, and so on, it is necessary as many I/O Drivers as the existing ports. Configurations for this Driver's **Tags** can be the same for all Driver objects or instances. The only difference is that one Driver must be configured to communicate via Serial 1 port, another Driver configured to communicate via Serial 2 port, and so on. Port configuration is performed on the **Serial** tab of this Driver's configuration window. For more information, please check topic **Properties**.

How to communicate with more than one device on a serial network with an RS485 converter?

- There is an RS485 network with several devices communicating via an RS232-RS485 converter using a Serial port. Whenever an address switches (*Slave ID*), that is, when this Driver requests data from another device, a time-out occurs. After retrying the same message, this device answers normally. Is there a way of preventing this time-out during an address or *Slave ID* switching?
 - **Answer:** Some RS232-RS485 converters require a time interval to switch, that is, commuting from transmission mode to reception mode, or vice versa. To circumvent this limitation, users can use the **Inter-frame delay** option on the **Serial** tab of **IOKit** library, available on the **configuration window**. This option defines a time interval between messages. The exact value of this interval depends on the converter in use but, if it is unknown, users are recommended to try values between 50 ms and 300 ms.

NOTE

The **Inter-frame delay** option of **IOKit** library may significantly degrade performance in some applications, and it must be used only when absolutely necessary. Please be sure that this converter is in good conditions, and if it effectively requires a delay. If needed, please check with manufacturer's technical support.

How to communicate with more than one device on an Ethernet network?

- There is more than one device connected to an Ethernet network, each one with a unique IP address. How to communicate with each one of them?
- **Answer:** Currently, for each IP address, users need as many I/O Drivers as the number of IP addresses to communicate. The configuration referring to this Driver **Tags** can be the same for all Drivers. The only difference is that one Driver must be configured to communicate with IP address 1 (one), another Driver must be configured to communicate with IP address 2 (two), and so on. The *Slave Id* parameter can be still used in **Modbus TCP** mode to differentiate devices connected to a Modbus Ethernet / RS485 gateway on the same IP address. Notice that this gateway not only must allow an interconnection among Ethernet and serial networks, but also convert **ModbusTCP** frames for the serial modes supported by devices, **ModbusRTU** or **ModbusASC**. The IP address must be configured on the **Ethernet** tab of **IOKit** library, on this Driver's **configuration window**.

TIP

Avoid using protocol's **RTU** or **ASC** mode encapsulated in **TCP/IP** layer. If users want to encapsulate serial communication of devices using **Modbus RTU** in **TCP/IP**, there are gateways available in the market that not only encapsulate serial communication in **Ethernet TCP/IP**, the physical, network, and transport layers, but also convert **Modbus RTU** into **Modbus TCP**, the application layer. As a last option, if it is inevitable to use **Modbus RTU** in **Ethernet TCP/IP** layer, enable the **Reconnect after Timeout** option, described on topic **Modbus Tab**.

Modbus Simulator Software

- Is there any software that simulates Modbus protocol and can be used for testing with this Driver?
- **Answer:** Yes, there are several alternatives. **Elipse Software** provides a free version (demo) of *Elipse Modbus Simulator* on its website, which allows simulating the most basic protocol features. There is also the possibility of using **Elipse Software**'s Modbus Slave Driver as an emulator. Another possibility is **Modsim**, one of the oldest and well known alternatives to emulate a slave Modbus device. This simulator can be purchased at <http://www.win-tech.com/html/modsim32.htm>. In addition to it, there is also a free and open alternative named **Free Modbus PLC Simulator**, available at <http://www.plcsimulator.org>. There are still many other options and a list with other software can be found at *Modbus.org* website.

How to configure the N4/B4 parameter of I/O Tags?

- Which address to use in the *N4/B4* parameter of an I/O Tag?
- **Answer:** This address varies from device to device. To know which is the exact address to use, please check device's documentation or contact its manufacturer's technical support. The topic **Addressing Tips (Modbus Convention)** provides tips on common additional offset conventions used by many manufacturers.

When to use RTS and DTR controls that appear on the Serial tab of this Driver's configuration window?

- The device is communicating directly on computer's RS232 serial port. How to configure **RTS** and **DTR** controls?
- **Answer:** Please check device's documentation or manufacturer's technical support for the correct configuration.

- The device is communicating through an RS232-RS485 converter connected to computer's RS232 serial port. How to configure **RTS** and **DTR** controls?
- **Answer:** When communicating with devices using RS232-RS485 converters, such configurations depend on the converter. The device, or Slave, does not influence it, as these signals only exist on the RS232 serial side, with no equivalent on the RS485 serial layer. The **RTS** control is commonly used by older converters to switch between transmission and reception modes, because RS485 is half-duplex, and in these cases it must be configured in **Toggle** mode. There are some rare devices that require other configurations. On most recent converters, however, switching between transmission and reception is automatic, and these signals in general are not used, and they may be ignored. For more information, please check converter's documentation or manufacturer's technical support.

When to use Swap Bytes, Swap Words, and Swap DWords options?

These options must be used for 16-, 32-, or 64-bit data types, whose byte order of this device-provided value does not correspond to Modbus default byte order, where the most significant bytes always come first, or big endian standard, also known as Motorola. If this Driver is reading nonsense values, or values different from the ones stored on the PLC, it may use a byte order different from protocol's default. For more information, please check section **Byte Order** on topic **String Configuration** or, if using the old **numerical configuration**, the *N/B* parameters, please check section **Byte Order** on topic **Operations Tab**. Users are also advised to check device's documentation.

- The application is trying to read a **Word** value, but this value appears different from the one configured in the PLC. If in the PLC it is configured as "1" (one), the application displays it as "256".
- **Answer:** Value 1 (one) in hexadecimal is 0001H and value 256 in hexadecimal corresponds to 0100H. This device has a non-standard byte order. Users must enable the **Swap Bytes** option, the "b1" option on the **String Configuration**, to read the correct value.
- The application contains a Tag configured to read a **DWord** value, but the value read by the application is different from the value stored in the PLC. When setting the value "258", for example, to a PLC register, the application displays a nonsense value of "16908288".
- **Answer:** The value 258 in hexadecimal is 00000102H and the value 16908288 in hexadecimal corresponds to 01020000H. This device has a byte order different from protocol's default, where the least significant **Word** comes first. In this case, users must enable the **Swap Words** option, the "b2" option on the **String Configuration**, to read the correct value.

How to correctly read Float data types from WEG TPW-03 PLCs?

- **Answer:** When configuring **I/O Tags**, users must enable the **Swap Words** option, which corresponds to the "b2" option on **String configuration**. If users are using the old numerical configuration, the *N/B* parameters, please check section **Byte Order** on topic **Operations Tab**.

Known peculiarities of devices from the ABB Advant Controller 31 series 90 family, such as ABB 07KT97

- An **E3** or **Elipse Power** application is trying to read registers or bits from a PLC, but there are always errors.
- **Answer:** Devices from this series do not allow using **E3** or **Elipse Power** Superblocks for the following reasons:
 - There are interruptions on the address map of device's registers, with undefined address intervals.
 - The maximum **PDU** size is different from the one established by protocol's default, and it is defined as a size that supports 96 **Words** or **Bits**. As this protocol groups eight bits at each data byte, that results in different maximum **PDU** sizes for reading functions for **Bits** and **Words**, which prevents customizing the maximum **PDU** size allowed by this Driver, which does not allow configuring different limits for each protocol function.

- **Solution:** Follow these steps:
 - Disable Superblock reading, by configuring this Driver's **EnableReadGrouping** property to False.
 - Prefer defining Block Tags, by grouping the largest possible number of variables in the smallest number of blocks, respecting device's limit of 96 **Words** or 96 **Bits** for each Block. For more information, please check section about **Manual Grouping** on topic **Superblock Reading (Grouping)**.

NOTE

Users can also use automatic grouping, or Superblocks, if there is no need to read **Words** and **Bits** on the same Driver object, obviously depending on the interval of addresses to read, more specifically, whether this interval contains interruptions or not. In this case, anyway, users must configure the **Customize Max. PDU Size** option on the **Modbus** tab, according to the limit of 96 **Words**, $96 \times 2 = 192$ bytes, or 96 **Bits**, $96 \div 8 = 12$ bytes. Such possibility can be carefully evaluated, in a case-by-case basis, by the application's developer.

The application is trying to read values with Float data types, and the following message appears on this Driver's log: "Warning: denormalized float number! Returning zero". What to do?

- **Answer:** This message does not mean a communication or configuration error. Users are advised to check PLC's programming why it is returning non-normalized values.
- **Additional Information:** Such message indicates that the device sent a floating point value, a **Float**, to this Driver in **IEEE 754** format, but non-normalized. Such values may be a result of arithmetical operations with results that extrapolate all representation possibilities of this format, such as overflow, underflow, $+\infty$, and $-\infty$, among others. Non-normalized values are described in IEEE 754 standard, and they are not supposed to raise problems for this Driver or for an application. However, due to previous error detections related to specific hardware, this Driver now returns 0 (zero) to an application when receiving non-normalized values from a device, registering this message on log.

List of Devices that Communicate with the Modbus Protocol

The next table contains a list of devices, separated by manufacturer, for which there is already some experience on communicating with Modbus protocol.

For a more complete list of devices already validated for this protocol, please check *Modbus Device Directory*, maintained by the Modbus Organization.

Devices that communicate with Modbus

MANUFACTURER	DEVICE
ABB	ETE30, MGE 144, KT97, and KT98
Altus	Almost all Altus devices have Modbus, except a few models from Piccolo series
Areva	MiCOM P127 and P632 relay
Atos	ATOS PLCs support Modbus RTU protocol with small variations regarding the maximum size of frames and byte order. For more information about these variations, please check the article <i>Using Modbus Master (ASC/RTU/TCP) drivers with ATOS's controllers</i> , in Elipse Knowledgebase
BCM	BCM1088, BCM1086, BCM-GP3000, and BCM2085
Ciber Brasil	Multivariable Meter for Electrical Quantities UDP200 and Multivariable Meter for Electrical Quantities UDP600

MANUFACTURER	DEVICE
Contemp	CPM45
Deep Sea	DSE5210, DSE5310, DSE5310M, DSE5320, DSE5510, DSE5510M, DSE5520, DSE7310, and DSE7320
Embrasul	MD4040 meter
Eurotherm	2500 Intelligent Data Acquisition and Precision Multi-Loop PID Control
Fatek	FB - 14MCU
GE	GE PAC RX7 and GE GEDE UPS
Gefran	Gefran Gilogk II PLC and Gefran 600RDR21
Hitachi	Hitachi HDL17264
Honeywell	HC-900 PLC
Horner APG	Devices from series XLe/XLt all-in-one control devices
Koyo	CPU 260 PLC
Kron	MKM-120 meter
LG	DMT40U
LG / LSIS	LG Master K120 S and LG XGB - XBM
Moeller	XC100 - Serial Port 232, XC200 - Serial Port 232/422/485 (XIO-SER communication module) and Ethernet Port, XV200 - Serial Port 232 and Ethernet Port, XVH300 - Serial Port 232 and Ethernet Port, and XV400 - Serial Port 232 and Ethernet Port
Novus	N1100, N1500, N2000, and N3000
Schneider	Twido, A340, M340 PLC, Premium Series, Frequency Converters and soft starter, MT and BT breakers, BT and MT protection relays, and SEPAM Series 20, 40, and 80 relays
Telemecanique	Zelio Logic controllers ending with "BD"
Unitronics	V120
Weg	TP 03 and Clic 02
Yaskawa	V1000

List of Default Exceptions of the Modbus Protocol

The next table contains a list of default exceptions defined by the Modbus protocol specification, version 1.1b.

Exceptions are registered on this Driver's log, whenever detected, and they can be read by an application by **reading the last exception code**.

Notice that, in addition to the exceptions listed here, a device may define other proprietary exceptions. In this case, they are supposed to be described on manufacturer's documentation of that device.

Exception codes standardized by the Modbus protocol

CODE (IN HEXADECIMAL)	NAME	MEANING
01	ILLEGAL FUNCTION	The received function code is not valid. This may indicate that this function is not implemented yet or the slave is in an inadequate status to process it
02	ILLEGAL DATA ADDRESS	The received data address is not a valid address. More specifically, the combination of a reference address and the amount of data to transfer is invalid
03	ILLEGAL DATA VALUE	The current value on Master's request is not valid. This indicates a failure on the remaining data structure of a complex request, such as when the informed size for a data block is not correct. This exception does not indicate that values submitted for writing are out of the expected scope by an application, as such information is not accessible to this protocol
04	SLAVE DEVICE FAILURE	An irrecoverable error occurred during the processing of the requested function
05	ACKNOWLEDGE	Used with programming commands. The Slave accepted this message and it is processing it. However, this processing demands a long time. This exception prevents a Master time-out. The end of this request must be tested by a polling process
06	SLAVE DEVICE BUSY	Used with programming commands. Indicates that the Slave is processing another long-lasting command and that this request must be sent again later, when the Slave is available again
08	MEMORY PARITY ERROR	Used with functions 20 and 21 , reference type 6, to indicate the extended file area failed on a consistency test. The Slave device may need maintenance
0A	GATEWAY PATH UNAVAILABLE	Used together with gateways, to indicate that a gateway was unable to allocate an internal path to process this request. It usually indicates that a gateway is not correctly configured or it is overloaded
0B	GATEWAY TARGET DEVICE FAILED TO RESPOND	Used together with gateways, to indicate that no response was received from the destination device. It usually indicates that such device is not available on the network

Maximum Limit for the Size of Blocks Supported by the Modbus Protocol

This topic presents the maximum limits for block sizes supported by the Modbus protocol, in its current version 1.1b of the implementation. Please check this specification at *protocol's official website*.

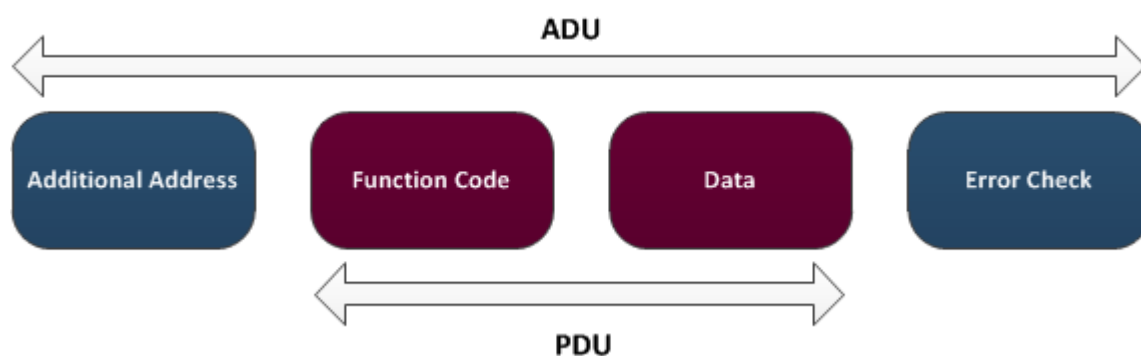
Notice that, due to **Superblock Reading** and **Automatic Block Partition** features, present in this Driver's current version, users hardly need to consider these limits in their applications, as this Driver already performs all optimizations automatically during communication.

However, as there are devices that do not support default limits established by this protocol, users may need to know these limits, and most of all they must know how to evaluate device's limits, in case they are forced to perform a manual grouping. For more information, please check topic **Superblock Reading (Grouping)**. In these cases, information on this topic can be very helpful.

Limits Supported by this Protocol

The Modbus protocol defines a simple data unit called **PDU**, or Protocol Data Unit, which is kept unchanged through several protocol modes and through several communication layers.

A full communication frame, including a PDU and other additional header fields, is called **ADU**, or Application Data Unit.



Limits supported by the Modbus protocol

According to this protocol's specification, a full Modbus frame, or ADU, can have a PDU with a maximum size of 253 bytes.

Thus, depending on the data type or Modbus function used in communication, this protocol imposes the limits for block Elements at each communication described on the next table.

Limits for Block Elements

MODBUS FUNCTION	DESCRIPTION	LIMIT
03, 04	Reading multiple 16-bit registers	125 registers (250 bytes)
16	Writing multiple 16-bit registers (Holding Registers)	123 registers (247 bytes)
01, 02	Reading multiple bits	2000 bits (250 bytes)
15	Writing multiple bits	1968 bits (247 bytes)
20	Reading file registers	124 registers (248 bytes)
21	Writing file registers	122 registers (244 bytes)

For more information, please check the *protocol's official website*.

The article *KB-23112: I/O Block's ideal size with Modbus driver* in **Eclipse Knowledgebase** presents a summary of questions relative to Tag grouping and block resizing in this Driver, discussed here and on other topics.

BCD Encoding

BCD Encoding, or Binary-Coded Decimal, was originally created to circumvent limitations regarding the maximum number of digits that can be represented in traditional formats for storing values. Formats such as the representation of real numbers in floating point are usually acceptable for math and scientific calculations. However, rounding errors caused by numbers that cannot be represented due to overflow or underflow problems may not be acceptable in certain applications, such as financial procedures. To overcome that limitation, a BCD encoding was developed to allow representing numbers up to the last digit.

In this representation, each decimal digit is represented only in a binary format, without limitations regarding its number of digits.

The next table shows decimal digits and their corresponding values in BCD.

Decimal digits in BCD encoding

DECIMAL	BCD	DECIMAL	BCD
0	0000b	5	0101b
1	0001b	6	0110b
2	0010b	7	0111b
3	0011b	8	1000b
4	0100b	9	1001b

To improve efficiency of this encoding, it is common to represent two digits per byte. Notice that, on the previous table, each decimal digit requires only 4 (four) bits, or a half byte, for its representation.

Such representation with 2 (two) digits in each byte is called **Packed BCD**, and this the representation used by this Driver, that is, packets sent by this Driver with BCD values use a data byte for every 2 (two) digits of the represented decimal value. That is why the **Size** field, for **BCD** data types, must be defined as half the maximum number of digits represented in read or written values.

Example

As an example, suppose that users want to send the value 84 in decimal, 0x54 in hexadecimal format, using a packed BCD encoding in one byte, the format used by this Driver.

The first step is separate the 2 (two) decimal digits that compose this value in its decimal representation:

- **Digit 1:** 8 (eight)
- **Digit 2:** 4 (four)

If users want to send this value to a device without a BCD encoding, then the value sent to the protocol is 84, which is represented in hexadecimal format by 0x54, or else 01010100b in binary format.

By using a packed BCD format, however, users represent these 2 (two) decimal digits separately in each half, or nibble, of the byte to send:

- **BCD:** 0x84 or 10000100b

Notice that, if this value 0x84 is mistakenly interpreted in **BCD** format as a value in hexadecimal format without this encoding, and this value is then converted to decimal, users get the value 132, which is meaningless.

The next table presents a few more examples of decimal values between 0 (zero) and 99 and their respective representations in Packed BCD format in 1 (one) byte, presented in hexadecimal and binary formats.

Decimal digits in Packed BCD encoding

DECIMAL	HEXADECIMAL	BCD (HEXADECIMAL)	BCD (BINARY)
10	0x0A	0x10	00010000b
0	0x00	0x00	00000000b
99	0x63	0x99	10011001b
81	0x51	0x81	10000001b
45	0x2D	0x45	01000101b

Example of Configuration for the BCD Format

1) Reading of value 11223344 in BCD format

Note that the integer 11223344 contains 8 (eight) digits, so in the properties dialog of this Driver, users must create a new Operation, defining the **Data** field as type **BCD** and the **Size** field with size 4 (four), in other words, the 8 (eight) digits of the BCD value must be sized to bytes, grouping the digits always in pairs, 11 22 33 44, to know how many bytes are required to represent it.

Reading a value with Packed BCD encoding

DEVICE MEMORY	TAG VALUE	NOTES
Register 1 = 0x1122 and Register 2 = 0x3344	11223344	The value 11223344 requires 4 (four) bytes to be represented in BCD format, and occupy 2 (two) Modbus registers in the device's memory

Documentation of I/O Interfaces

This section contains the documentation of I/O Interfaces referring to **Modbus** Driver.

Driver Configuration

I/O Interface configuration is performed on a Driver's configuration dialog box. To access the configuration of this dialog box in **E3** (version 1.0), follow these steps:

1. Right-click a Driver object (IODriver).
2. Select the **Properties** item on the contextual menu.
3. Select the **Driver** tab.
4. Click **Other parameters**.

In **E3** version 2.0 or later, click **Configure driver**  on a Driver's toolbar. In **Elipse SCADA**, follow these steps:

1. Open the Organizer.
2. Select a Driver on Organizer's tree.
3. Click **Extras** on the **Driver** tab.

Currently, an I/O Interface allows opening only one connection for each Driver. This means that, if users want to access two serial ports, they must add two Drivers to an application and then configure each Driver for each serial port.

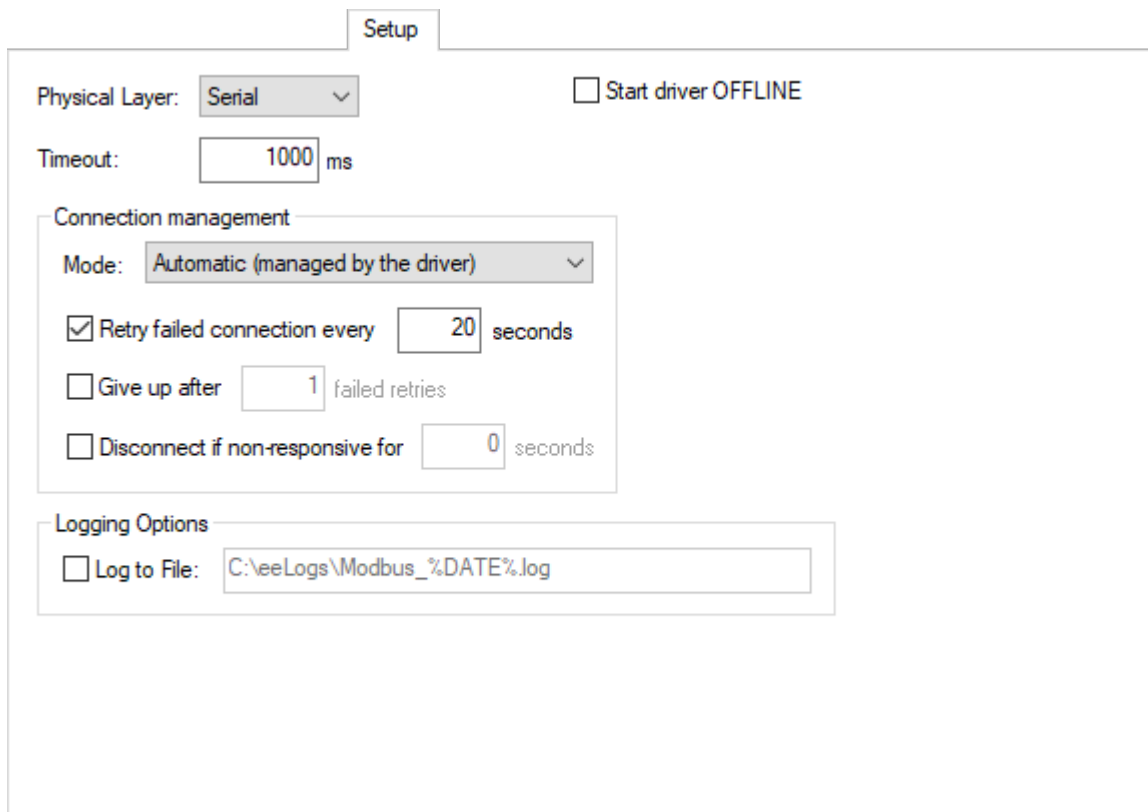
Configuration Dialog Box

The I/O Interfaces dialog box allows configuring the I/O connection used by a Driver. This dialog box contains the **Setup**, **Serial**, **Ethernet**, **Modem**, and **RAS** tabs, described on the next topics. If a Driver does not implement a specific I/O connection, its corresponding tab is not available for configuration. Some Drivers may contain additional tabs, specific for each Driver, on the configuration dialog box.

Setup Tab

The **Setup** tab contains Driver's general configurations. This tab is divided into the following groups:

- **General configurations:** Configurations of Driver's physical layer, time-out, and initialization mode
- **Connection management:** Configurations on how the I/O Interface keeps a connection and which recovery policy is used on failure
- **Logging options:** Controls the generation of log files



Setup tab

General options on the Setup tab

OPTION	DESCRIPTION
Physical Layer	Select the physical layer on a list. Available options are Serial , Ethernet , Modem , and RAS . The selected interface must be configured on its specific tab
Timeout	Configure a time-out, in milliseconds, for the physical layer. This is the amount of time an I/O interface waits to receive any byte from reception's buffer
Start driver OFFLINE	Select this option so that the Driver starts in Offline mode or stopped. This means that the I/O interface is not created until this Driver is configured to Online mode by using a Tag in an application. This mode enables a dynamic configuration of an I/O interface at run time

Options on the Connection management group

OPTION	DESCRIPTION
Mode	Selects a management mode of a connection. Selecting the Automatic option allows a Driver to manage the connection automatically, as specified in the next options. Selecting the Manual option allows an application to fully manage a connection. Please check topic Driver Statuses for more details
Retry failed connection every ... seconds	Select this option to enable a Driver's connection retry in a certain interval, in seconds. If the Give up after failed retries option is not selected, the Driver keeps retrying until a connection is performed, or until the application is stopped
Give up after ... failed retries	Enable this option to define a maximum number of connection retries. When the specified number of consecutive connection retries is reached, the Driver goes to the Offline mode, assuming that a hardware problem was detected. If a Driver establishes a successful connection, the number of unsuccessful retries is cleared. If this new connection is lost, then the retry counter starts at zero
Disconnect if non-responsive for ... seconds	Enable this option to force a Driver to disconnect if no byte was received by the I/O interface during the specified time-out, in seconds. This time-out must be greater than the time-out configured in the Timeout option

Options on the Logging Options group

OPTION	DESCRIPTION
Log to File	Enable this option and configure the name of a file to write a log. Log files can be large, so use this option for short periods of time, only for testing and debugging purposes. If the %PROCESS% macro is used in the log file name, it is replaced by the ID of the current process. This option is particularly useful when using several instances of the same Driver in E3, thus allowing each instance to generate a separate log file. For example, when configuring this option as c:\e3logs\drivers\sim_%PROCESS%.log, a file named c:\e3logs\drivers\sim_00000FDA.log is generated for process 0FDAh. Users can also use the %DATE% macro in the file name. In this case a log file is generated every day (in the format aaaa_mm_dd). For example, when configuring this option as c:\e3logs\drivers\sim_%DATE%.log, a file named c:\e3logs\drivers\sim_2005_12_31.log is generated in 12/31/2005 and a file named c:\e3logs\drivers\sim_2006_01_01.log is generated in 01/01/2006

Serial Tab

Use this tab to configure parameters for a **Serial** Interface.

Serial

Port:

COM1

Baud rate:

9600

Data bits:

8 data bits

Parity:

None

Stop bits:

1 stop bit

☐

Enable 'ECHO' suppression

Handshaking

DTR control:

OFF

RTS control:

OFF

☐

Wait for CTS before send

CTS timeout:

0

ms

Delay before send:

0

ms

Delay after send:

0

ms

Inter-byte delay (microseconds):

0

µs

Inter-frame delay (milliseconds):

0

ms

Serial tab

General options on the Serial tab

OPTION	DESCRIPTION
Port	Select a serial port on the list, from COM1 to COM4 , or type the name of a serial port in the format COMn , such as "COM15". When typing a port's name manually, the dialog box only accepts port names starting with the expression "COM"
Baud rate	Select a baud rate on the list (1200, 2400, 4800, 9600, 19200, 38400, 57600, or 115200) or type a baud rate, such as 600
Data bits	Select 7 (seven) or 8 (eight) data bits on the list
Parity	Select a parity on the list. The available options are None, Even, Odd, Mark, or List
Stop bits	Select the number of stop bits on the list. The available options are 1, 1.5, or 2 stop bits
Enable 'ECHO' supression	Enable this option to remove the echo received after the I/O Interface sends data via serial port. If this echo is not equal to the bytes just sent, then the I/O Interface aborts communication.
Inter-byte delay (microseconds)	Defines a delay between each byte transmitted by the I/O Interface, in millionths of a second, that is, 1000000 is equal to a second. This option must be used with small delays of less than a millisecond
Inter-frame delay (milliseconds)	Defines a delay between packets sent or received by the I/O Interface, in thousandths of a second, that is, 1000 is equal to a second. This delay is applied if the I/O Interface sends two consecutive packets, or between a received packet and the next sending

The **Handshaking** group configures the usage of **RTS**, **CTS**, and **DTR** signals in the handshaking process (controlling when data can be sent or received via serial line). Most of the time, configuring the **DTR control** option to **ON** and the **RTS control** option to **Toggle** works with RS232 serial lines as well as with RS485 serial lines.

Available options on the Handshaking group

OPTION	DESCRIPTION
DTR control	Select ON to keep the DTR signal always on while the serial port is open. Select OFF to turn the DTR signal off while the serial port is open. Some devices require the DTR signal always on to allow communication.
RTS control	Select ON to keep the RTS signal always on while the serial port is open. Select OFF to turn the RTS signal off while the serial port is open. Select Toggle to turn the RTS signal on while sending bytes via serial port and turn it off when not sending bytes, therefore enabling the reception.
Wait for CTS before send	Available only when the RTS control option is configured to Toggle . Use this option to force a Driver to check the CTS signal before sending bytes via serial port, after

OPTION	DESCRIPTION
	turning the RTS signal on. In this mode the CTS signal is handled as a permission flag for sending.
CTS timeout	Determines a maximum time, in milliseconds, that a Driver waits for the CTS signal after turning the RTS signal on. If the CTS signal is not turned on within this time-out, the Driver then fails the current communication and returns an error.
Delay before send	Some serial port hardware have a delay when enabling a data sending circuit after the RTS signal is turned on. Configure this option to wait a certain number of milliseconds after turning the RTS signal on and before sending the first byte. IMPORTANT: This delay must be used carefully, because it uses 100% of CPU resources while waiting. System's general performance degrades as this value increases.
Delay after send	This is the same effect of the Delay before send option, but in this case the delay is performed after sending the last byte, before turning the RTS signal off.

Ethernet Tab

Use this tab to configure parameters of an **Ethernet** Interface. These parameters, except port configurations, must also be configured for use in the **RAS**.

Ethernet

Transport: TCP/IP ▼

☐ PING before connecting
Timeout: 4000 ms
Retries: 1

☐ Listen for connections on port: 0
☐ Share listen port with other processes
☐ Interface: (All Interfaces) ▼
☐ Use IPv6 ☐ Use SSL SSL Settings
☐ Enable 'ECHO' supression
IP Filter:

Connect to

Main IP: 	Port: 502	☐ Local port: 0
☐ Backup IP 1: 	Port: 0	☐ Local port: 0
☐ Backup IP 2: 	Port: 0	☐ Local port: 0
☐ Backup IP 3: 	Port: 0	☐ Local port: 0

Ethernet tab

Available options on Ethernet tab

OPTION	DESCRIPTION
Transport	Select TCP/IP for a TCP socket (stream). Select UDP/IP to use a UDP socket (connectionless datagram)
Listen for connections on port	Use this option to wait for new connections in a specific IP port, common in Slave Drivers. If this option remains unselected, the Driver connects to the address and port specified in the Connect to option
Share listen port with other processes	Select this option to share the listen port with other Drivers and processes
Interface	Select the local network interface, identified by its IP address, that is used by the Driver to establish and receive connections, or select the (All Interfaces) item to use any local network interface
Use IPv6	Check this option to force the Driver to use IPv6 addresses on all Ethernet connections. If this option is unchecked the Driver will work with IPv4 addresses
Enable 'ECHO' suppression	Enable this option to remove the echo from received data. An echo is a copy of sent data, which can be returned before a reply message
IP Filter	List of restricted or allowed IP addresses from where a Driver accepts connections (Firewall). Please check the IO.Ethernet.IPFilter property for more details
PING before connecting	Enable this option to execute a ping command, that is, check whether a device can be reached on a network, for a device before trying a socket connection. This is a quick way of determining a successful connection before trying to open a socket with a device. The time-out of a connection with a socket can be very high. The available options are: • Timeout: Specify the number of milliseconds to wait for a reply from the ping command. Users must use the ping command to check the normal reply time, configuring this option for a value above that average. Usually this value can be configured between 1000 and 4000 milliseconds, that is, between one and four seconds • Retries: Number of retries of a ping command, not counting the first attempt. If all attempts fail, then the socket connection is aborted

Available options on the Connect to group

OPTION	DESCRIPTION
Main IP	Type the IP address of the remote device. Users can use an IP address separated by dots, as well as a URL. In case of a URL, the Driver uses the available DNS service to map that URL to an IP address, such as "192.168.0.13" or "Server1"
Port	Type the IP port of the remote device, between 0 (zero) and 65535

OPTION	DESCRIPTION
Local port	Select this option to use a fixed local IP port when connecting to a remote device
Backup IP 1, 2, and 3	Indicate here the IP address, the IP port, and the fixed local IP port of up to 3 (three) backup addresses of a remote device

Modem Tab

Use this tab to configure parameters of a **Modem** Interface. Some options on the **Serial** tab affect the modem configuration, therefore users must also configure the **Serial** Interface.

Modem tab

The **Modem** Interface uses the TAPI modems installed on the computer.

Available options on the Modem tab

OPTION	DESCRIPTION
Select the modem to use	Select a modem on the list of available modems on the computer. If the Default modem option is selected, then the first available modem is used. Selecting this option is recommended specially when an application is used on another computer
Modem settings	Click to open the configuration window of the selected modem
Dial Number	Type a default number for dialing. This value can be changed at run time. Users can use the w character to represent a pause or a waiting time for a dial tone. For example, "0w33313456" dials the number zero, waits, and then dials the number "33313456"

OPTION	DESCRIPTION
Accept incoming calls	Enable this option so that the Driver answers the phone when receiving an external call. To use this option, users must configure the Connection management option on the Setup tab to Manual

RAS Tab

Use this tab configure parameters of a **RAS** Interface. Users must also configure the **Ethernet** tab.

A **RAS** Interface opens a socket connection with a RAS device. A RAS device is a server of modems available through TCP/IP, waiting for socket connections on an IP port. For each connection accepted on this port, users have access to one modem.

When connecting to a RAS device, first the I/O Interface **IOKit** connects to the socket on the IP address and port configured on the **Ethernet** tab. After opening the socket, the following initialization or connection steps are performed:

1. Clear the socket, that is, remove any TELNET greeting message received from a RAS device.
2. Send an **AT** dial message (in ASCII) in the socket.
3. Wait for a **CONNECT** reply.
4. If the time-out expires, the connection is aborted.
5. If the **CONNECT** reply is received within the time-out, the socket is available for communication with the device, that is, the connection was established.

If step 5 is successful, then the socket behaves as a normal socket, with the RAS device working as a router between the Driver and the device. Bytes sent by the Driver are received by the RAS device and sent to the destination device using a modem. Bytes received by the modem's RAS device are sent back to the Driver using the same socket.

After establishing the connection, the **RAS** interface monitors data received from a Driver. If a "NO CARRIER" **String** is found, the socket is closed. If the RAS device does not send a **NO CARRIER** signal, the **RAS** Interface cannot detect when the modem connection between the RAS device and the final I/O device fails. To recover from this failure, users are strongly advised to enable the **Disconnect if non-responsive** option on the **Setup** tab.

RAS

AT command:

Connection timeout: seconds

Other socket settings should be configured in the "Ethernet" tab!

RAS tab

Available options on RAS tab

OPTION	DESCRIPTION
AT command	A String with the full AT command used to dial to a destination device. For example, "ATDT33313456" dials by tone to number "33313456"
Connection timeout	Number of seconds to wait for a modem's CONNECT reply, after sending an AT command

General Configurations

This section contains information about the configuration of general **I/O Tags** and **Properties** of I/O Interfaces.

I/O Tags

General I/O Interfaces Tags (N2/B2 = 0)

The Tags described next are provided for all supported I/O Interfaces.

IO.IOKitEvent

Type of Tag	Block Tag
Type of Access	Read-Only
B1 Parameter	-1 (minus one)
B2 Parameter	0 (zero)
B3 Parameter	0 (zero)
B4 Parameter	1 (one)
Size Property	4 (four)
ParamItem Property	IO.IOKitEvent

This Block returns Driver events generated by several sources in I/O Interfaces. The **TimeStamp** property of this Block represents the moment this event occurred. The Block Elements are the following:

- **Element 0:** Type of event. Possible values are **0:** Information, **1:** Warning, or **2:** Error
- **Element 1:** Source of event. Possible values are **0:** Driver (specific of a Driver), **-1:** IOKit (generic events of I/O Interfaces), **-2:** **Serial** Interface, **-3:** **Modem** Interface, **-4:** **Ethernet** Interface, or **-5:** **RAS** Interface
- **Element 2:** Error number, specific for each source of event
- **Element 3:** Event message, a **String** specific for each event

NOTE

A Driver keeps a maximum number of 100 events internally. If additional events are reported, older events are discarded.

IO.PhysicalLayerStatus

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	2 (two)
String Configuration	IO.PhysicalLayerStatus

This Tag indicates the status of a physical layer. Its possible values are the following:

- **0:** Physical layer stopped, that is, the Driver is in **Offline** mode, the physical layer failed when initializing, or exceeded the maximum number of reconnection attempts
- **1:** Physical layer started but not connected, that is, the Driver is in **Online** mode but the physical layer is not connected. If the **Connection management** option is configured as **Automatic**, the physical layer can be connecting,

disconnecting, or waiting for a reconnection attempt. If the **Connection management** option is configured as **Manual**, then the physical layer remains in this status until forced to connect

- **2:** Physical layer connected, that is, the physical layer is ready for use. This **DOES NOT** mean the device is connected, only the access layer is working

IO.SetConfigurationParameters

Type of Tag	Block Tag
Type of Access	Read-Only
B1 Parameter	-1 (minus one)
B2 Parameter	0 (zero)
B3 Parameter	0 (zero)
B4 Parameter	3 (three)
Size Property	2 (two)
ParamItem Property	IO.SetConfigurationParameters

Use this Tag to change any property of a Driver's configuration dialog box at run time.

This Tag works only while a Driver is in **Offline** mode. To start a Driver in **Offline** mode, select the **Start driver OFFLINE** option on the Driver's configuration dialog box. Users can write to a PLC Tag or to a Block Tag containing the parameters to change. Writing individual Block Elements is not supported, the whole Block must be written at once.

In **Elipse SCADA**, users must use a Block Tag. Every parameter to configure uses two Block Elements. For example, if users want to configure three parameters, then the size of the Block must be 6 (3 × 2). The first Element is the property's name, as a **String**, and the second Element is the property's value, according to the next example.

```
// 'Block' must be a Block Tag with automatic reading,
// scan reading, and automatic writing disabled.
// Configure all parameters
Block.element001 = "IO.Type" // Parameter 1
Block.element002 = "Serial"
Block.element003 = "IO.Serial.Port" // Parameter 2
Block.element004 = 1
Block.element005 = "IO.Serial.BaudRate" // Parameter 3
Block.element006 = 19200
// Writes the whole Block
Block.Write()
```

When using **E3**, the ability to create arrays at run time allows using an I/O Tag as well as a Block Tag. Users can use Driver's **Write** method to send all parameters to the Driver, without creating a Tag, according to the next example.

```
Dim arr(6)
' Configure all array elements
arr(1) = "IO.Type"
arr(2) = "Serial"
arr(3) = "IO.Serial.Port"
arr(4) = 1
arr(5) = "IO.Serial.BaudRate"
arr(6) = 19200
' There are two methods to send parameters
' Method 1: Using an I/O Tag
tag.WriteEx arr
' Method 2: Without using a Tag
Driver.Write -1, 0, 0, 3, arr
```

A variation of the previous example uses a bidimensional array.

```
Dim arr(10)
' Configure all array elements. Notice the array was resized
' to 10 elements. Empty array elements are ignored by a Driver
arr(1) = Array("IO.Type", "Serial")
arr(2) = Array("IO.Serial.Port", 1)
arr(3) = Array("IO.Serial.BaudRate", 19200)
Driver.Write -1, 0, 0, 3, arr
```

A Driver does not validate parameter names or passed values, therefore be careful when writing parameters and values. The **Write** method fails if the configuration array is incorrectly created. Users can check Driver's log or use the *writeStatus* parameter of the **WriteEx** method to find out the exact cause of the error.

```
Dim arr(10), strError
arr(1) = Array("IO.Type", "Serial")
arr(2) = Array("IO.Serial.Port", 1)
arr(3) = Array("IO.Serial.BaudRate", 19200)
If Not Driver.WriteEx -1, 0, 0, 3, arr, , , strError Then
    MsgBox "Failed configuring Driver parameters: " + strError
End If
```

IO.WorkOnline

Type of Tag	I/O Tag
Type of Access	Reading or Writing
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	4 (four)
String Configuration	IO.WorkOnline

This Tag informs the current status of a Driver and allows starting or stopping the physical layer. Possible values are the following:

- **0 - Driver Offline:** Physical layer is closed or stopped. This mode allows a dynamic configuration of a Driver's parameters using the **IO.SetConfigurationParameters** Tag
- **1 - Driver Online:** Physical layer is open or executing. While in **Online** mode, the physical layer can be connected or disconnected and its current status can be checked on the **IO.PhysicalLayerStatus** Tag

In the next example, using **E3**, a Driver is configured to **Offline** mode, its COM port is changed, and then configured to **Online** mode again.

```
'Configure to Offline mode
Driver.Write -1, 0, 0, 4, 0
'Change port to COM2
Driver.Write -1, 0, 0, 3, Array("IO.Serial.Port", 2)
'Configure to Online mode
Driver.Write -1, 0, 0, 4, 1
```

The **Write** method can fail when configuring a Driver to **Online** mode, that is, writing the value 1 (one). In this case, the Driver remains in **Offline** mode. The cause of failure can be:

- Type of physical layer incorrectly configured, probably an invalid value was configured in the **IO.Type** property

- Driver may have run out of memory
- Physical layer probably did not create its working thread. Search the log file for a message "Failed to create physical layer thread!"
- Physical layer could not start. The cause of failure depends on the type of physical layer. It can be an invalid serial port number, a failure when starting Windows Sockets, or a failure when starting TAPI (modem), among others. This cause is recorded on the log file

IMPORTANT

Even if the configuration of a Driver to **Online** mode is successful, this does not necessarily mean the physical layer is ready to use, that is, ready to execute input and output operations with an external device. The **IO.PhysicalLayerStatus** Tag must be checked to ensure the physical layer is connected and ready for communication.

Properties

These are general properties of all supported I/O Interfaces.

IO.ConnectionMode

9 Controls the management mode of a Connection. Possible values are **0**: Automatic mode, where a Driver manages the connection or **1**: Manual mode, where an application manages the connection.

IO.GiveUpEnable

☑ When configured to True, defines a maximum number of reconnection attempts. If all reconnection attempts fail, the Driver enters the **Offline** mode. When configured to False, the Driver tries until a reconnection is successful.

IO.GiveUpTries

9 Number of reconnection attempts before this one is aborted. For example, if the value of this property is equal to 1 (one), a Driver tries only one reconnection when the reconnection is lost. If this one fails, a Driver enters the **Offline** mode.

IO.InactivityEnable

☑ Configure to True to enable and to False to disable inactivity detection. The physical layer is disconnected if inactive for a certain period of time. The physical layer is considered inactive only if it is capable of sending data but not capable of receiving it back.

IO.InactivityPeriodSec

9 Number of seconds to check for inactivity. If the physical layer is inactive for this period of time, it is disconnected.

IO.RecoverEnable

☑ Configure to True to enable a Driver to recover lost connections and to False to leave a Driver in **Offline** mode when a connection is lost.

IO.RecoverPeriodSec

9 Delay time between two connection attempts, in seconds.

NOTE

The first reconnection is executed immediately after a connection is lost.

IO.StartOffline

☑ Configure to True to start a Driver in **Offline** mode and to False to start a Driver in **Online** mode.

NOTE

It is pointless to change this property at run time, as it can only be changed when a Driver is already in **Offline** mode. To configure a Driver in **Online** mode at run time, write the value 1 (one) to the **IO.WorkOnline** Tag.

IO.TimeoutMs

9 Defines a time-out for the physical layer, in milliseconds. One second is equal to 1000 milliseconds.

IO.Type

A Defines the type of physical interface used by a Driver. Possible values are the following:

- **N or None**: Does not use a physical interface, that is, a Driver must provide a customized interface
- **S or Serial**: Uses a local serial port (COM n)
- **M or Modem**: Uses a local modem, internal or external, accessed via TAPI (*Telephony Application Programming Interface*)
- **E or Ethernet**: Uses a TCP/IP or UDP/IP socket
- **R or RAS**: Uses a **RAS** (*Remote Access Server*) Interface. A Driver connects to a RAS device using the **Ethernet** Interface and then sends an **AT** (*dial*) command

Statistical Configuration

This section contains information about the configuration of **I/O Tags** and **Properties** of I/O Interfaces statistics.

I/O Tags

Tags of I/O Interface statistics (N2/B2 = 0)

The Tags described next display statistics for all I/O Interfaces.

IO.Stats.Partial.BytesRecv

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1101
Configuration by String	IO.Stats.Partial.BytesRecv

This Tag returns the number of bytes received in the current connection.

IO.Stats.Partial.BytesSent

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1100
Configuration by String	IO.Stats.Partial.BytesSent

This Tag returns the number of bytes sent through the current connection.

IO.Stats.Partial.TimeConnectedSeconds

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1102
Configuration by String	IO.Stats.Partial.TimeConnectedSeconds

This Tag returns the number of seconds a Driver is connected in the current connection or 0 (zero) if a Driver is disconnected.

IO.Stats.Partial.TimeDisconnectedSeconds

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1103
Configuration by String	IO.Stats.Partial.TimeDisconnectedSeconds

This Tag returns the number of seconds a Driver is disconnected since the last connection ended or 0 (zero) if a Driver is connected.

IO.Stats.Total.BytesRecv

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1001
Configuration by String	IO.Stats.Total.BytesRecv

This Tag returns the number of bytes received since a Driver was loaded.

IO.Stats.Total.BytesSent

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1000
Configuration by String	IO.Stats.Total.BytesSent

This Tag returns the number of bytes sent since a Driver was loaded.

IO.Stats.Total.ConnectionCount

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1004
Configuration by String	IO.Stats.Total.ConnectionCount

This Tag returns the number of connections a Driver already established, successfully, since it was loaded.

IO.Stats.Total.TimeConnectedSeconds

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1002
Configuration by String	IO.Stats.Total.TimeConnectedSeconds

This Tag returns the number of seconds a Driver remained connected since it was loaded.

IO.Stats.Total.TimeDisconnectedSeconds

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	0 (zero)
N4 Parameter	1003
Configuration by String	IO.Stats.Total.TimeDisconnectedSeconds

This Tag returns the number of seconds a Driver remained disconnected since it was loaded.

Properties

Currently, there are no properties defined specifically to display I/O Interface statistics at run time.

Ethernet Interface Configuration

This section contains information about the configuration of **I/O Tags** and **Properties** of an **Ethernet** Interface.

I/O Tags

Tags of an Ethernet Interface (N2/B2 = 4)

The Tags described next allow controlling and identifying an **Ethernet** Interface at run time and they are also valid when the **RAS** Interface is selected.

IMPORTANT

These Tags are available **ONLY** while a Driver is in **Online** mode.

IO.Ethernet.IPSelect

Type of Tag	I/O Tag
Type of Access	Reading or Writing
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	4 (four)
N4 Parameter	0 (zero)
String Configuration	IO.Ethernet.IPSelect

Indicates the active IP address. Possible values are **0**: The main IP address is selected, **1**: The first alternative or backup IP address is selected, **2**: The second alternative or backup IP address is selected, or **3**: The third alternative or backup IP address is selected.

If the **Ethernet** or **RAS** Interface is connected, this Tag indicates which one of the four configured IP addresses is in use. If the Interface is disconnected, this Tag indicates which IP address is used first on the next connection attempt.

During the connection process, if the active IP address is not available, the I/O Interface tries to connect using the next alternative IP address. If the connection with the alternative IP address works, it is configured as the active IP address (automatic switchover).

To force a manual switchover, write values from 0 (zero) to 3 (three) to this Tag. This forces a reconnection with the specified IP address (**0**: Main IP address or **1, 2, 3**: Alternative IP address) if the Driver is currently connected. If the Driver is disconnected, this Tag configures the active IP address for the next connection attempt.

IO.Ethernet.IPSwitch

Type of Tag	I/O Tag
Type of Access	Write-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	4 (four)
N4 Parameter	1 (one)
String Configuration	IO.Ethernet.IPSwitch

Any value written to this Tag forces a manual switchover. If the main IP address is active, then the Driver tries to connect to each one of the alternative or backup IP addresses and back to the main IP address until a connection is established.

If the Driver is disconnected, this Tag configures the active IP address for the next connection attempt.

Properties

These properties control the configuration of an **Ethernet** Interface.

NOTE

The **Ethernet** Interface is also used by the **RAS** Interface.

IO.Ethernet.AcceptConnection

■ **Configure to False** if the Driver must not accept external connections, that is the Driver behaves as a master, or **configure to True** to enable the reception of connections, that is, the Driver behaves as a slave.

IO.Ethernet.BackupEnable[2,3]

■ **Configure to True** to enable an alternative or backup IP address. If the reconnection attempt with the main IP address fails, the Driver tries to use an alternative IP address. Configure to **False** to disable its usage.

IO.Ethernet.BackupLocalPort[2,3]

9 Local port number to be used when connecting to an alternative IP address of a remote device. Used only if **IO.Ethernet.BackupLocalPortEnable** is equal to True.

IO.Ethernet.BackupLocalPortEnable[2,3]

☒ **Configure to True** to force the use of a specific local port when connecting to an alternative or backup IP address of a remote device or configure to **False** to use any available local port.

IO.Ethernet.BackupIP[2,3]

A Alternative or backup IP address of a remote device. Users can use a numerical address, as well as a device's host name, such as "192.168.0.7" or "SERVER2".

IO.Ethernet.BackupPort[2,3]

9 Port number of an alternative or backup IP address of a remote device, used with the **IO.Ethernet.BackupIP** property.

IO.Ethernet.IPFilter

A List with a comma-separated IPv4 or IPv6 addresses, which defines from which addresses a Driver accepts or blocks connections. Users can use asterisks, such as "192.168.*.*", or intervals, such as "192.168.0.41-50", in any part of IP addresses. To block an IP address or a range of IP addresses, use the tilde ("~") character at the beginning of the address. Examples:

- **192.168.0.24**: Accepts only connections from IPv4 address 192.168.0.24
- **192.168.0.41-50**: Accepts connections from IPv4 addresses from 192.168.0.41 to 192.168.0.50
- **192.168.0.***: Accepts connections from IPv4 addresses from 192.168.0.0 to 192.168.0.255
- **fe80:3bf:877::*:*** (**expands to fe80:03bf:0877:0000:0000:0000:0000:0000 to fe80:03bf:0877:0000:0000:0000:ffff:ffff**): Accepts connections from IPv6 addresses from fe80:03bf:0877:0000:0000:0000:0000:0000 to fe80:03bf:0877:0000:0000:0000:ffff:ffff
- **192.168.0.10, 192.168.0.15, 192.168.0.20**: Accepts connections from IPv4 addresses 192.168.0.10, 192.168.0.15, and 192.168.0.20
- **~192.168.0.95, 192.168.0.***: Accepts connections from IPv4 addresses from 192.168.0.0 to 192.168.0.255, except the IPv4 address 192.168.0.95

When a Driver receives a connection attempt, the list of filters is scanned sequentially from left to right, searching for a specific authorization or block for the IP address where the connection comes from. If no element on the list corresponds to the IP address, the authorization or block are dictated by the last element of that list:

- If the last element on the list is an authorization, such as "192.168.0.24", then all IP addresses not found on the list are blocked
- If the last element on the list is a block, such as "~192.168.0.24", then all IP addresses not found on the list are authorized

If an IP address appears on more than one filter on the list, the leftmost filter has precedence. For example, in case of "~192.168.0.95, 192.168.0.*", the IP address 192.168.0.95 fits both rules, but the rule that wins is the leftmost one, "~192.168.0.95", and therefore this IP address is blocked.

When **IOKit** blocks a connection, it logs a message "Blocked incoming socket connection from {IP}!".

In case of UDP connections in broadcast listen mode, where a Driver can receive packets from different IP addresses, blocks or permissions are performed at each packet received. If a packet is received from a blocked IP address, it logs a message "Blocked incoming packet from {IP} (discarding {N} bytes)!".

IO.Ethernet.ListenIP

 IP address of the local network interface that a Driver uses to establish and receive connections. Leave this property empty to use any local network interface.

IO.Ethernet.ListenPort

 Number of the IP port used by a Driver to listen to connections.

IO.Ethernet.MainIP

 IP address of a remote device. Users can use a numerical address, as well as a device's host name, such as "192.168.0.7" or "SERVER2".

IO.Ethernet.MainLocalPort

 Local port number to use when connecting to the main IP address of a remote device. This value is only used if the **IO.Ethernet.MainLocalPortEnable** property is equal to True.

IO.Ethernet.MainLocalPortEnable

 Configure to True to force the use of a specific local port when connecting to the main IP address of a remote device or configure to False to use any available local port.

IO.Ethernet.MainPort

 Number of the IP port of a remote device, used with the **IO.Ethernet.MainIP** property.

IO.Ethernet.PingEnable

 Configure to True to enable sending a **ping** command to the IP address of a remote device, before trying to connect to the socket. This socket's connection time-out cannot be controlled, therefore sending a **ping** command before connecting is a fast way to detect if the connection is going to fail. Configure to False to disable a **ping** command.

IO.Ethernet.PingTimeoutMs

 Delay time to wait for a response from a **ping** command, in milliseconds.

IO.Ethernet.PingTries

 Maximum number of attempts of a **ping** command. Minimum value is 1 (one), including the first **ping** command.

IO.Ethernet.ShareListenPort

 Configure to True to share a listening port with other Drivers and processes or False to open a listening port in exclusive mode. To successfully share a listening port, all Drivers and processes that use that port must open it in shared mode. When a listening port is shared, each incoming connection is distributed to one of the processes listening. This way, if a Slave Driver only supports one connection at a time, users can use several instances of this Driver listening on the same port, therefore simulating a Driver with support for multiple connections.

IO.Ethernet.SuppressEcho

 Configure to True to eliminate echoes in communication. An echo is the unwanted reception of an exact copy of all data packets a Driver sends to a device.

IO.Ethernet.Transport

 Defines a transport protocol. Possible values are **T** or **TCP**: Uses the TCP/IP protocol or **U** or **UDP**: Uses the UDP/IP protocol.

IO.Ethernet.UseIPv6

 Configure to True to use IPv6 addresses on all Ethernet connections or configure to False to use IPv4 addresses (default).

Modem Interface Configuration

This section contains information about the configuration of **I/O Tags** and **Properties** of a **Modem** (TAPI) Interface.

I/O Tags

Tags of a Modem Interface (N2/B2 = 3)

The Tags described next allow controlling and diagnosing a **Modem** (TAPI) Interface at run time.

IMPORTANT

These Tags are available **ONLY** while the Driver is in **Online** mode.

IO.TAPI.ConnectionBaudRate

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	3 (three)
N4 Parameter	5 (five)
String Configuration	IO.TAPI.ConnectionBaudRate

Indicates a baud rate value for the current connection. If a modem is not connected, returns the value 0 (zero).

IO.TAPI.Dial

Type of Tag	I/O Tag
Type of Access	Write-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	3 (three)
N4 Parameter	1 (one)
String Configuration	IO.TAPI.Dial

Write any value to this Tag to force a **Modem** Interface to start a call. This is an asynchronous command, only starting the call process. Users can monitor the **IO.TAPI.IsModemConnected** Tag to detect when a call is established.

IO.TAPI.HangUp

Type of Tag	I/O Tag
Type of Access	Write-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	3 (three)
N4 Parameter	4 (four)
String Configuration	IO.TAPI.HangUp

Any value written to this Tag turns the current call off.

NOTE

Use this command only when managing the physical layer manually or when explicitly trying to force a Driver to restart the communication. If the physical layer is configured for automatic reconnection, the Driver immediately tries to reestablish the connection.

IO.TAPI.IsModemConnected

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	3 (three)
N4 Parameter	3 (three)
String Configuration	IO.TAPI.IsModemConnected

This Tag indicates the status of a modem connection. Possible values are **0**: The modem is not connected, but it may be performing or receiving an external call or **1**: The modem is connected and the Driver completed or received an external call successfully. While it is in this status, the physical layer can send or receive data.

IO.TAPI.IsModemConnecting

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	3 (three)
N4 Parameter	6 (six)
String Configuration	IO.TAPI.IsModemConnecting

This Tag indicates the status of a modem connection, with more details than the **IO.TAPI.IsModemConnected** Tag. Possible values are **0**: Modem is not connected, **1**: Modem is connecting, that is, performing or receiving an external call, **2**: Modem is connected. While in this status, the physical layer can send or receive data, or **3**: Modem is disconnecting the current call.

IO.TAPI.ModemStatus

Type of Tag	I/O Tag
Type of Access	Read-Only
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	3 (three)
N4 Parameter	2 (two)
String Configuration	IO.TAPI.ModemStatus

Returns a **String** with the current status of a modem. Possible values are the following:

- **"No status!"**: The **Modem** Interface was not open yet or was already closed
- **"Modem initialized OK!"**: The **Modem** Interface was initialized successfully
- **"Modem error at initialization!"**: Driver could not initialize modem's line. Check Driver's log file for more details
- **"Modem error at dial!"**: Driver could not start or accept a call
- **"Connecting..."**: Driver started a call successfully, and is currently processing that call
- **"Ringing..."**: Indicates that the modem is receiving an external call, but it did not accepted it yet
- **"Connected!"**: Driver connected successfully, that is, completed or accepted an external call
- **"Disconnecting..."**: Driver is turning the current call off
- **"Disconnected OK!"**: Driver turned the current call off
- **"Error: no dial tone!"**: Driver aborted a call because the available line signal was not detected
- **"Error: busy!"**: Driver aborted a call because the line was busy
- **"Error: no answer!"**: Driver aborted a call because no answer was received from the other modem
- **"Error: unknown!"**: Current call was aborted because of an unknown error

IO.TAPI.PhoneNumber

Type of Tag	I/O Tag
Type of Access	Reading or Writing
N1 Parameter	-1 (minus one)
N2 Parameter	0 (zero)
N3 Parameter	3 (three)
N4 Parameter	0 (zero)
String Configuration	IO.TAPI.PhoneNumber

This Tag is a **String** that reads or changes the telephone number used by the **IO.TAPI.Dial** Tag. When changing this Tag, the new value is used only on the next **Dial** command.

Properties

These properties control the configuration of a **Modem** (TAPI) Interface.

IO.TAPI.AcceptIncoming

9 Configure to False if a modem cannot accept external calls, that is, the Driver behaves as a master, and configure to True to enable receiving calls, that is, the Driver behaves as a slave.

IO.TAPI.ModemID

9 This is the identification number of a modem. This ID is created by Windows and used internally to identify a modem on a list of devices installed on a computer. This ID may not remain valid if a modem is reinstalled or an application is executed on another computer.

NOTE

It is advisable to configure this property as 0 (zero), indicating that a Driver must use the first available modem.

IO.TAPI.PhoneNumber

A A telephone number used by **Dial** commands, such as "0w01234566", where the "w" character forces a modem to wait for a call sign.

RAS Interface Configuration

This section contains information about the configuration of **I/O Tags** and **Properties** of a **RAS** Interface.

I/O Tags

Tags of a RAS Interface (N2/B2 = 5)

Currently, there are no Tags defined specifically to manage a **RAS** Interface at run time.

Properties

These properties control the configuration of a **RAS** Interface.

NOTE

A **RAS** Interface uses the **Ethernet** Interface, which for this reason must be also configured.

IO.RAS.ATCommand

A An **AT** command to send through a socket to force a RAS device to perform a call using the current RAS channel, such as "ATDT6265545".

IO.RAS.CommandTimeoutSec

9 Time to wait for a **CONNECT** message in response to an **AT** command, in seconds.

Serial Interface Configuration

This section contains information about the configuration of **I/O Tags** and **Properties** of a **Serial** Interface.

I/O Tags

Tags of a Serial Interface (N2/B2 = 2)

Currently, there are no Tags defined specifically to manage a **Serial** Interface at run time.

Properties

These properties control the configuration of a **Serial** Interface.

IO.Serial.Baudrate

9 Specifies a baud rate of a serial port, such as 9600.

IO.Serial.CTSTimeoutMs

9 Time to wait for a **CTS** signal, in milliseconds. After turning the **RTS** signal on, a timer is started to wait for a **CTS** signal. If this timer expires, the Driver aborts sending bytes through the serial port. Available only when the **IO.Serial.RTS** property is configured as **Toggle** and the **IO.Serial.WaitCTS** property is configured to True.

IO.Serial.DataBits

9 Specifies the number of data bits to configure a serial port. Possible values are **5**: Five data bits, **6**: Six data bits, **7**: Seven data bits, or **8**: Eight data bits.

IO.Serial.DelayAfterMs

9 Number of milliseconds to delay after the last byte is sent through a serial port, but before turning the **RTS** signal off. Available only when the **IO.Serial.RTS** property is configured to **Toggle** and the **IO.Serial.WaitCTS** property is configured to False.

IO.Serial.DelayBeforeMs

9 Number of milliseconds to delay after turning the **RTS** signal on, but before data is sent. Available only when the **IO.Serial.RTS** property is configured to **Toggle** and the **IO.Serial.WaitCTS** property is configured to False.

IO.Serial.DTR

A Indicates how a Driver deals with the **DTR** signal. Possible values are **OFF**: **DTR** signal is always turned off or **ON**: **DTR** signal is always turned on.

IO.Serial.InterbyteDelayUs

9 Delay time, in milliseconds (1/1000000 of a second), for each two bytes sent through a **Serial** Interface.

IO.Serial.InterframeDelayMs

9 Delay time, in milliseconds, before sending a packet after the last packet sent or received.

IO.Serial.Parity

A Specifies a parity for the configuration of a serial port. Possible values are **E or Even**: Even parity, **N or None**: No parity, **O or Odd**: Odd parity, **M or Mark**: Mark parity, or **S or Space**: Space parity.

IO.Serial.Port

9 Number of the local serial port. Possible values are **1**: Uses the COM1 port, **2**: Uses the COM2 port, **3**: Uses the COM3 port, or **n**: Uses the COMn port.

IO.Serial.RTS

A Indicates how a Driver deals with the **RTS** signal. Possible values are **OFF**: **RTS** signal always off, **ON**: **RTS** signal always on, or **Toggle**: Turns the **RTS** signal on when transmitting data and turns the **RTS** signal off when not transmitting data.

IO.Serial.StopBits

9 Specifies the number of stop bits for the configuration of a serial port. Possible values are **1**: One stop bit, **2**: One and a half stop bit, or **3**: Two stop bits.

IO.Serial.SuppressEcho

9 Use a value different from 0 (zero) to enable suppressing the echo or 0 (zero) to disable it.

IO.Serial.WaitCTS

☑ Configure to True to force a Driver to wait for the **CTS** signal before sending bytes when the **RTS** signal is turned on. Available only when the **IO.Serial.RTS** property is configured to **Toggle**.

Driver Revision History

VERSION	DATE	AUTHOR	COMMENTS
4.0.10	03/21/2023	C. Mello	Added an automatic reconnection for Modbus RTU/ASC serial protocol, when communicating over Ethernet TCP/IP (Case 16216).
4.0.9	12/07/2022	C. Mello	- Adjustments to recover valid Modbus RTU packets in TCP encapsulation containing extra bytes (Case 33398). - Fixed an inconsistency in values with a String data type when read via Superblocks (Case 33313).
4.0.8	09/27/2022	C. Mello	Added a 64-bit Integer data type, converted to String

VERSION	DATE	AUTHOR	COMMENTS
			format (Case 33213).
4.0.7	12/01/2021	C. Mello	Added support for Superblocks in String-configured Tags (Case 20900).
4.0.6	04/07/2021	C. Mello	Added a quality handling for non-normalized Float values (Case 30742).
4.0.5	11/05/2020	C. Mello	Added support for addressing slaves with ID equal to 0 (zero) in Modbus TCP mode (Case 29886).
4.0.4	06/22/2020	C. Mello	Added a 52-bit Integer data type via conversion to a 64-bit Double data type (Case 21840).
4.0.3	03/17/2020	C. Mello	Fixed a problem in offline configuration for Strings <i>ModiconModbus.ConfigFile</i> and <i>ModiconModbus.UserTypesConfigFile</i> (Case 28566).
4.0.2	08/05/2019	M. Ludwig	Driver ported to Visual Studio 2017 (Case 27087).
4.0.1	11/13/2018	C. Mello	- Fixed a problem with configuration 1000, which links a Tag with the Default Slave Address on properties window (Case 20713). - Implemented a return for BYTE data types for values of Coil registers (Case 20927). - Fixed a problem with writing String data types with an odd size and the Swap bytes / words / dwords option enabled (Case 23694). - Consistency adjustments when editing structures of user-defined data types (Case 20454).
3.1.36	03/17/2017	C. Mello	Added support for integers with a magnitude signal (Case 22091).
3.1.31	07/10/2016	F. Englert	Fixed a GPF that may occur when reading a Driver's error history ($B2 = 9998$) if

VERSION	DATE	AUTHOR	COMMENTS
			any communication error occurred at the same time (<i>Case 21637</i>).
3.1.30	04/01/2016	C. Mello	Fixed a lack of values in Tags configured by Strings when the Superblock service is activated (<i>Case 20754</i>).
3.1.28	01/28/2016	A. Quites	Implemented a new Tag Browser with templates for String-configured Tags (<i>Case 20460</i>).
3.1.27	01/27/2016	A. Quites	- Implemented an option for addressing Tags by Strings in the Item and Device parameters (<i>Case 19119</i>). - Fixed an error when reading real-time Tags of events of a PAC RX7 device by callbacks (<i>Case 20374</i>).
3.1.26	01/20/2016	A. Quites	Fixed an error in which the Elipse SOE function did not return values reported by events for native, non-structured, data types (<i>Case 20364</i>).
3.1.25	01/19/2016	A. Quites	Implemented a configuration by Strings for user-defined data types (<i>Case 19807</i>).
3.1.24	01/18/2016	A. Quites	- Added a check to prevent the definition of user-defined data types with names conflicting with this Driver's native data types (<i>Case 19816</i>). - Fixed an error when reading UTC32 data types in Block, which returned wrong values after the second Block Element (<i>Case 19819</i>).
3.1.23	01/14/2016	A. Quites	Added a validation to avoid user errors when setting date and time data types to elements of a structure of native data types (<i>Case 20415</i>).
3.1.15	12/22/2015	A. Quites	Fixed a block writing of Block Tags with Double data types (<i>Case 20053</i>).

VERSION	DATE	AUTHOR	COMMENTS
3.1.13	12/02/2015	A. Quites	- Fixed an error when exporting INI files in a format compatible with version 1.0 of this Driver, with data types specified by numbers and not by Strings, as it is occurring since version 2.8 (<i>Case 20203</i>). - Fixed an error in byte order when writing BCD data types with eight digits and a size of four bytes (<i>Case 20204</i>).
3.1.9	10/01/2015	A. Quites	Enabled the storage of last exceptions for I/O Tags configured by Strings (<i>Case 19808</i>).
3.1.8	09/28/2016	A. Quites	Fixed and error when reading and writing BCD data types with eight digits and four bytes larger than 9999999, which finished the process (<i>Case 19733</i>).
3.0.11	05/29/2015	A. Quites	- Driver changed to avoid a possible strange behavior when reconnecting after a time-out in callback Tags (SOE), with the Reconnect after Timeout (Ethernet only) option enabled. Two simultaneous connections followed by two disconnections may occur, for example (<i>Case 14775</i>). - Fixed an error when recording operations on this Driver's configuration window, on the Operations tab, which could eventually, when removing operations at the end of the list, lead these operations to return to the list when the window is closed by clicking OK and reopened right after (<i>Case 14874</i>). - Fixed a problem that could generate a loss of information about the last exception in rare cases when valid frames are received with a correct CRC,

VERSION	DATE	AUTHOR	COMMENTS
			but they are not meant to the current request. It may occur when using Modbus RTU in Ethernet TCP/IP layer on slow networks (Case 15314). - Fixed a leak of handles of a download thread of mass memory of GE PAC RX7 (Case 16404). - Fixed an error in which this Driver could ignore the last characters of Strings read from a device when one of the swap options were configured in the operation (Case 16744). - Fixed an error when reading operations with the Use bit mask option enabled when used with this Driver's EnableReadGrouping property enabled (Case 18340). - Fixed a validation error in this Driver's special Tags (Case 16433).
3.0.8	07/31/2014	A. Quites	Fixed an error that could generate GPFs or unexpected behavior when executing multiple instances of this Driver on a single I/O Server, specially if these instances have different configurations on the Modbus and Operations tab of this Driver's configuration window (Case 14856).
3.0.6	06/12/2014	A. Quites	Driver ported to IOKit 2.0 (Case 13891).
2.8.17	10/19/2012	A. Quites	- Added user-defined data types or structures as part of the implementation of a Generic SOE feature (Case 12038). - Implemented a SOE reading and commands for clock syncing in Schneider Electric series SEPAM 20, 40, and 80 relays (Case 12106).

VERSION	DATE	AUTHOR	COMMENTS
			- The Reverse Frame option was removed from this Driver's configuration window, for obsolescence. This options is still supported in legacy applications as an offline configuration only (Case 12443). - Added a reading by callbacks for SOE's reading Tags (Case 12464). - Added a reconnection option in case of a time-out when receiving frames in the Ethernet physical layer (Case 12537). - The Swap Address Delay option was removed from this Driver's configuration window. This option was replaced by the Inter-frame delay option of IOKit library, but it is still available as an offline configuration, for compatibility with legacy applications (Case 13285). - Fixed an error when writing float_GE values (Case 12298). - Fixed an error when loading a configuration file on Windows CE ARM HPC2000 (Case 12352). - Fixed an error when reading odd-sized Strings could lack the last character (Case 12466).
2.7.1	06/30/2010	A. Quites	Implemented a feature to turn this Driver immune to noise before receiving a frame in RTU mode (Case 11394).
2.6.1	11/26/2009	A. Quites	- Driver ported to Windows CE (Case 10914). - Driver reports a false error in its log when reading a register address in zero (Case 10654). - Optimized the reading of bits using group readings

VERSION	DATE	AUTHOR	COMMENTS
			or Superblocks (<i>Case 10971</i>).
		C. Mello	The Wait Silence option does not work for all errors (<i>Case 10850</i>).
2.5.1	06/30/2009	A. Quites	- Updated the maximum PDU size for this Driver (<i>Case 10274</i>). - Fixed an error when reading the Modbus 20 function, Read File Record (<i>Case 10312</i>). - Fixed an error in the Special Tag to return the last exception. This Tag may not report some exceptions (<i>Case 10337</i>). GE SOE events with the current day returned the wrong year (<i>Case 10382</i>). - Swap address was not working (<i>Case 10425</i>).
		M. Ludwig	Created a new Tag to receive a list of events from specific GE SOE points (<i>Case 10370</i>).
2.4.1	02/17/2009	A. Quites	- Added reading and writing functions by default (<i>Case 9185</i>). - Driver with GE events, last-event reading by polling (<i>Case 10178</i>). - Error when reading blocks of bits with grouped readings or Superblocks enabled (<i>Case 10100</i>).
2.3.1	09/02/2008	A. Quites	- Implemented the CMS Extended Device Addressing feature (<i>Case 8665</i>). - Implemented a configuration for the maximum size of Superblocks (<i>Case 9154</i>). - This Driver accepts sending a 32-bit DWord using the Modbus function 6 (<i>Case 8663</i>). - Revision on non-standard or rarely used functions 07,

VERSION	DATE	AUTHOR	COMMENTS
			20, 21, 65 01, 65 02, and 65 03 (Case 8730). - The Swap byte option does not work correctly with Superblocks (Case 9220). - Configuration in offline mode of the <i>ModiconModbus.ModbusMode</i> parameter (Case 9831).
2.2.1	05/11/2007	A. Quites	Using an Int16 data type to read blocks only returns the first element (Case 8243).
2.1.1	01/23/2007	A. Quites	- Implemented Superblocks (Case 6185). - Master Modicon Modbus does not test correctly the consistency of <i>N2/B2</i> parameter (Case 7714). - Offset error when reading blocks with BCD data type and the size of the data type equal to four (Case 7728). - Reading blocks of Strings was not working correctly (Case 7804).
2.0.1	09/14/2005	A. Quites	Driver ported to IOKit (Case 2050).
1.3.1	12/19/2006	C. Mello	- Driver communicating with more than one IP address changes the values read (Case 7191). - Possibility of defining a local TCP/IP port (Case 7109). - Fixed an offset error when reading blocks with BCD data type and the size of the data type equal to four (Case 7729). - Driver was not correctly testing consistency of <i>N2/B2</i> parameters (Case 7735).
1.2.1	12/15/2005	C. Mello	- Adjustments to communicate with ICP Flow Correctors (Case 4979). - Fixed a leak on handles if a ping command fails (Case 6497).
1.1.1	12/22/2004	C. Mello	Added retries to this Driver (Case 3365).

VERSION	DATE	AUTHOR	COMMENTS
			• Added the Broadcast function (<i>Case 4045</i>). • Added the HALT option to this Driver (<i>Case 4429</i>). • Fixed an error when reading and writing Strings (<i>Case 4386</i>). • Fixed all configurations incompatible with previous versions (<i>Case 4431</i>).

Headquarters

**Rua Mostardeiro, 322/Cj. 902, 1001 e
1002**

90510-002 — Porto Alegre — RS

Phone: (+55 51) 3346-4699

Fax: (+55 51) 3222-6226

E-mail: elipse-rs@elipse.com.br

Branch in Taiwan

9F., No.12, Beiping 2nd St., Sanmin Dist.

807 — Kaohsiung City — Taiwan

Phone: (+886 7) 323-8468

Fax: (+886 7) 323-9656

E-mail: evan@elipse.com.br

Check our website for information about a representative in your country.

www.elipse.com.br

kb.elipse.com.br

forum.elipse.com.br

www.youtube.com/elipsesoftware

elipse@elipse.com.br



Gartner, Cool Vendors in Brazil 2014, April 2014.

Gartner does not endorse any vendor, product or service depicted in its research publications, and does not advise technology users to select only those vendors with the highest ratings. Gartner research publications consist of the opinions of Gartner's research organization and should not be construed as statements of fact. Gartner disclaims all warranties, expressed or implied, with respect to this research, including any warranties of merchantability of fitness for a particular purpose.

Microsoft Partner

Gold Independent Software Vendor (ISV)